

LIST OF EXPERIMENTS

EXPERIMENT 1: Installation, Enviornment setup and starting with C language

1. Write a C program to print "helllo world".

Algorithm

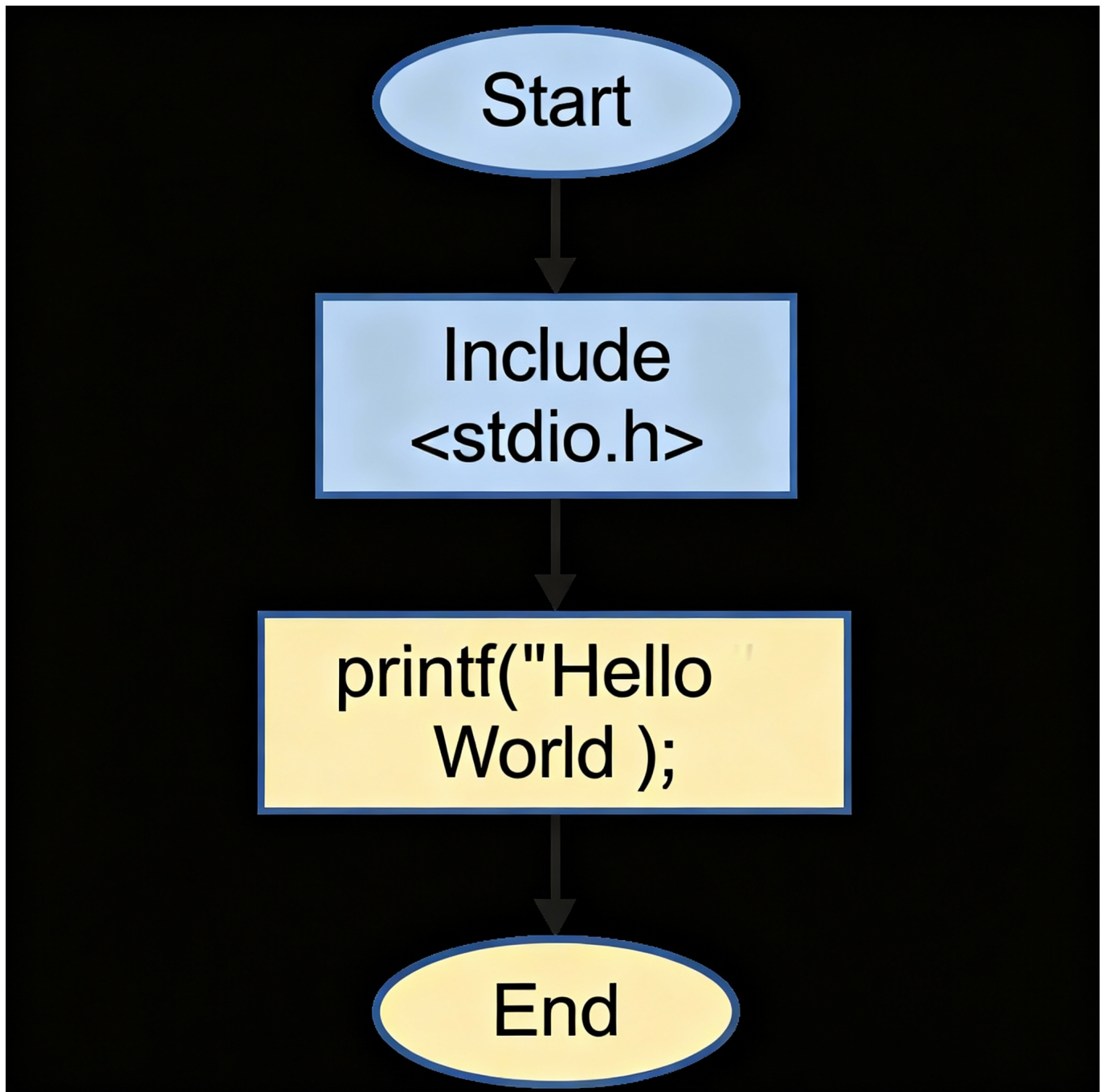
Step 1: Start Step 2: Include the standard input/output header file (stdio.h) Step 3: Define the main() function
Step 4: Inside main(), use printf() to display "Hello, World!" Step 5: Return 0 to indicate successful program execution Step 6: Stop

CODE:

```
#include <stdio.h>
```

```
int main() { printf("Hello, World!\n"); return 0; }
```

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c> cd C:\Users\dell\Documents\experiments in c\ ; if ($?) { gcc exp101.c -o exp101 } ; if ($?) { .\exp101 }  
Hello, World!  
PS C:\Users\dell\Documents\experiments in c>
```

2. Write a C program to print address in multiple lines

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h> for input and output functions

Step 3: Define the main() function

Step 4: Use printf() statements to print each line of the address

- Print the Name
- Print the Street
- Print the City and State

- Print the ZIP code (or Country)

Step 5: Return 0 to indicate successful execution

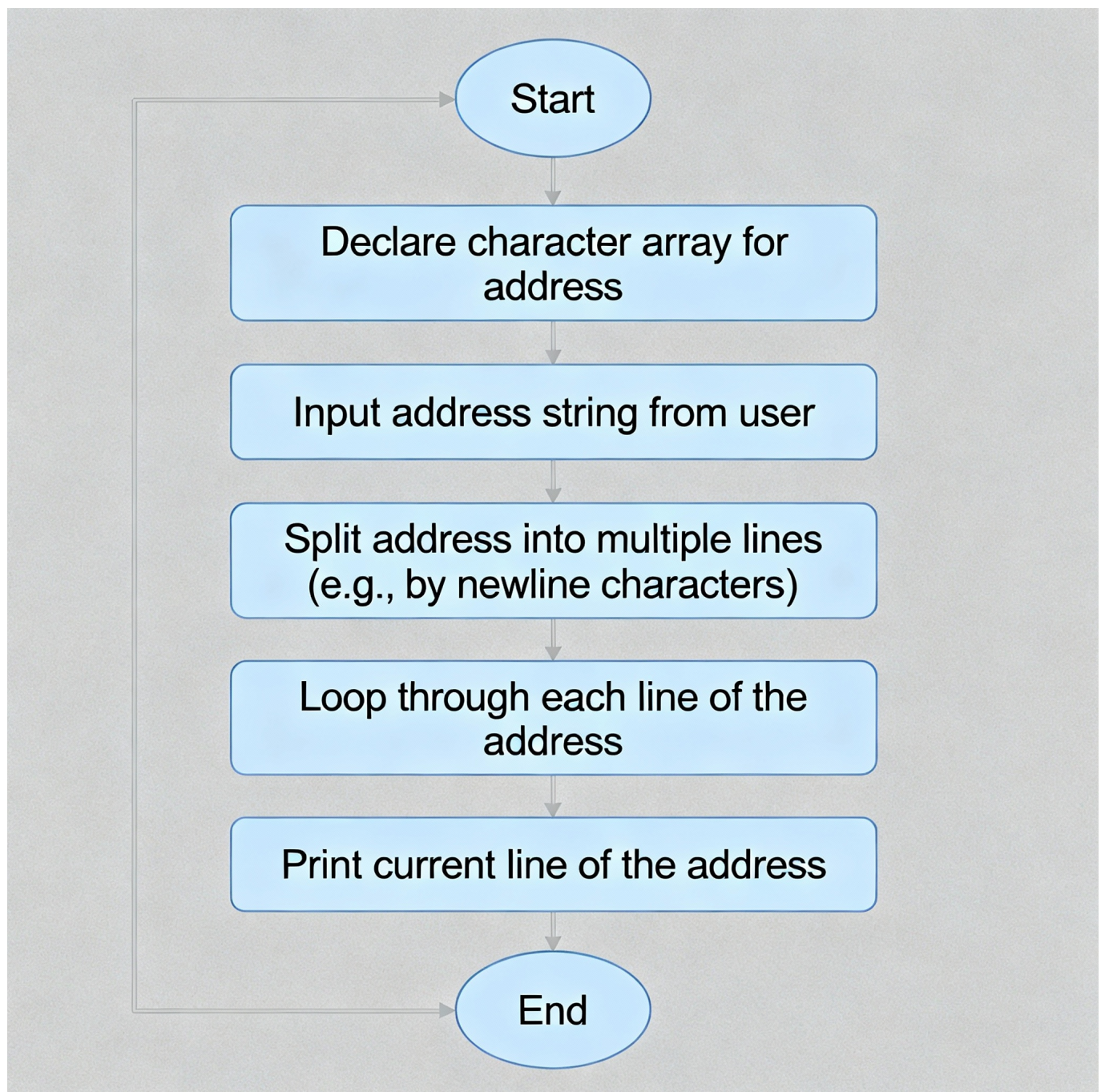
Step 6: Stop

CODE:

```
#include <stdio.h>
```

```
int main() { printf("Aditya Mishra\n"); printf("15A,\n"); printf("Kunraghat\n"); printf("Gorakhpur-273008\n");  
return 0; }
```

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>  
cd "c:\Users\dell\Documents\experiments in c\" ; if ($?) { gcc exp1Q2.c -o exp1Q2 } ; if ($?) { .\exp1Q2 }  
Aditya Mishra  
15A,  
Kunraghat  
Gorakhpur-273008  
PS C:\Users\dell\Documents\experiments in c>
```

3:Write a C Program that prompts the user to enter their name and age.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare a character array 'name' to store the user's name

Step 4: Declare an integer variable 'age' to store the user's age

Step 5: Display the message "Enter your name:"

Step 6: Read the user's name using scanf()

Step 7: Display the message "Enter your age:"

Step 8: Read the user's age using scanf()

Step 9: Display the output message "Hello ! You are years old."

Step 10: Return 0 to indicate successful program execution

Step 11: Stop

CODE:

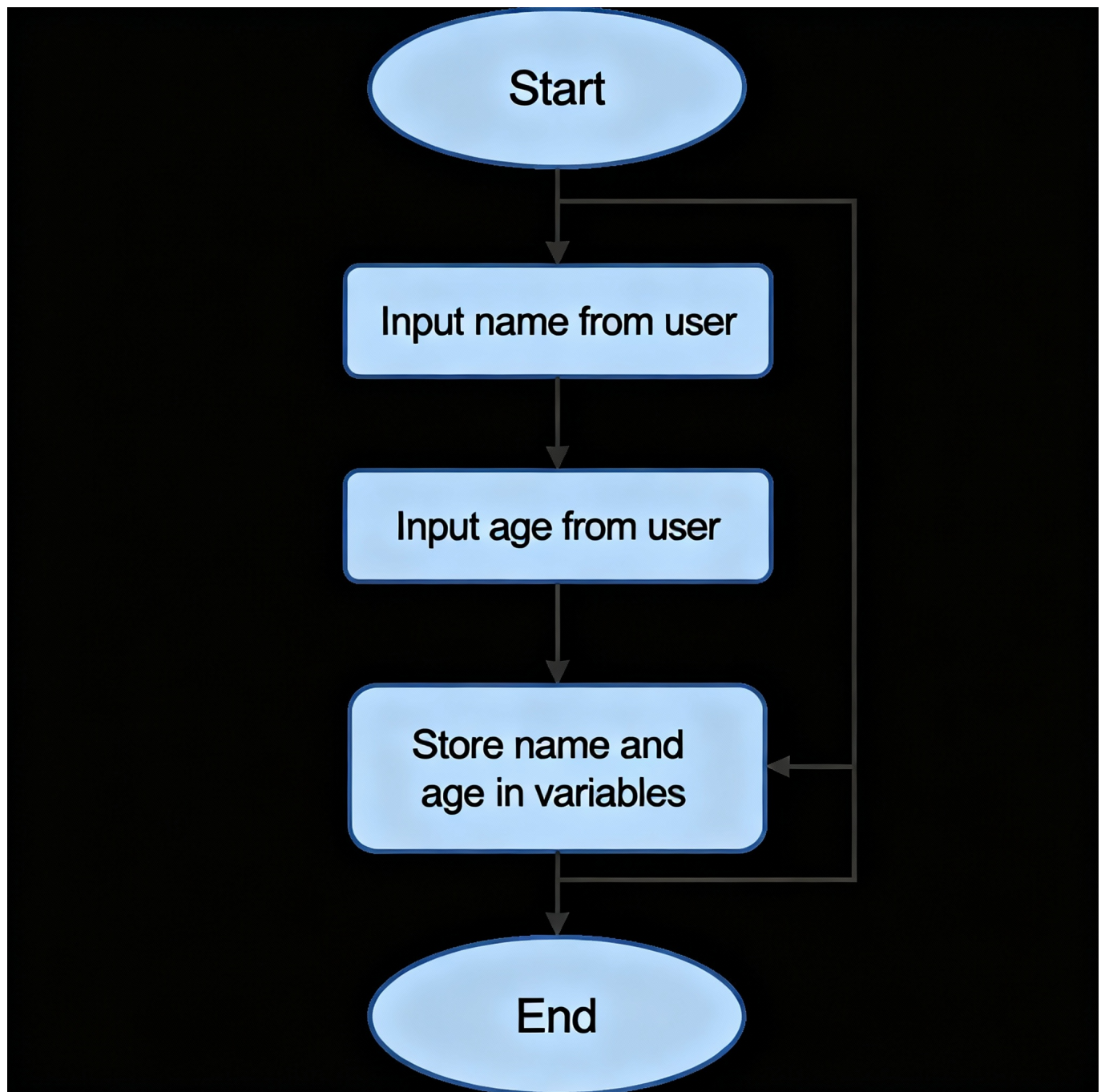
```
#include <stdio.h>
```

```
int main() { char name[50]; int age;
```

```
    printf("Enter your name: ");  
    scanf("%s", name);  
  
    printf("Enter your age: ");  
    scanf("%d", &age);  
  
    printf("\nHello %s! You are %d years old.\n", name, age);  
  
    return 0;
```


}

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\" ; if ($?) { gcc exp1Q3.c -o exp1Q3 } ; if ($?) { .\exp1Q3 }
Enter your name: aditya
Enter your age: 19

Hello aditya! You are 19 years old.
PS C:\Users\dell\Documents\experiments in c> |
```

4: Write a C program to add two numbers ,take numbers from users.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare three integer variables: num1, num2, and sum

Step 4: Display the message "Enter first number:"

Step 5: Read the first number using scanf()

Step 6: Display the message "Enter second number:"

Step 7: Read the second number using scanf()

Step 8: Calculate the sum as $sum = num1 + num2$

Step 9: Display the result using printf()

Step 10: Return 0 to indicate successful execution

Step 11:

Code:

```
#include <stdio.h>
```

```
int main() { int num1, num2, sum;
```

```
    printf("Enter first number: ");
    scanf("%d", &num1);

    printf("Enter second number: ");
    scanf("%d", &num2);

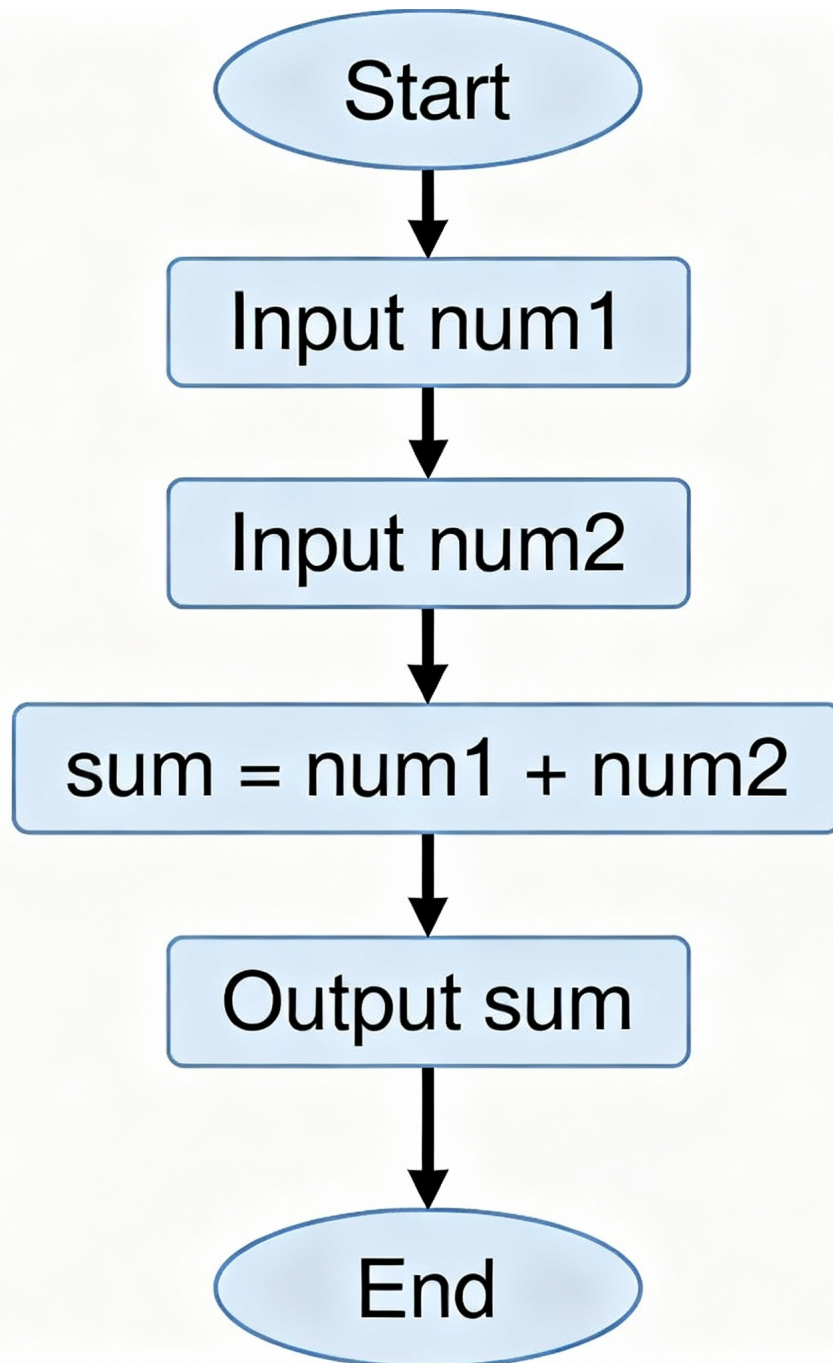
    sum = num1 + num2;

    printf("The sum of %d and %d is %d.\n", num1, num2, sum);

    return 0;
```

```
}
```

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\" ; if ($?) { gcc exp1Q4.c -o exp1Q4 } ; if ($?) { .\exp1Q4 }
Enter first number: 12
Enter second number: 13
The sum of 12 and 13 is 25.
PS C:\Users\dell\Documents\experiments in c> |
```

EXPERIMENT 2: OPERATORS

1: Write a C program to Calculate the area and perimeter of a Rectangle based on its length and width.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare three variables: length, width, area, and perimeter

Step 4: Display the message "Enter the length of the rectangle:"

Step 5: Read the length using scanf()

Step 6: Display the message "Enter the width of the rectangle:"

Step 7: Read the width using scanf()

Step 8: Calculate the area using the formula: $\text{area} = \text{length} * \text{width}$

Step 9: Calculate the perimeter using the formula: $\text{perimeter} = 2 * (\text{length} + \text{width})$

Step 10: Display the area and perimeter using printf()

Step 11: Return 0 to indicate successful execution

Step 12: Stop

Code:

```
#include <stdio.h>
```

```
int main() { float length, width, area, perimeter;
```

```
    printf("Enter the length of the rectangle: ");
    scanf("%f", &length);

    printf("Enter the width of the rectangle: ");
    scanf("%f", &width);

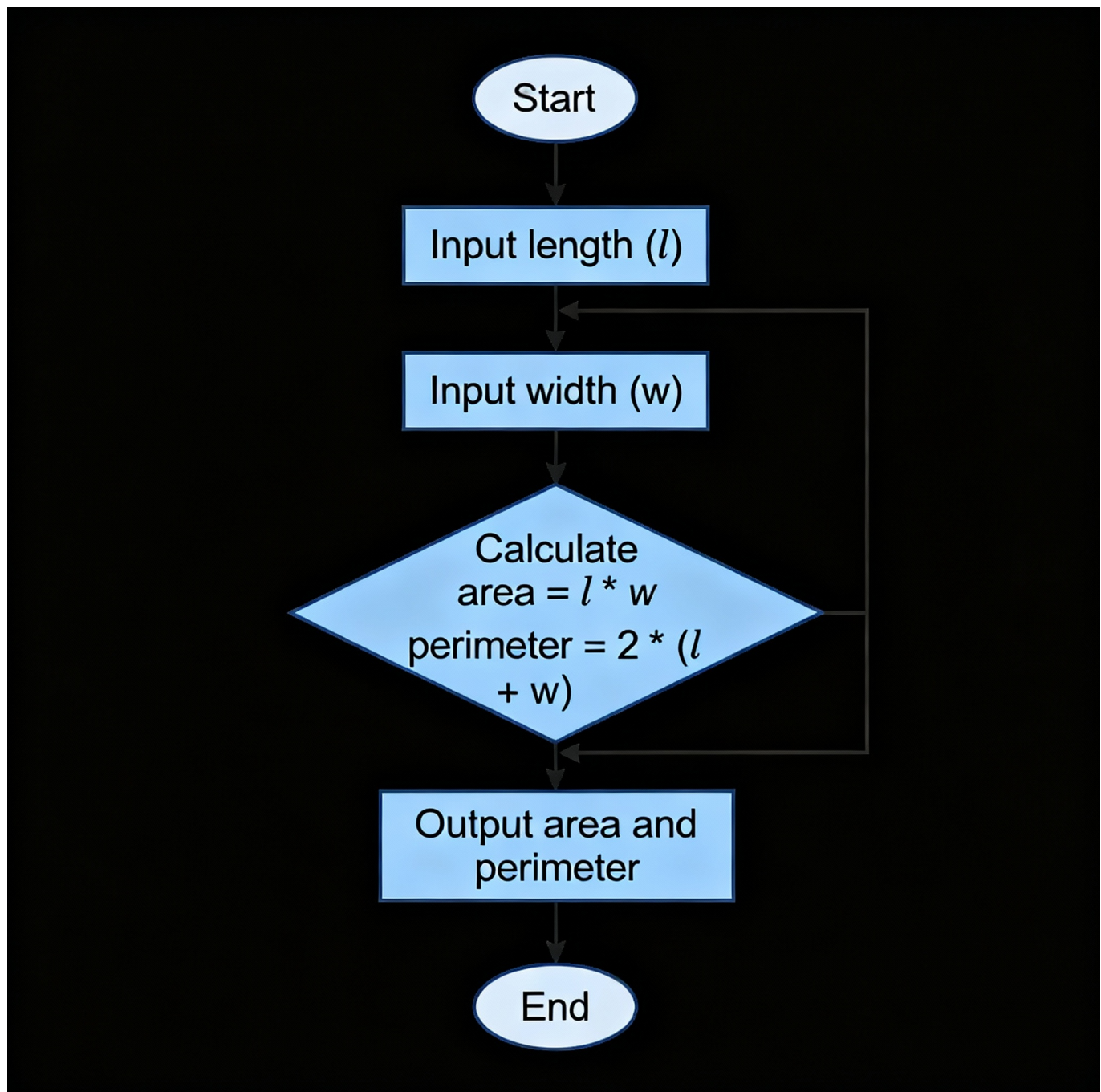
    area = length * width;
    perimeter = 2 * (length + width);

    printf("\nArea of the rectangle = %.2f\n", area);
    printf("Perimeter of the rectangle = %.2f\n", perimeter);

    return 0;
```

```
}
```

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp2\" ; if ($?) { gcc exp2Q1.c -o exp2Q1 } ; if ($?) { .\exp2Q1 }
Enter the length of the rectangle: 12
Enter the width of the rectangle: 13

Area of the rectangle = 156.00
Perimeter of the rectangle = 50.00
PS C:\Users\dell\Documents\experiments in c\exp2> |
```

2: Write a C program to convert temperature from Celsius to Fahrenheit using the formula $F = (C * 9/5) + 32$

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare two variables: celsius and fahrenheit

Step 4: Display the message "Enter temperature in Celsius:"

Step 5: Read the Celsius temperature using scanf()

Step 6: Calculate Fahrenheit using the formula: $fahrenheit = (celsius * 9 / 5) + 32$

Step 7: Display the Fahrenheit temperature using printf()

Step 8: Return 0 to indicate successful execution

Step 9: Stop

Code:

```
#include <stdio.h>
```

```
int main() { float celsius, fahrenheit;
```

```
    printf("Enter temperature in Celsius: ");
    scanf("%f", &celsius);

    fahrenheit = (celsius * 9 / 5) + 32;

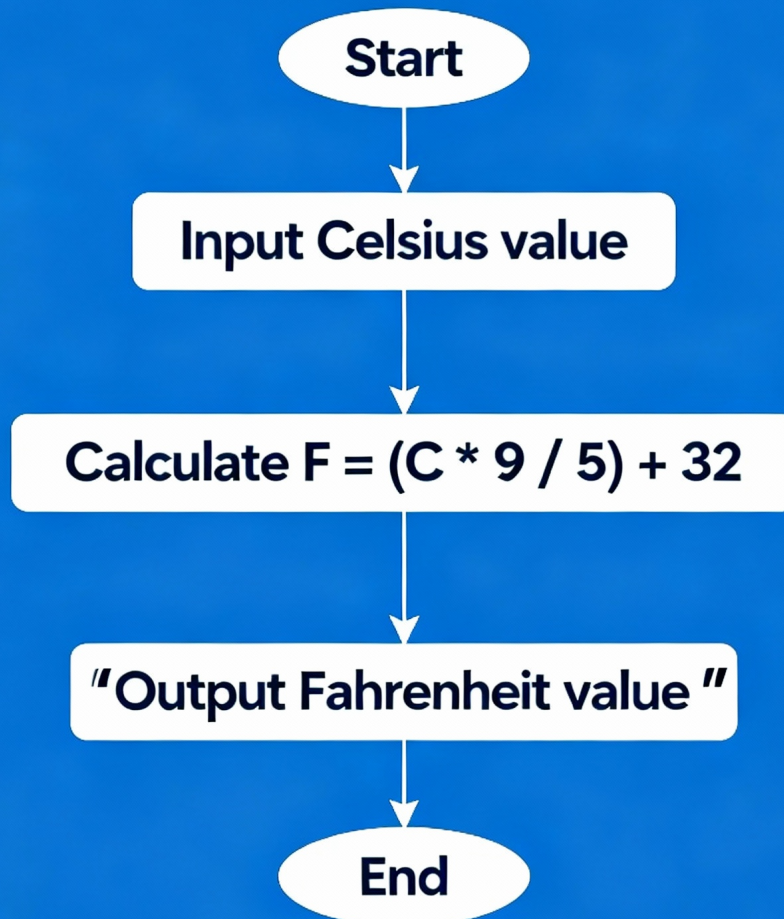
    printf("Temperature in Fahrenheit = %.2f\n", fahrenheit);

    return 0;
```

```
}
```

Flowchart

Celsius to Fahrenheit Conversion Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp2\" ; if ($?) { gcc exp2Q2.c -o exp2Q2 } ; if ($?) { .\exp2Q2 }
Enter temperature in Celsius: 79
Temperature in Fahrenheit = 174.20
PS C:\Users\dell\Documents\experiments in c\exp2> |
```

EXPERIMENT 3.1: Conditional Statements

1. WAP to take check if the triangle is valid or not.if the validity is established ,do check if the tringle is isosceles,equilateral,right angle or scalene. Take sides of the triangle as input from a user.

Algorithm

Start

Input three sides: a, b, c

Check triangle validity using the condition:

If $(a + b > c)$ AND $(a + c > b)$ AND $(b + c > a) \rightarrow$ Triangle is valid Else $\rightarrow$ Triangle is not valid and stop.

If valid:

If $a == b == c \rightarrow$ Triangle is Equilateral

Else If $a == b$ OR $b == c$ OR $a == c \rightarrow$ Triangle is Isosceles

Else $\rightarrow$ Triangle is Scalene

Check right-angle condition using Pythagoras theorem:

If $a^2 + b^2 == c^2$ OR $a^2 + c^2 == b^2$ OR $b^2 + c^2 == a^2 \rightarrow$ Triangle is Right-angled

Display result

Stop

Code:

```
#include <stdio.h>
```

```
int main() { float a, b, c;
```

```
    printf("Enter the three sides of the triangle: ");
    scanf("%f %f %f", &a, &b, &c);
```

```
    if (a + b > c && a + c > b && b + c > a) {
        printf("Triangle is valid.\n");
```

```
        if (a == b && b == c)
            printf("It is an Equilateral triangle.\n");
```

```
        else if (a == b || b == c || a == c)
            printf("It is an Isosceles triangle.\n");
```

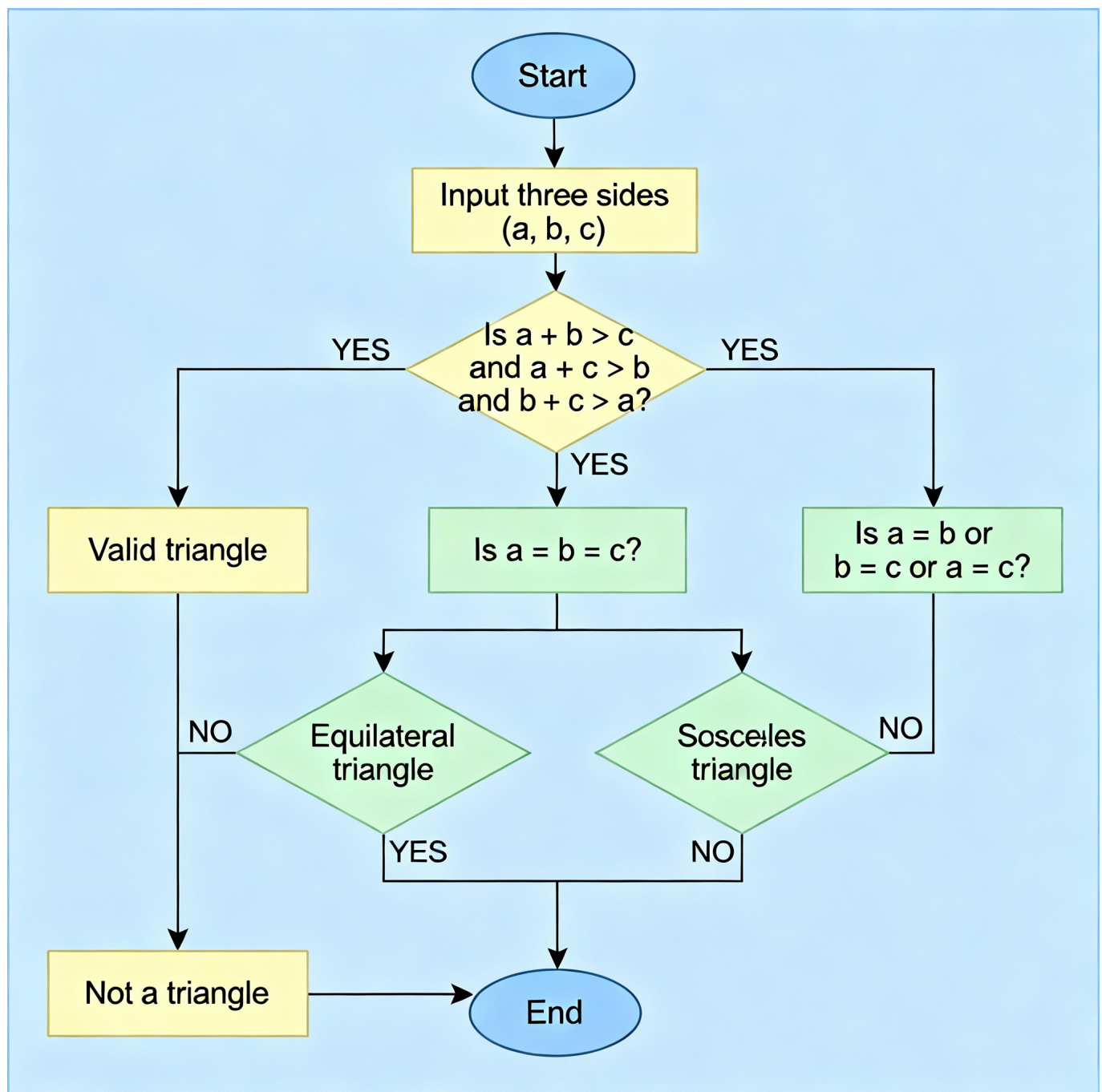
```
        else if ((a*a + b*b == c*c) ||
                 (a*a + c*c == b*b) ||
                 (b*b + c*c == a*a))
            printf("It is a Right-angled triangle.\n");
```

```
    else
```

```
        printf("It is a Scalene triangle.\n");
    }
    else {
        printf("Triangle is not valid.\n");
    }

    return 0;
}
```

Flowchart



Output


```

PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\" ; if ($?) { gcc exp3Q1.c -o exp3Q1 } ; if ($?) { .\exp3Q1 }
Enter the three sides of the triangle: 2.0 3.0 2.0
Triangle is valid.
It is an Isosceles triangle.
PS C:\Users\dell\Documents\experiments in c\exp3_1>

```

2:WAP to compute the BMI index of the person and print the BMI values as per the following ranges. You can use the following formula to compute BMI=weight(kgs)/Height(Mts)*Height(Mts).

	BMI
Starvation	<15
Anorexic	15.1 to 17.5
Underweight	17.6 to 18.5
Ideal	18.6 to 24.9
Overweight	25 to 25.9
Obese	30 to 39.9

Morbidity Obese	40.0 above
-----------------	------------

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare variables: weight, height, and bmi

Step 4: Prompt the user to enter weight (in kilograms)

Step 5: Read weight using scanf()

Step 6: Prompt the user to enter height (in meters)

Step 7: Read height using scanf()

Step 8: Compute BMI using the formula:

$$\text{bmi} = \text{weight} / (\text{height} * \text{height})$$

Step 9: Display the BMI value

Step 10: Determine the BMI category:

- If $\text{bmi} < 15 \rightarrow$ "Starvation"
- Else if $\text{bmi} \geq 15.1$ and $\text{bmi} \leq 17.5 \rightarrow$ "Anorexic"
- Else if $\text{bmi} \geq 17.6$ and $\text{bmi} \leq 18.5 \rightarrow$ "Underweight"
- Else if $\text{bmi} \geq 18.6$ and $\text{bmi} \leq 24.9 \rightarrow$ "Ideal"
- Else if $\text{bmi} \geq 25$ and $\text{bmi} \leq 25.9 \rightarrow$ "Overweight"
- Else if $\text{bmi} \geq 30$ and $\text{bmi} \leq 39.9 \rightarrow$ "Obese"
- Else if $\text{bmi} \geq 40 \rightarrow$ "Morbidly Obese"

Step 11: Stop

Code:

```
#include <stdio.h>
```

```
int main() { float weight, height, bmi;
```

```
    printf("Enter your weight (in kg): ");
    scanf("%f", &weight);

    printf("Enter your height (in meters): ");
    scanf("%f", &height);

    bmi = weight / (height * height);

    printf("\nYour BMI is: %.2f\n", bmi);

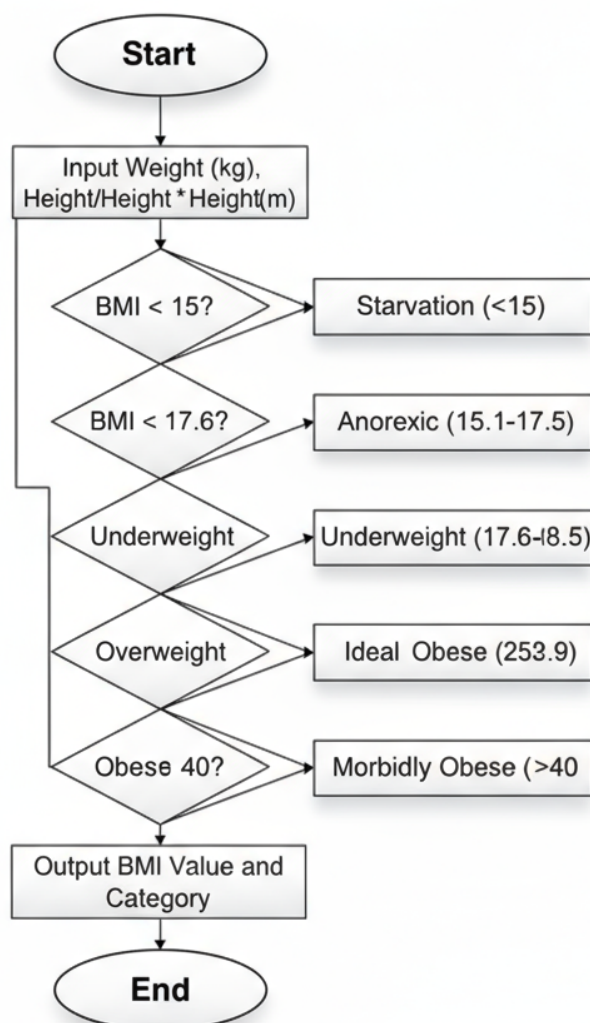
    if (bmi < 15)
        printf("Category: Starvation\n");
    else if (bmi >= 15.1 && bmi <= 17.5)
        printf("Category: Anorexic\n");
    else if (bmi >= 17.6 && bmi <= 18.5)
        printf("Category: Underweight\n");
    else if (bmi >= 18.6 && bmi <= 24.9)
        printf("Category: Ideal\n");
    else if (bmi >= 25 && bmi <= 25.9)
        printf("Category: Overweight\n");
    else if (bmi >= 30 && bmi <= 39.9)
```

```
    printf("Category: Obese\n");  
else if (bmi >= 40)  
    printf("Category: Morbidly Obese\n");  
else  
    printf("Invalid input.\n");  
  
return 0;
```

}

Flowchart

BMI Calculation Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\" ; if ($?) { gcc exp3Q2.c -o exp3Q2 } ; if ($?) { .\exp3Q2 }
Enter your weight (in kg): 60
Enter your height (in meters): 159

Your BMI is: 0.00
Category: Starvation
PS C:\Users\dell\Documents\experiments in c\exp3_1> |
```

3: WAP to check if three points (x1,y1),(x2,y2)and(x3,y3)are collinear or not.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare six variables: x1, y1, x2, y2, x3, y3

Step 4: Prompt the user to enter coordinates of the three points

Step 5: Read values of x1, y1, x2, y2, x3, y3 using scanf()

Step 6: Calculate the value: $\text{area} = x1*(y2 - y3) + x2*(y3 - y1) + x3*(y1 - y2)$

Step 7: If $\text{area} == 0$ Display "The points

Code:

```
#include <stdio.h>
```

```
int main() { float x1, y1, x2, y2, x3, y3, area;
```

```
    printf("Enter coordinates of first point (x1, y1): ");
    scanf("%f %f", &x1, &y1);

    printf("Enter coordinates of second point (x2, y2): ");
    scanf("%f %f", &x2, &y2);

    printf("Enter coordinates of third point (x3, y3): ");
    scanf("%f %f", &x3, &y3);

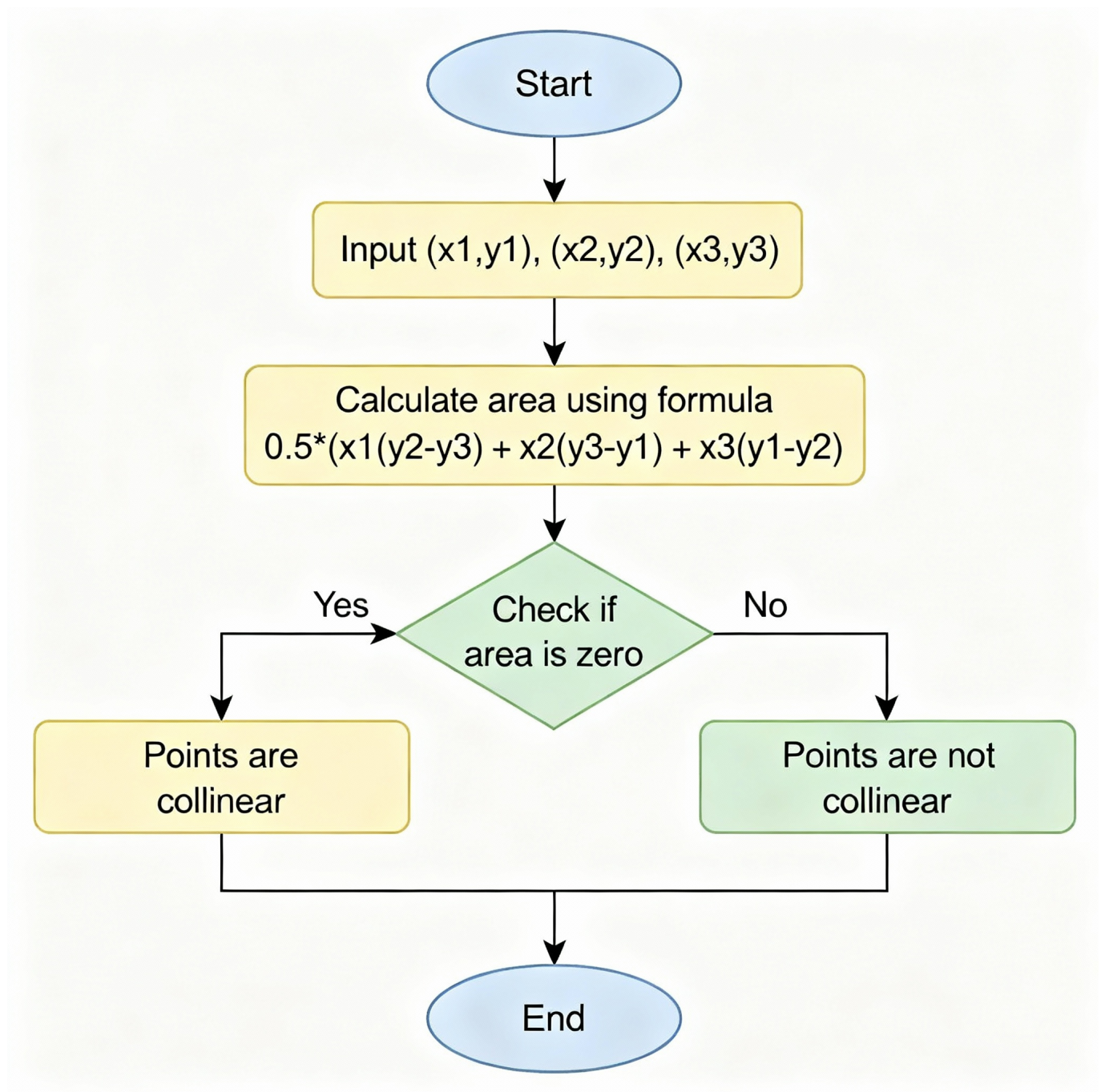
    area = x1*(y2 - y3) + x2*(y3 - y1) + x3*(y1 - y2);

    if (area == 0)
        printf("The points are collinear.\n");
    else
        printf("The points are not collinear.\n");

    return 0;
```

}

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\" ; if ($?) { gcc exp3Q3.c -o exp3Q3 } ; if ($?) { .\exp3Q3 }
Enter coordinates of first point (x1, y1): 2,2
Enter coordinates of second point (x2, y2): Enter coordinates of third point (x3, y3): The points are not collinear.
PS C:\Users\dell\Documents\experiments in c\exp3_1> |
```

4:According to the gregorian calender,it was Monday on the date 01/01/01. if any year is input through the keyboard write a program to find out what is the day on 1st january of this year.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare variables: - year (input year) - total_days (number of days between 01/01/01 and 01/01/year) - leap_years (number of leap years before the given year) - normal_years (number of normal years before the given year) - day (to store remainder when total_days is divided by 7)

Step 4: Input the year from the user.

Step 5: Calculate the number of leap years as: $\text{leap_years} = (\text{year} - 1) / 4 - (\text{year} - 1) / 100 + (\text{year} - 1) / 400$

Step 6: Calculate the number of normal years as: $\text{normal_years} = (\text{year} - 1) - \text{leap_years}$

Step 7: Compute $\text{total_days} = (\text{leap_years} * 366) + (\text{normal_years} * 365)$

Step 8: Compute $\text{day} = \text{total_days} \% 7$

Step 9: Since 01/01/01 was Monday, map the remainder to a day: 0 → Monday

1 → Tuesday

2 → Wednesday

3 → Thursday

4 → Friday

5 → Saturday

6 → Sunday

Step 10: Display the corresponding day of the week.

Step 11: Stop

Code:

```
#include <stdio.h>
```

```
int main() { int y, leap, days; printf("Enter year: "); scanf("%d", &y);
```

```
    leap = (y - 1) / 4 - (y - 1) / 100 + (y - 1) / 400;
    days = ((y - 1) * 365 + leap) % 7;
```

```
    printf("1st January %d is ", y);
```

```
    switch (days) {
```

```
        case 0: printf("Monday\n"); break;
```

```
        case 1: printf("Tuesday\n"); break;
```

```
        case 2: printf("Wednesday\n"); break;
```

```
        case 3: printf("Thursday\n"); break;
```

```
        case 4: printf("Friday\n"); break;
```

```
        case 5: printf("Saturday\n"); break;
```

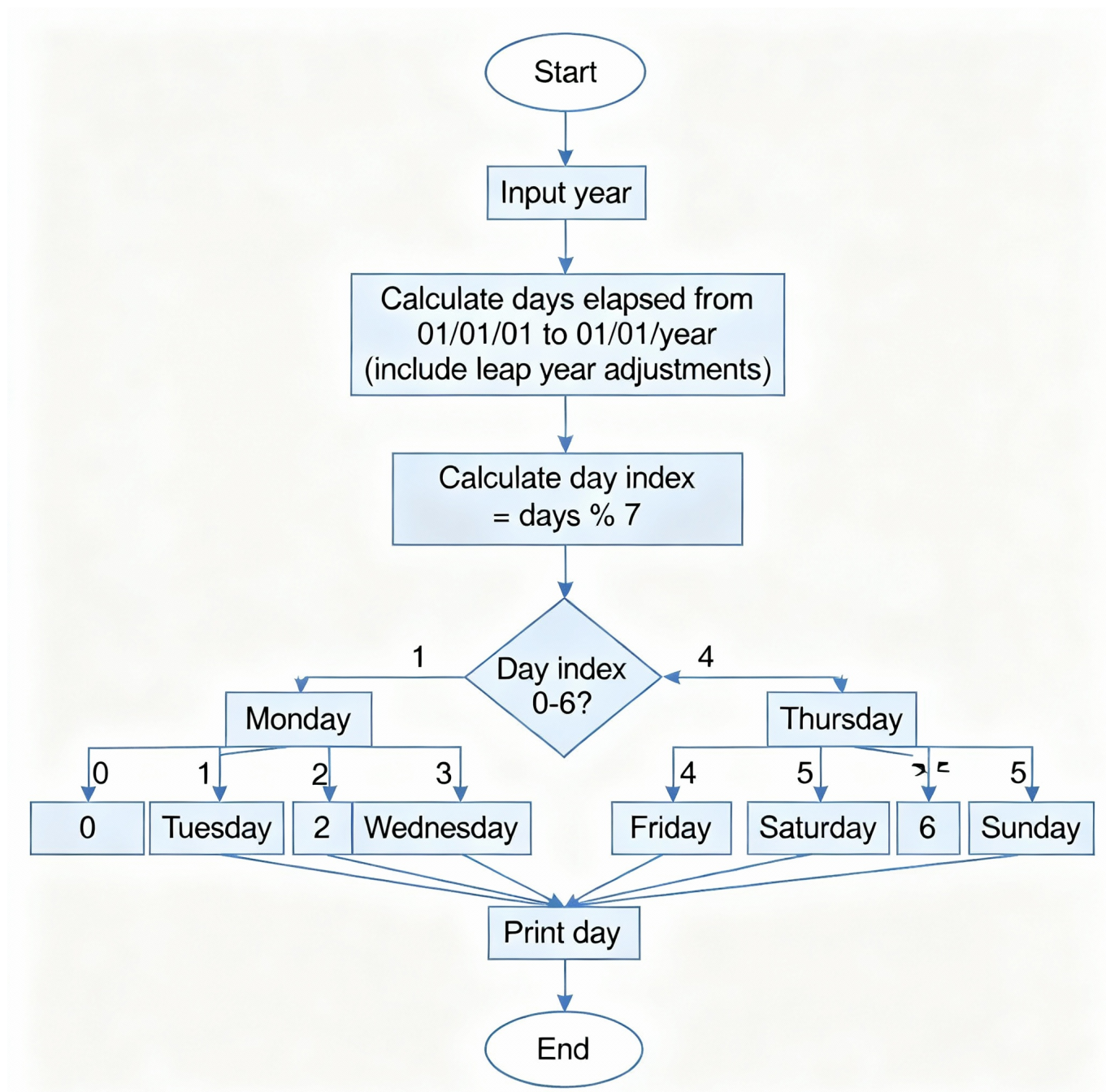
```
        case 6: printf("Sunday\n"); break;
```

```
    }
```

```
    return 0;
```

}

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1" ; if ($?) { gcc exp3Q4.c -o exp3Q4 } ; if ($?) { .\exp3Q4 }
Enter year: 2004
1st January 2004 is Thursday
PS C:\Users\dell\Documents\experiments in c\exp3_1>
```

5:WAP using ternary operator,the user should input the length and breadth of a rectangle,one has to find out which rectangle has the highest perimeter. The minimum no of rectangles should be three.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare variables: l1, b1, l2, b2, l3, b3 for length and breadth p1, p2, p3 for perimeters max for the highest perimeter

Step 4: Input length and breadth for all three rectangles

Step 5: Compute perimeters using: $p1 = 2 * (l1 + b1)$ $p2 = 2 * (l2 + b2)$ $p3 = 2 * (l3 + b3)$

Step 6: Find the highest perimeter using the ternary operator: $max = (p1 > p2 \ \&\& \ p1 > p3) ? p1 : (p2 > p3 ? p2 : p3)$

Step 7: Display which rectangle has the highest perimeter

Step 8: Stop

Code:

```
#include <stdio.h>
```

```
int main() { float l1, b1, l2, b2, l3, b3; float p1, p2, p3, max;
```

```
    printf("Enter length and breadth of Rectangle 1: ");
    scanf("%f %f", &l1, &b1);
    printf("Enter length and breadth of Rectangle 2: ");
    scanf("%f %f", &l2, &b2);
    printf("Enter length and breadth of Rectangle 3: ");
    scanf("%f %f", &l3, &b3);

    p1 = 2 * (l1 + b1);
    p2 = 2 * (l2 + b2);
    p3 = 2 * (l3 + b3);

    max = (p1 > p2 && p1 > p3) ? p1 : (p2 > p3 ? p2 : p3);

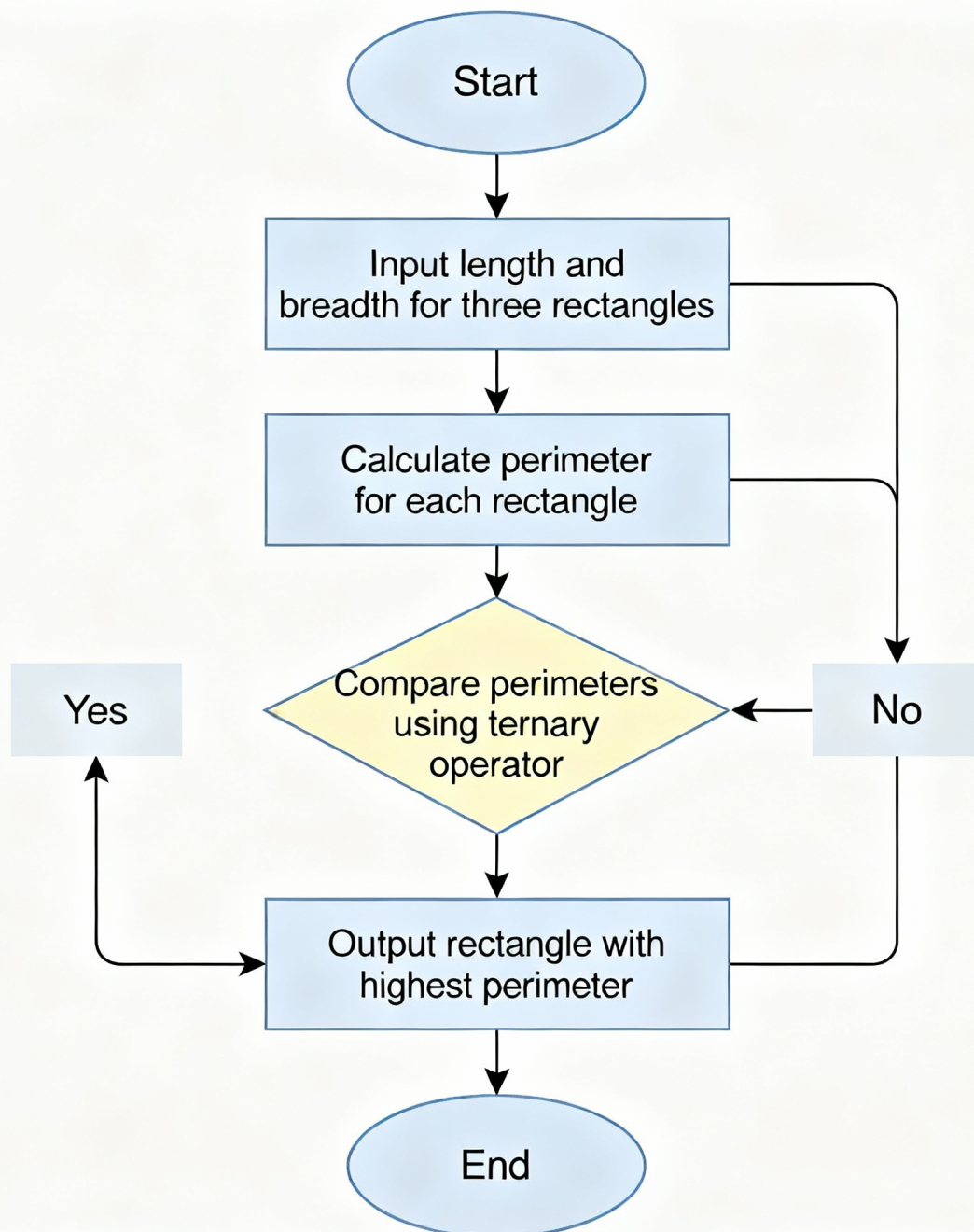
    printf("\nPerimeter of Rectangle 1 = %.2f", p1);
    printf("\nPerimeter of Rectangle 2 = %.2f", p2);
    printf("\nPerimeter of Rectangle 3 = %.2f", p3);

    if (max == p1)
```

```
printf("\n\nRectangle 1 has the highest perimeter: %.2f\n", max);  
else if (max == p2)  
    printf("\n\nRectangle 2 has the highest perimeter: %.2f\n", max);  
else  
    printf("\n\nRectangle 3 has the highest perimeter: %.2f\n", max);  
  
return 0;
```

}

Flowchart



Output


```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\" ; if ($?) { gcc exp3Q5.c -o exp3Q5 } ; if ($?) { .\exp3Q5 }
Enter length and breadth of Rectangle 1: 12 5
Enter length and breadth of Rectangle 2: 13 5
Enter length and breadth of Rectangle 3: 5 6

Perimeter of Rectangle 1 = 34.00
Perimeter of Rectangle 2 = 36.00
Perimeter of Rectangle 3 = 22.00

Rectangle 2 has the highest perimeter: 36.00
PS C:\Users\dell\Documents\experiments in c\exp3_1> |
```

EXPERIMENT 3.2: LOOPS

1:WAP to enter numbers till the user wants. At the end,it should display the count of positive,negative and zeroes entered.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare integer variables: num (for input number), pos = 0, neg = 0, zero = 0 (for counting), choice (for user decision)

Step 4: Repeat the following steps using a loop: a) Prompt the user to enter a number b) Read the number using scanf() c) If number > 0 → increment pos Else if number < 0 → increment neg Else → increment zero d) Ask the user if they want to continue (1 = yes, 0 = no) e) Read the choice

Step 5: Continue the loop while choice == 1

Step 6: After the loop ends, display: - Total count of positive numbers - Total count of negative numbers - Total count of zeroes

Step 7: Stop

Code:

```
#include <stdio.h>
```

```
int main() { int num, pos = 0, neg = 0, zero = 0; int choice = 1;
```

```
while (choice == 1) {
    printf("Enter a number: ");
    scanf("%d", &num);

    if (num > 0)
        pos++;
    else if (num < 0)
        neg++;
    else
        zero++;
}
```

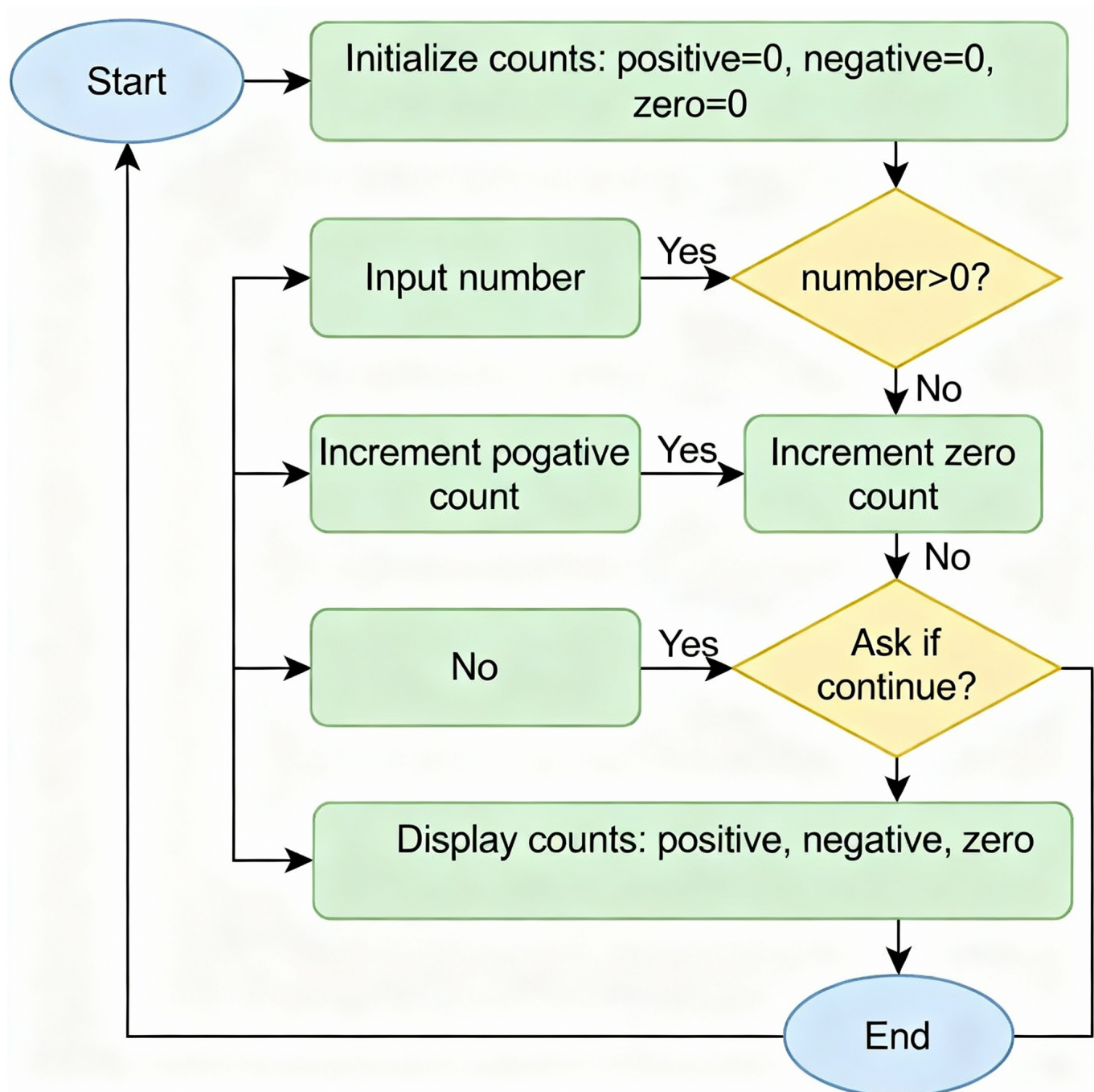
```
printf("Do you want to continue? (1 = Yes, 0 = No): ");
scanf("%d", &choice);
}

printf("\nCount of Positive Numbers: %d", pos);
printf("\nCount of Negative Numbers: %d", neg);
printf("\nCount of Zeroes: %d\n", zero);

return 0;
```

}

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp3_2\" ; if ($?) { gcc exp3_2Q1.c -o exp3_2Q1 } ; if ($?) { .\exp3_2Q1 }
Enter a number: 12
Do you want to continue? (1 = Yes, 0 = No): 1
Enter a number: 13
Do you want to continue? (1 = Yes, 0 = No): 0

Count of Positive Numbers: 2
Count of Negative Numbers: 0
Count of Zeroes: 0
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp3_2> █
```

2:WAP to print the multiplication table of the numbers entered by the user. it should be in the correct formatting. Num*1=Num.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare integer variables: num and i

Step 4: Prompt the user to enter a number

Step 5: Read the number using scanf()

Step 6: Use a loop (for i = 1 to 10) - Calculate product = num * i - Display the result in the format: num * i = product

Step 7: Stop

Code:

```
#include <stdio.h>
```

```
int main() { int num, i;
```

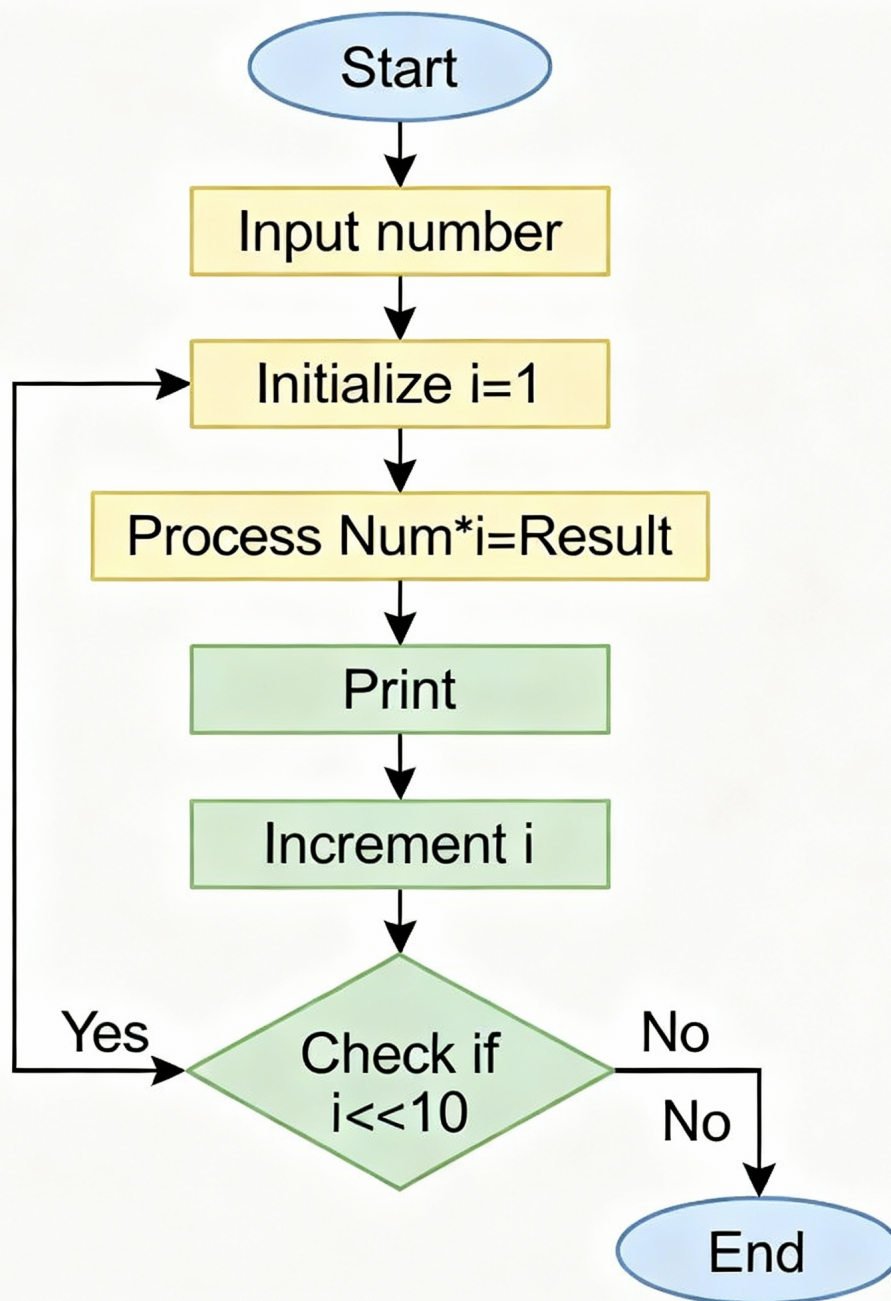
```
    printf("Enter a number: ");
    scanf("%d", &num);

    printf("\nMultiplication Table of %d:\n", num);
    for (i = 1; i <= 10; i++) {
        printf("%d * %d = %d\n", num, i, num * i);
    }

    return 0;
```

```
}
```

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp3_2\" ; if ($?) { gcc exp3_202.c -o exp3_202 } ; if ($?) { .\exp3_202 }
Enter a number: 12

Multiplication Table of 12:
12 * 1 = 12
12 * 2 = 24
12 * 3 = 36
12 * 4 = 48
12 * 5 = 60
12 * 6 = 72
12 * 7 = 84
12 * 8 = 96
12 * 9 = 108
12 * 10 = 120
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp3_2> 
```

3: WAP to generate the following set of output.

a.



2 3

4 5 6

b.

1

1 1

1 2 1

1 3 3 1

Algorithm

a)

Step 1: Start

Step 2: Initialize variable num = 1 to keep track of current number

Step 3: Loop for rows from 1 to 3 (i)

Step 4: For each row i, loop from j = 1 to i

Step 5: Print the current num and increment num by 1

Step 6: Move to the next line after each row

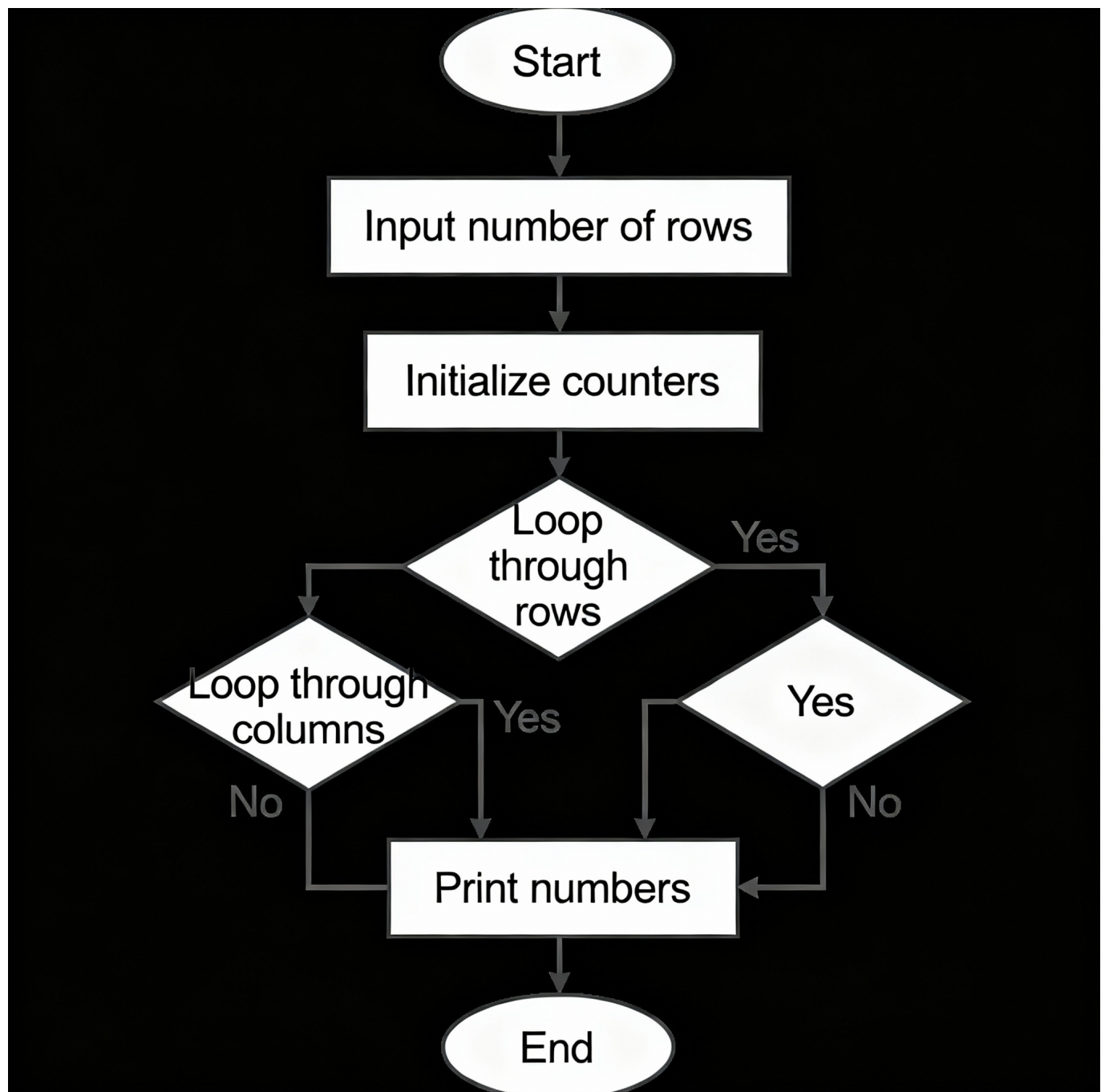
Step 7: Stop

Code:

```
#include <stdio.h>
```

```
int main() { int num = 1; for (int i = 1; i <= 3; i++) { for (int j = 1; j <= i; j++) { printf("%d ", num++); }  
printf("\n"); } return 0; }
```

Flowchart



Output

```
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp3_2\" ; if ($?) { gcc exp3_Q3a.c -o exp3_Q3a } ; if ($?) { .\exp3_Q3a }  
1  
2 3  
4 5 6  
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp3_2>
```

b)

Algorithm

Step 1: Start

Step 2: Declare an array or variables to hold current and previous row values

Step 3: Loop through rows from 0 to 3 (4 rows total)

Step 4: For each row i , print the values using the formula for combinations or

```
For j from 0 to i:  
    If j == 0 or j == i, print 1  
    Else print sum of previous row's j and j-1 elements
```

Step 5: Move to the next line after each row

Step 6: Stop

Code:

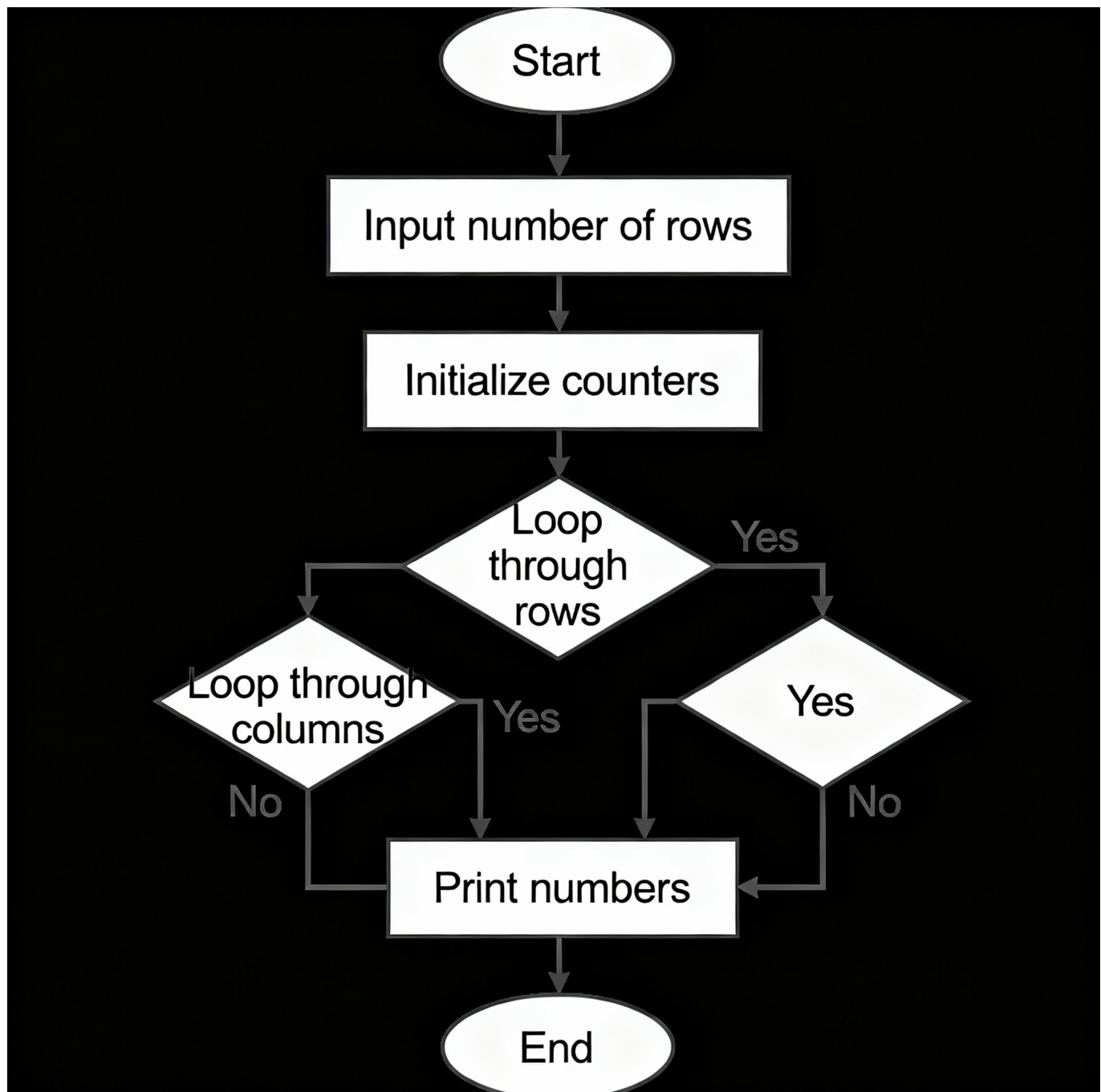
```
#include <stdio.h>
```

```
int main() { int n = 4; int arr[n][n];
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j <= i; j++) {  
            if (j == 0 || j == i)  
                arr[i][j] = 1;  
            else  
                arr[i][j] = arr[i-1][j-1] + arr[i-1][j];  
  
            printf("%d ", arr[i][j]);  
        }  
        printf("\n");  
    }  
  
    return 0;
```

```
}
```

Flowchart



Output

```

PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp3_2\" ; if ($?) { gcc exp3_Q3b.c -o exp3_Q3b } ; if ($?) { .\exp3_Q3b }
1
1 1
1 2 1
1 3 3 1
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp3_2>

```

4: The population of a town is 100000. The population has increased steadily at the rate of 10% per year for the last 10 years. Write a program to determine the population at the end of the year in the last decade.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare variables: - population (float) - year (int)

Step 4: Initialize population = 100000

Step 5: Loop through each year from 1 to 10 a) Compute new population: $\text{population} = \text{population} + (\text{population} * 0.10)$ OR $\text{population} *= 1.10$ b) Display the population at the end of the current year

Step 6: Stop

Code:

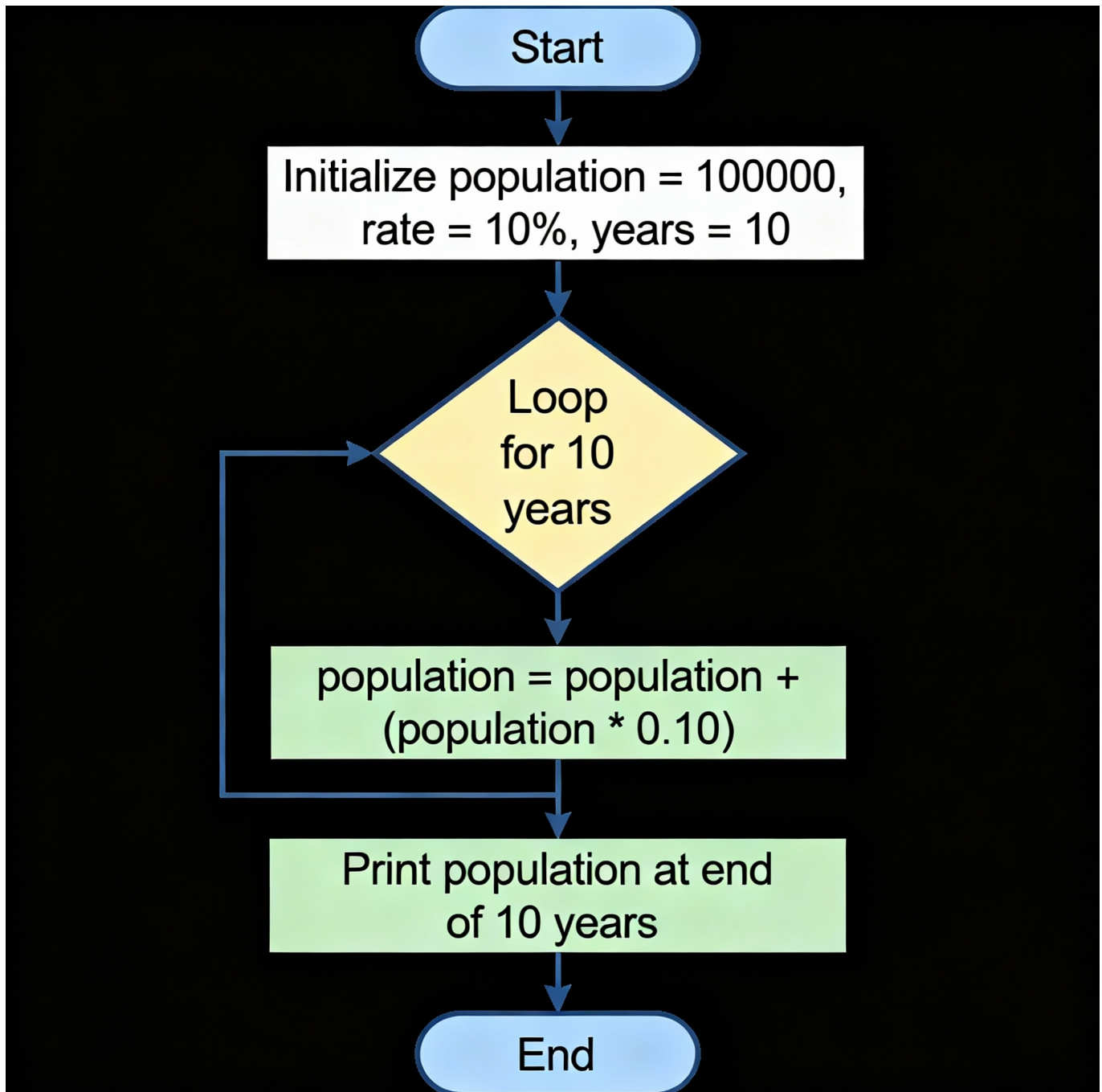
```
#include <stdio.h>
```

```
int main() { float population = 100000; int year;
```

```
    for (year = 1; year <= 10; year++) {  
        population = population * 1.10; // increase by 10%  
        printf("Population at the end of year %d: %.0f\n", year, population);  
    }  
  
    return 0;
```

```
}
```

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp3_2\" ; if ($?) { gcc exp3_2Q4.c -o exp3_2Q4 } ; if ($?) { .\exp3_2Q4 }
Population at the end of year 1: 110000
Population at the end of year 2: 121000
Population at the end of year 3: 133100
Population at the end of year 4: 146410
Population at the end of year 5: 161051
Population at the end of year 6: 177156
Population at the end of year 7: 194872
Population at the end of year 8: 214359
Population at the end of year 9: 235795
Population at the end of year 10: 259374
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp3_2>
```

5: Ramanujan Number is the smallest number that can be expressed as the sum of two cubes in two different ways.WAP to print all such numbers up to a resonable limit

Example of Ramanujan Number: 1729

$12^3 + 1^3$ and $10^3 + 9^3$ for a number $L=20$ (that is limit).

: Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare variables: i, j, k, l → for looping limit → user input n1, n2 → for storing sums of cubes

Step 4: Input limit from user

Step 5: Use four nested loops: For i = 1 to limit For j = i + 1 to limit Compute $n1 = i^3 + j^3$

```
For k = i + 1 to limit
  For l = k + 1 to limit
    Compute  $n2 = k^3 + l^3$ 
    If  $n1 == n2$ 
      Print the Ramanujan number and combinations
```

Step 6: Stop

Code:

```
#include <stdio.h>
```

```
int main() { int i, j, k, l, limit; int n1, n2;
```

```
printf("Enter the limit (e.g., 20): ");
scanf("%d", &limit);

printf("\nRamanujan Numbers up to %d:\n", limit);

for (i = 1; i <= limit; i++) {
    for (j = i + 1; j <= limit; j++) {
        n1 = i*i*i + j*j*j;

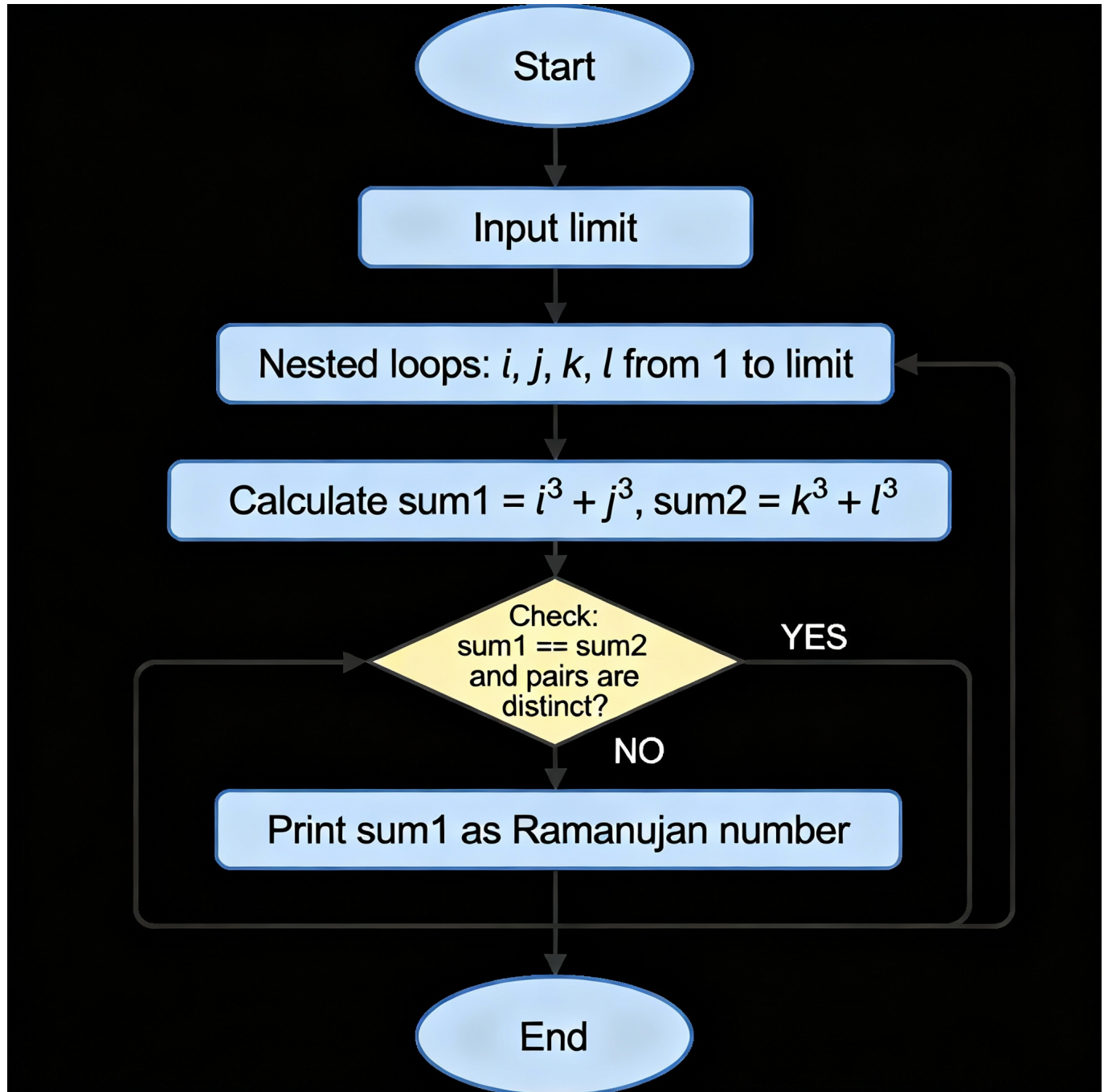
        for (k = i + 1; k <= limit; k++) {
            for (l = k + 1; l <= limit; l++) {
                n2 = k*k*k + l*l*l;

                if (n1 == n2) {
                    printf("%d = %d^3 + %d^3 = %d^3 + %d^3\n", n1, i, j, k, l);
                }
            }
        }
    }
}
```

```
return 0;
```

```
}
```

Flowchart



Output


```
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp3_2\" ; if ($?) { gcc exp3_2Q5.c -o exp3_2Q5 } ; if ($?) { .\exp3_2Q5 }
Enter the limit (e.g., 20): 50

Ramanujan Numbers up to 50:
1729 = 1^3 + 12^3 = 9^3 + 10^3
4104 = 2^3 + 16^3 = 9^3 + 15^3
13832 = 2^3 + 24^3 = 18^3 + 20^3
39312 = 2^3 + 34^3 = 15^3 + 33^3
46683 = 3^3 + 36^3 = 27^3 + 30^3
32832 = 4^3 + 32^3 = 18^3 + 30^3
110656 = 4^3 + 48^3 = 36^3 + 40^3
110808 = 6^3 + 48^3 = 27^3 + 45^3
40033 = 9^3 + 34^3 = 16^3 + 33^3
20683 = 10^3 + 27^3 = 19^3 + 24^3
65728 = 12^3 + 40^3 = 31^3 + 33^3
64232 = 17^3 + 39^3 = 26^3 + 36^3
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp3_2>
```

EXPERIMENT 4: VARIABLE AND SCOPE OF VARIABLES

1: Declare a global variable outside all functions and use it inside various functions to understand its accessibility.

Algorithm

Step 1: Start

Step 2: Declare a global variable outside all functions (e.g., int count = 10)

Step 3: Define a function display() that prints the value of the global variable

Step 4: Define another function modify() that changes the value of the global variable

Step 5: In main(): a) Print the initial value of the global variable b) Call display() to print the value c) Call modify() to change the value d) Call display() again to show the modified value

Step 6: Stop

Code:

```
#include <stdio.h>

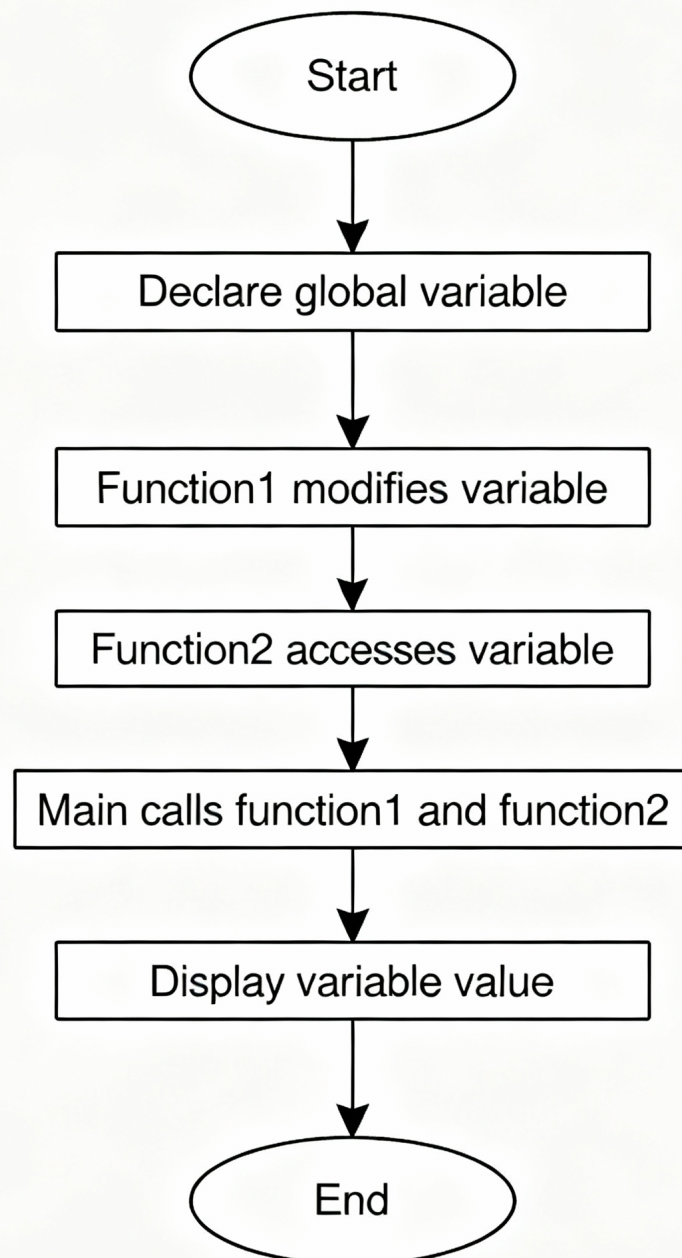
int count = 10;

void display() { printf("Value of global variable (in display): %d\n", count); }

void modify() { count += 5; printf("Global variable modified (in modify): %d\n", count); }

int main() { printf("Initial value of global variable (in main): %d\n", count); display();
modify();
display();
return 0; }
```

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp4\" ; if ($?) { gcc exp4Q1.c -o exp4Q1 } ; if ($?) { .\exp4Q1 }
Initial value of global variable (in main): 10
Value of global variable (in display): 10
Global variable modified (in modify): 15
Value of global variable (in display): 15
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp4>
```

2: Declare a local variable inside a function and try to access it outside the function. Compare this with accessing the global variable from within the function.

Algorithm

Step 1: Start

Step 2: Declare a global variable outside all functions (e.g., `int globalVar = 10`)

Step 3: Define a function `testFunction()`: a) Declare a local variable (e.g., `int localVar = 5`) b) Print both `localVar` and `globalVar`

Step 4: In `main()`: a) Print the global variable b) Call `testFunction()` c) Try to access `localVar` (which should give an error or be invalid)

Step 5: Observe that: - The global variable can be accessed anywhere. - The local variable is only accessible inside the function where it's declared.

Step 6: Stop

Code:

```
#include <stdio.h>
```

```
int globalVar = 10;
```

```
void testFunction() { int localVar = 5; printf("Inside testFunction:\n"); printf("Local variable = %d\n", localVar);  
printf("Global variable = %d\n\n", globalVar); }
```

```
int main() { printf("Inside main:\n"); printf("Global variable = %d\n", globalVar);
```

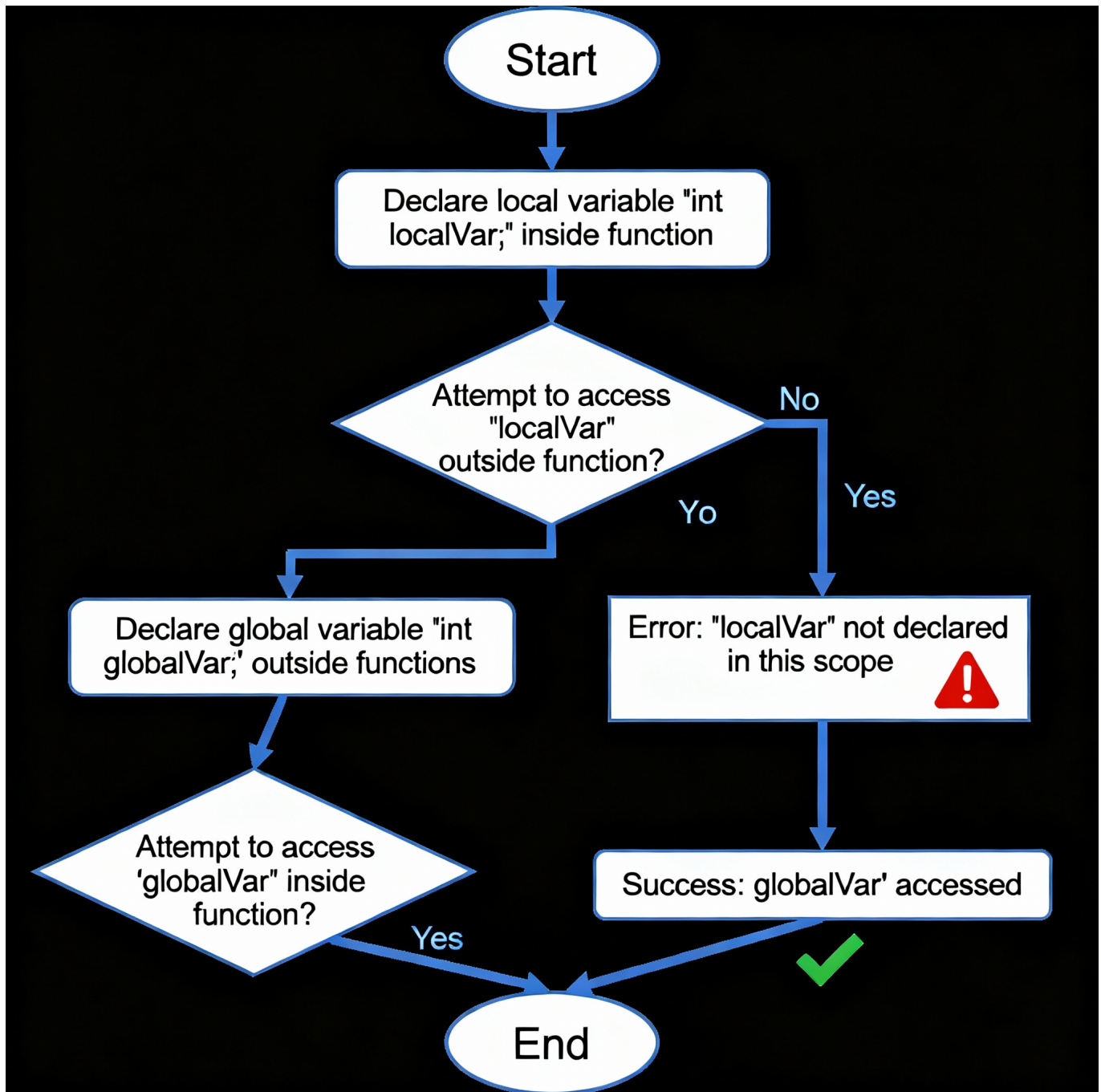
```
    testFunction();
```

```
    printf("Back in main, global variable still accessible = %d\n", globalVar);
```

```
    return 0;
```

```
}
```

Flowchart



Output

```

PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp4\" ; if ($?) { gcc exp4Q2.c -o exp4Q2 } ; if ($?) { .\exp4Q2 }
Inside main:
Global variable = 10
Inside testFunction:
Local variable = 5
Global variable = 10

Back in main, global variable still accessible = 10
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp4>

```

3: Declare variables within different code blocks (enclosed by curly braces) and test their accessibility within and outside those blocks

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: In main(), declare a variable 'a' and assign it a value (e.g., a = 10)

Step 4: Print the value of 'a'

Step 5: Create a new block using { } a) Declare another variable 'b' inside this block b) Print 'a' (accessible from outer block) c) Print 'b' (accessible within this block)

Step 6: Outside the inner block: a) Print 'a' again (still accessible) b) Try to print 'b' (will cause an error if attempted, because 'b' is not in scope)

Step 7: End the program

Step 8: Stop

Code:

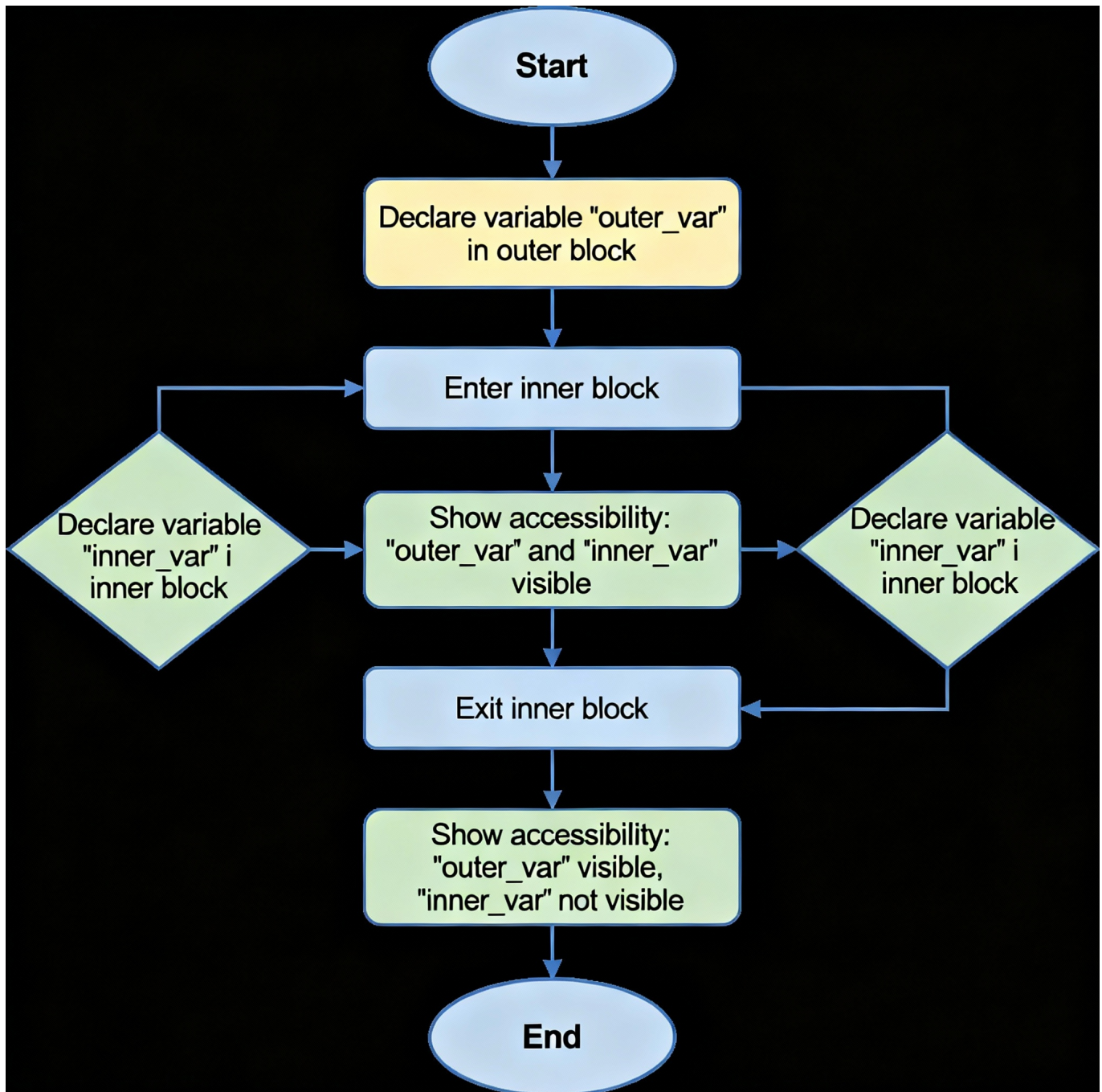
```
#include <stdio.h>
```

```
int main() { int a = 10; printf("Inside main (outer block): a = %d\n", a);
```

```
    {  
        int b = 20;  
        printf("Inside inner block: a = %d, b = %d\n", a, b);  
    }  
    printf("Outside inner block: a = %d\n", a);  
  
    return 0;
```

```
}
```

Flowchart



Output

```

PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp4\" ; if ($?) { gcc exp4Q4.c -o exp4Q4 } ; if ($?) { .\exp4Q4 }
Inside main (outer block): a = 10
Inside inner block: a = 10, b = 20
Outside inner block: a = 10
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp4>

```

4: Declare a static local variable inside a function. Observe how its value persists across function calls.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Define a function `display()`: a) Declare a static local variable `count` and initialize it to 1 b) Print the value of `count` c) Increment `count` by 1

Step 4: In `main()`: a) Call `display()` multiple times (e.g., 3 times) b) Observe that the value of `count` persists between function calls

Step 5: Stop

Code:

```
#include <stdio.h>
```

```
void display() { static int count = 1; printf("Count = %d\n", count); count++;  
}
```

```
int main() { printf("First call:\n"); display();
```

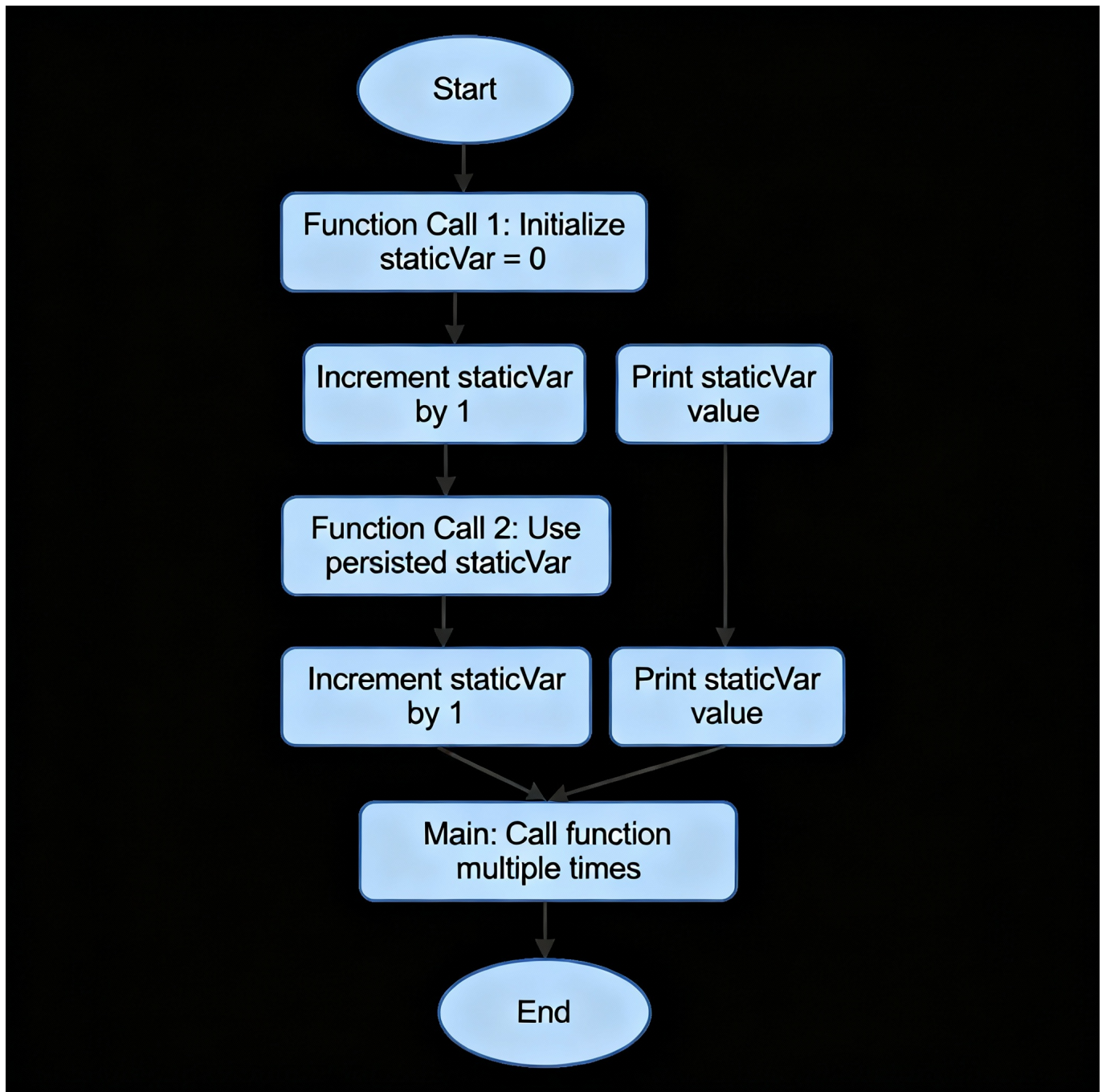
```
    printf("Second call:\n");  
    display();
```

```
    printf("Third call:\n");  
    display();
```

```
    return 0;
```

```
}
```

Flowchart



Output

```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp4\" ; if ($?) { gcc exp4Q4.c -o exp4Q4 } ; if ($?) { .\exp4Q4 }
First call:
Count = 1
Second call:
Count = 2
Third call:
Count = 3
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp4>
```

EXPERIMENT 5: ARRAY

1:WAP to read a list of integers and store it in a single dimensional array. Write a C program to print the second largest integer in a list of integers.

Algorithm

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare variables: - arr[] → to store integers - n → number of elements - i → loop counter - first, second → to store the largest and second largest numbers

Step 4: Ask the user to enter the number of elements (n)

Step 5: Input all n integers into the array

Step 6: Initialize: first = second = smallest possible value (e.g., -9999 or arr[0])

Step 7: Loop through the array: For i = 0 to n-1 - If arr[i] > first second = first first = arr[i] - Else if arr[i] > second and arr[i] != first second = arr[i]

Step 8: Print the value of 'second' as the second largest integer

Step 9: Stop

Code:

```
#include <stdio.h>
```

```
int main() { int arr[100], n, i; int first, second;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d integers:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    first = second = -9999;

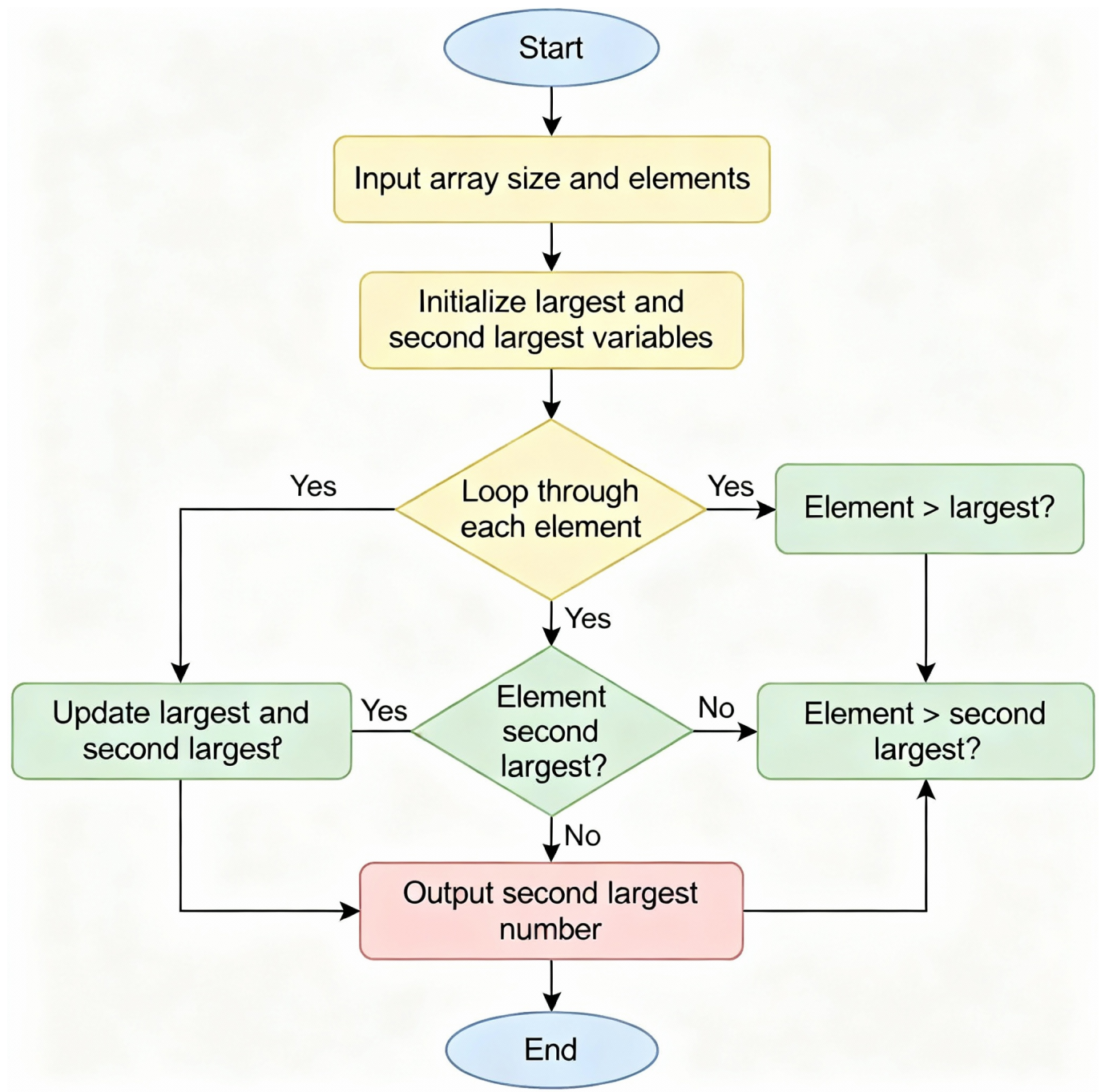
    for (i = 0; i < n; i++) {
        if (arr[i] > first) {
            second = first;
            first = arr[i];
        } else if (arr[i] > second && arr[i] != first) {
            second = arr[i];
        }
    }

    printf("\nThe second largest number is: %d\n", second);

    return 0;
```

}

Flowchart



Output

```

PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp4\exp5\" ; if ($?) { gcc exp5Q1.c -o exp5Q1 } ; if ($?) { .\exp5Q1 }
Enter number of elements: 20
Enter 20 integers:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

The second largest number is: 19
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp4\exp5>

```

2:WAP to read a list of integers and store it in a single dimensional array. Write a C program to count and display positive ,negative,odd and even numbers in a array.

Algorithm

Algorithm: Count and Display Positive, Negative, Odd, and Even Numbers

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare variables: - arr[] → array to store integers - n → number of elements - i → loop counter - pos, neg, odd, even → counters initialized to 0

Step 4: Ask the user to enter the number of elements (n)

Step 5: Input n integers into the array using a loop

Step 6: Loop through the array from i = 0 to n-1: a) If arr[i] > 0, increment pos b) Else if arr[i] < 0, increment neg c) If arr[i] % 2 == 0, increment even Else increment odd

Step 7: Display the counts of: - Positive numbers - Negative numbers - Even numbers - Odd numbers

Step 8: Stop

Code:

```
#include <stdio.h>
```

```
int main() { int arr[100], n, i; int pos = 0, neg = 0, even = 0, odd = 0;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d integers:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for (i = 0; i < n; i++) {
        if (arr[i] > 0)
            pos++;
        else if (arr[i] < 0)
            neg++;

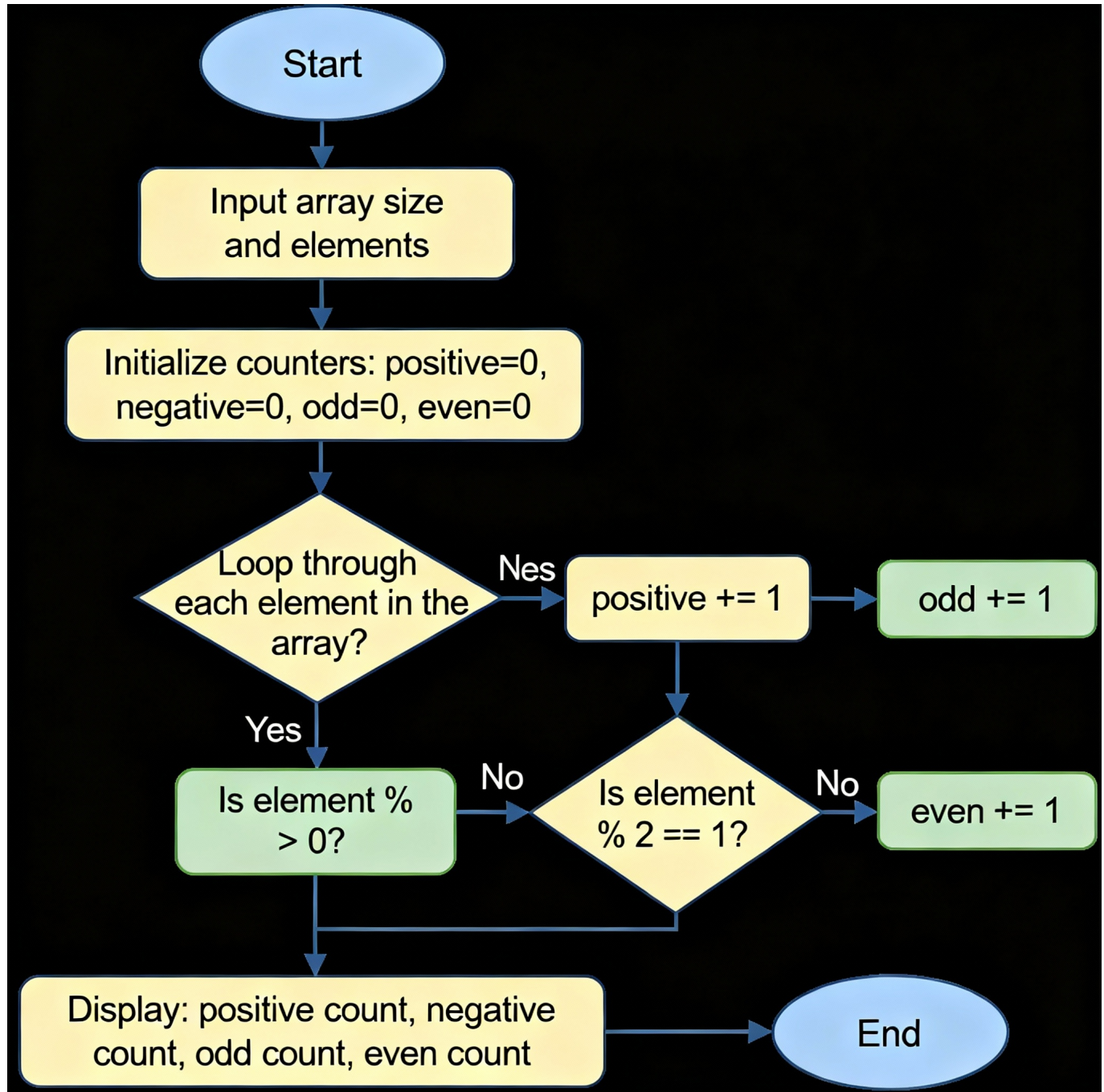
        if (arr[i] % 2 == 0)
            even++;
        else
            odd++;
    }

    printf("\nCount of Positive numbers: %d", pos);
    printf("\nCount of Negative numbers: %d", neg);
    printf("\nCount of Even numbers: %d", even);
    printf("\nCount of Odd numbers: %d\n", odd);
```

```
return 0;
```

```
}
```

Flowchart



Output


```
PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp4\exp5\" ; if ($?) { gcc exp5Q2.c -o exp5Q2 } ; if ($?) { .\exp5Q2 }
Enter number of elements: 4
Enter 4 integers:
1 2 3 4

Count of Positive numbers: 4
Count of Negative numbers: 0
Count of Even numbers: 2
Count of Odd numbers: 2
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp4\exp5> |
```

3:WAP to read a list of integers and store it in a single dimensional array. Write a C program to find the frequency of a particular numbers in list of integers

Algorithm

Algorithm: Find Frequency of a Given Number in an Array

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare variables: - arr[] → array to store integers - n → number of elements - i → loop counter - num → number whose frequency is to be found - count → variable to count frequency (initialize to 0)

Step 4: Ask the user to enter the number of elements (n)

Step 5: Input n integers into the array using a loop

Step 6: Ask the user to enter the number (num) whose frequency is to be found

Step 7: Loop through the array from i = 0 to n - 1: - If arr[i] == num increment count by 1

Step 8: Display the frequency (count) of the number

Step 9: Stop

Code:

```
#include <stdio.h>
```

```
int main() { int arr[100], n, i, num, count = 0;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);
```

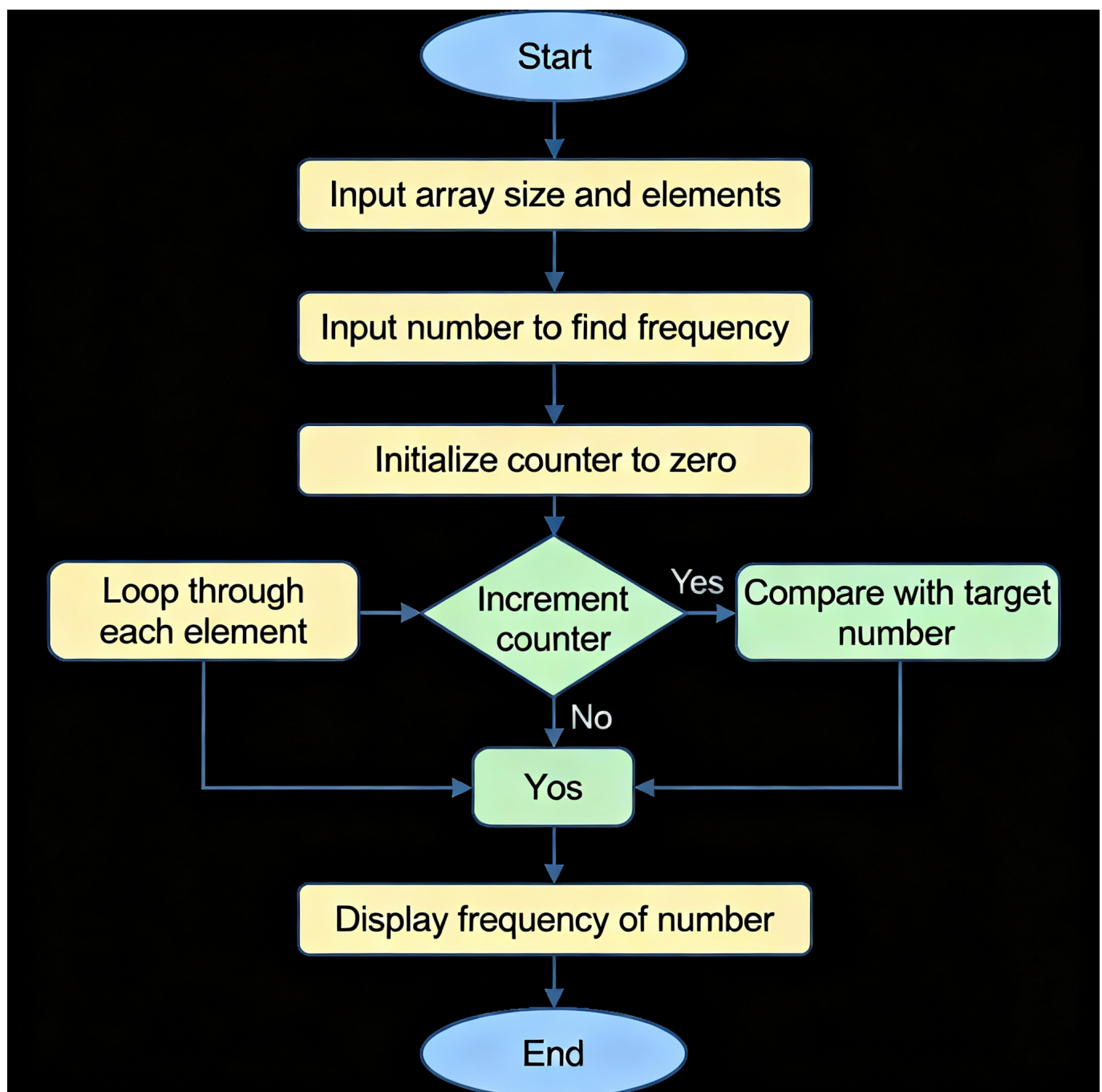
```
    printf("Enter %d integers:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
```

```
    printf("Enter the number to find its frequency: ");
    scanf("%d", &num);
```

```
for (i = 0; i < n; i++) {  
    if (arr[i] == num)  
        count++;  
}  
  
printf("\nThe frequency of %d is: %d\n", num, count);  
  
return 0;
```

}

Flowchart



Output

```

PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp4\exp5\" ; if ($?) { gcc exp5Q3.c -o exp5Q3 } ; if ($?) { .\exp5Q3 }
Enter number of elements: 5
Enter 5 integers:
1 20 37 34 7
Enter the number to find its frequency: 34

The frequency of 34 is: 1
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp4\exp5>

```

4: WAP a program that reads two matrices A(m X n) and B(p X q) and computes the product A and B. Read matrix A and B in a row major order respectively. Print both the input matrices and resultant matrix with suitable headings and output should be in matrix form only. Program must check the compatibility of orders of the matrices for multiplication. Report appropriate message in case of incompatibility.

Algorithm

Algorithm: Matrix Multiplication with Compatibility Check

Step 1: Start

Step 2: Include the header file <stdio.h>

Step 3: Declare matrices: A[m][n], B[p][q], C[m][q] and variables m, n, p, q, i, j, k

Step 4: Input order of matrix A (m × n) Input elements of matrix A (row-wise)

Step 5: Input order of matrix B (p × q) Input elements of matrix B (row-wise)

Step 6: Check compatibility for multiplication: If n != p Print "Matrix multiplication not possible!" Stop the program

Step 7: Else Initialize all elements of C to 0 For i = 0 to m-1 For j = 0 to q-1 For k = 0 to n-1 C[i][j] = C[i][j] + A[i][k] * B[k][j]

Step 8: Print Matrix A Step 9: Print Matrix B Step 10: Print Resultant Matrix C

Step 11: Stop

Code:

```
#include <stdio.h>
```

```
int main() { int A[10][10], B[10][10], C[10][10] = {0}; int m, n, p, q, i, j, k;
```

```

printf("Enter rows and cols of Matrix A: ");
scanf("%d %d", &m, &n);
printf("Enter Matrix A elements:\n");
for(i=0;i<m;i++) for(j=0;j<n;j++) scanf("%d",&A[i][j]);

printf("Enter rows and cols of Matrix B: ");

```

```
scanf("%d %d", &p, &q);
printf("Enter Matrix B elements:\n");
for(i=0;i<p;i++) for(j=0;j<q;j++) scanf("%d",&B[i][j]);

if(n != p) {
    printf("\nMatrix multiplication not possible!\n");
    return 0;
}

for(i=0;i<m;i++)
    for(j=0;j<q;j++)
        for(k=0;k<n;k++)
            C[i][j] += A[i][k] * B[k][j];

printf("\nMatrix A:\n");
for(i=0;i<m;i++){ for(j=0;j<n;j++) printf("%4d",A[i][j]); printf("\n"); }

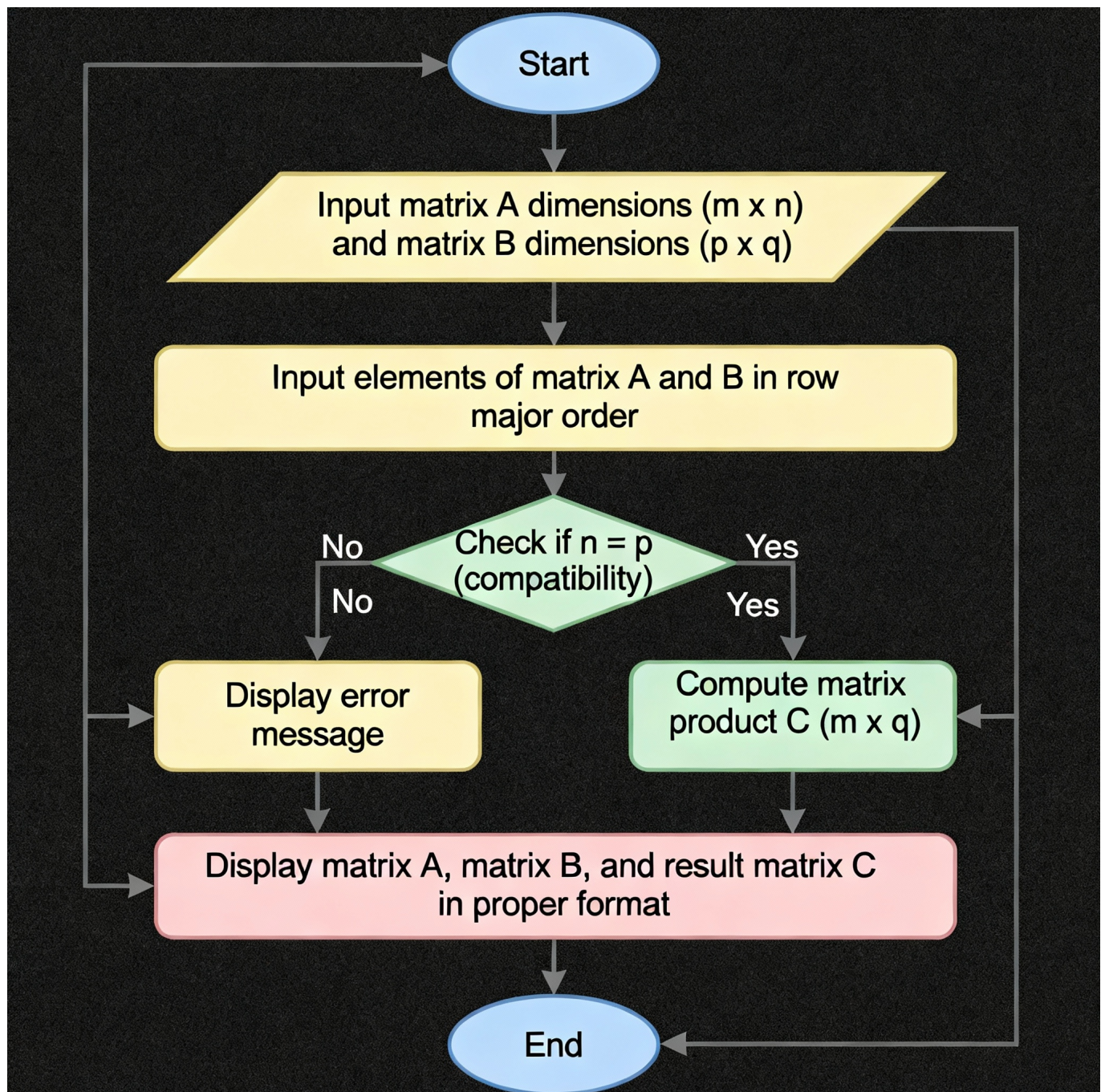
printf("\nMatrix B:\n");
for(i=0;i<p;i++){ for(j=0;j<q;j++) printf("%4d",B[i][j]); printf("\n"); }

printf("\nResultant Matrix (A × B):\n");
for(i=0;i<m;i++){ for(j=0;j<q;j++) printf("%4d",C[i][j]); printf("\n"); }

return 0;

}
```

Flowchart



Output

```

PS C:\Users\dell\Documents\experiments in c>
cd "c:\Users\dell\Documents\experiments in c\exp3_1\exp4\exp5\" ; if ($?) { gcc exp5Q4.c -o exp5Q4 } ; if ($?) { .\exp5Q4 }
Enter rows and cols of Matrix A: 2
4
Enter Matrix A elements:
1 2
3 4
4 5
2 3
Enter rows and cols of Matrix B: 2 2
Enter Matrix B elements:
2 3
3 4

Matrix multiplication not possible!
PS C:\Users\dell\Documents\experiments in c\exp3_1\exp4\exp5>

```

EXPERIMENT:6 FUNCTIONS

Q1.Develop a recursive and non recursive function FACT(num) to find the factorial of a number, $n!$ defined by $FACT(n)=1, n=0$. Otherwise, $FACT(n)=n*FACT(n-1)$.using this function write a C program to compute the binomial coffecient . Tabulate the result for different values of n and r with suitable messages.

Algorithm:

Step 1: Start Step 2: Read values of n and r Step 3: If $(r > n \text{ OR } n < 0 \text{ OR } r < 0)$

Print "Invalid Input" and Stop

Step 4: Compute factorial of n using recursive FACT(n)

FACT_rec(n): If $n = 0 \rightarrow$ return 1 Else $\rightarrow$ return $n \times FACT_rec(n-1)$

Step 5: Compute factorial of r using non-recursive FACT(r)

FACT_nonrec(r): Initialize $fact = 1$ For $i = 1$ to r : $fact = fact \times i$ Return $fact$

Step 6: Compute factorial of $(n - r)$ using non-recursive FACT($n-r$) Step 7: Compute Binomial Coefficient

$C(n, r) = FACT(n) / (FACT(r) \times FACT(n - r))$

Step 8: Display the value of $C(n, r)$ Step 9: Print table of binomial coefficients

For $i = 0$ to n For $j = 0$ to i Compute $C(i, j)$ Display result

Step 10: Stop

code:

```
#include <stdio.h>

// Recursive factorial function
long long FACT_rec(int n) {
    if (n == 0)
        return 1;
    else
        return n * FACT_rec(n - 1);
}

// Non-recursive factorial function
long long FACT_nonrec(int n) {
    long long fact = 1;
    for (int i = 1; i <= n; i++)
        fact *= i;
    return fact;
}

// Function to compute binomial coefficient C(n, r)
```

```

long long binomial(int n, int r) {
    long long num = FACT_rec(n);          // Using recursive factorial
    long long den = FACT_nonrec(r) * FACT_nonrec(n - r); // Using non-
recursive factorial
    return num / den;
}

int main() {
    int n, r;

    printf("BINOMIAL COEFFICIENT CALCULATOR USING FACTORIAL FUNCTIONS\n");
    printf("-----\n");

    printf("Enter value of n: ");
    scanf("%d", &n);

    printf("Enter value of r: ");
    scanf("%d", &r);

    if (r > n || n < 0 || r < 0) {
        printf("Invalid input! r must be <= n and both must be non-
negative.\n");
        return 1;
    }

    printf("\nUsing recursive FACT(n) and non-recursive FACT(r) & FACT(n-
r)...\n");
    printf("C(%d, %d) = %lld\n\n", n, r, binomial(n, r));

    // Display table
    printf("-----\n");
    printf("    TABLE OF BINOMIAL COEFFICIENTS (C(n, r))\n");
    printf("-----\n");
    printf("    n    r    C(n,r)\n");
    printf("-----\n");

    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= i; j++) {
            printf("    %3d %3d    %10lld\n", i, j, binomial(i, j));
        }
    }

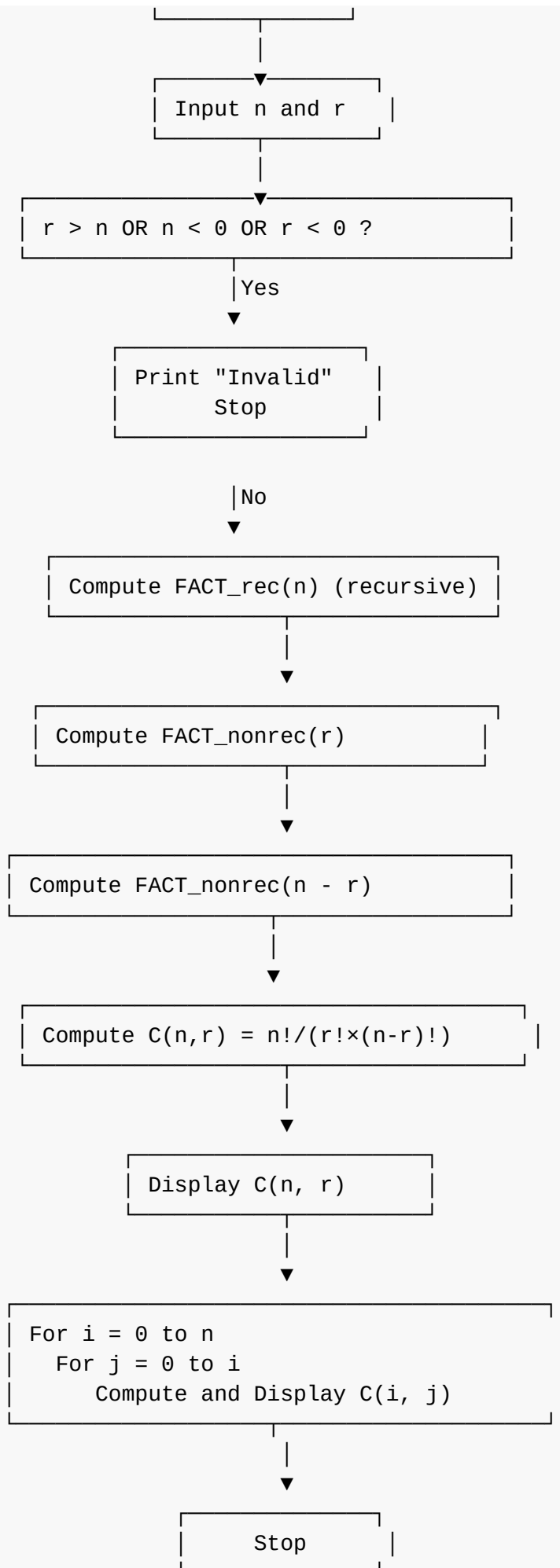
    printf("-----\n");

    return 0;
}

```

Flowchart:





Output:

Flowchart:

```

graph TD
    A[Input n and r] --> B{r > n OR n < 0 OR r < 0 ?}
    B -- Yes --> C[ ]
    B -- No --> D[ ]
  
```

TABLE OF BINOMIAL COEFFICIENTS (C(n, r))

n	r	C(n, r)
0	0	1
1	0	1
1	1	1
2	0	2
2	1	1
2	2	2
3	0	6
3	1	2
3	2	2
3	3	6
4	0	24
4	1	6
4	2	4
4	3	6
4	4	24
5	0	120
5	1	24
5	2	12
5	3	12
5	4	24
5	5	120

Terminal Output:

```

aadiya@fedora:~/experiments-in-c$ cd ~/home/aadiya/experiments-in-c/exp6/ && gcc exp6Q1.c -o exp6Q1 && ./exp6Q1
BINOMIAL COEFFICIENT CALCULATOR USING FACTORIAL FUNCTIONS
Enter value of n: 5
Enter value of r: 3
Using recursive FACT(n) and non-recursive FACT(x) & FACT(n-x)...
C(5, 3) = 120
  
```

Q2. Develop a recursive GCD function(num1, num2) that accepts two integer arguments. Write a C program that invokes this function to find the greatest common divisor of two given integers.

Algorithm:

Step 1: Start Step 2: Read two integers num1 and num2 Step 3: Call the function GCD(num1, num2) Inside GCD(a, b):

If b == 0, return a

Otherwise, return GCD(b, a % b)

Step 4: Display the returned GCD value Step 5: Stop

Code:

```

#include <stdio.h>

// Recursive function to find GCD
int GCD(int num1, int num2) {
    if (num2 == 0)
        return num1;
    else
        return GCD(num2, num1 % num2);
}

int main() {
  
```

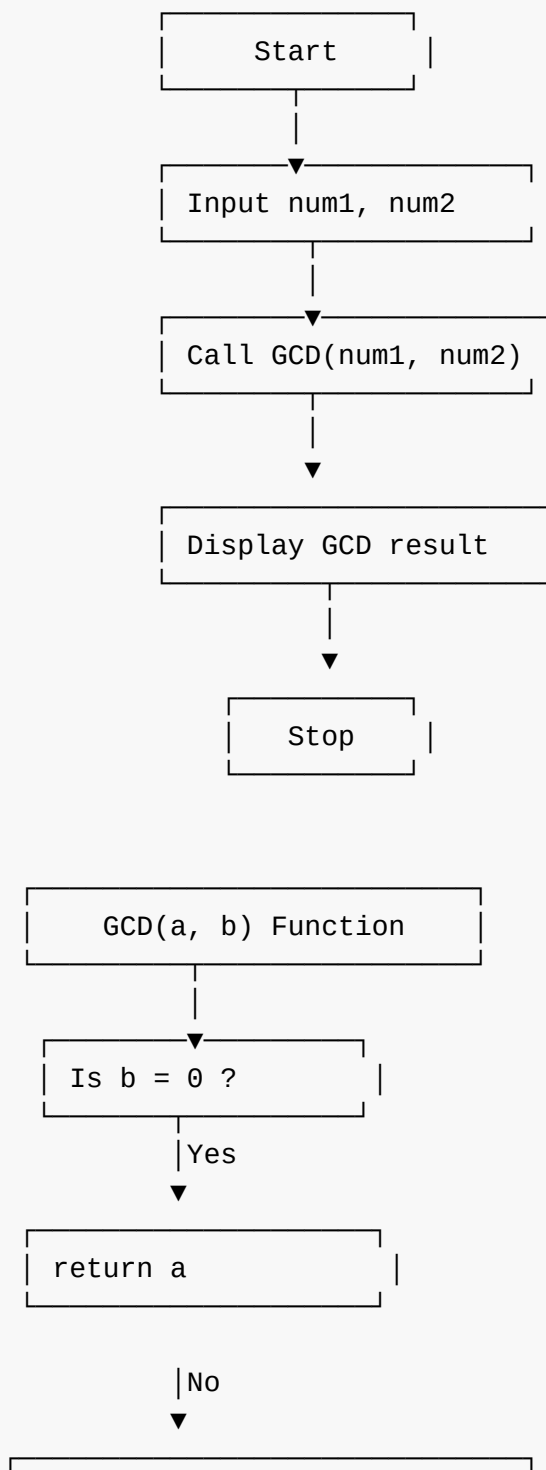
```
int a, b;

printf("Enter two integers: ");
scanf("%d %d", &a, &b);

printf("The GCD of %d and %d is: %d\n", a, b, GCD(a, b));

return 0;
}
```

Flowchart:



```
return GCD(b, a % b)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q2.c -o exp6Q2 && "/home/aaditya/experiments-in-c/exp6/"exp6Q2
• aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q2.c -o exp6Q2 && "/home/aaditya/experiments-in-c/exp6/"exp6Q2
Enter two integers: 1 2
The GCD of 1 and 2 is: 1
o aaditya@fedora:~/experiments-in-c/exp6$
```

Q3. Develop a recursive function FIBO(num) that accepts an integer argument. Write a C program that invokes this function to generate the Fibonacci sequence up to num.

Algorithm:

Step 1: Start Step 2: Read the value of num (the limit of the sequence) Step 3: For each value i from 0 to num-1

Call FIBO(i) Display the returned value

Step 4: End loop Step 5: Stop Recursive Function FIBO(n):

If n == 0, return 0

If n == 1, return 1

Otherwise return FIBO(n-1) + FIBO(n-2)

Code:

```
#include <stdio.h>

// Recursive Fibonacci function
int FIBO(int num) {
    if (num == 0)
        return 0;
    if (num == 1)
        return 1;
    return FIBO(num - 1) + FIBO(num - 2);
}

int main() {
    int num;

    printf("Enter the number of terms: ");
    scanf("%d", &num);

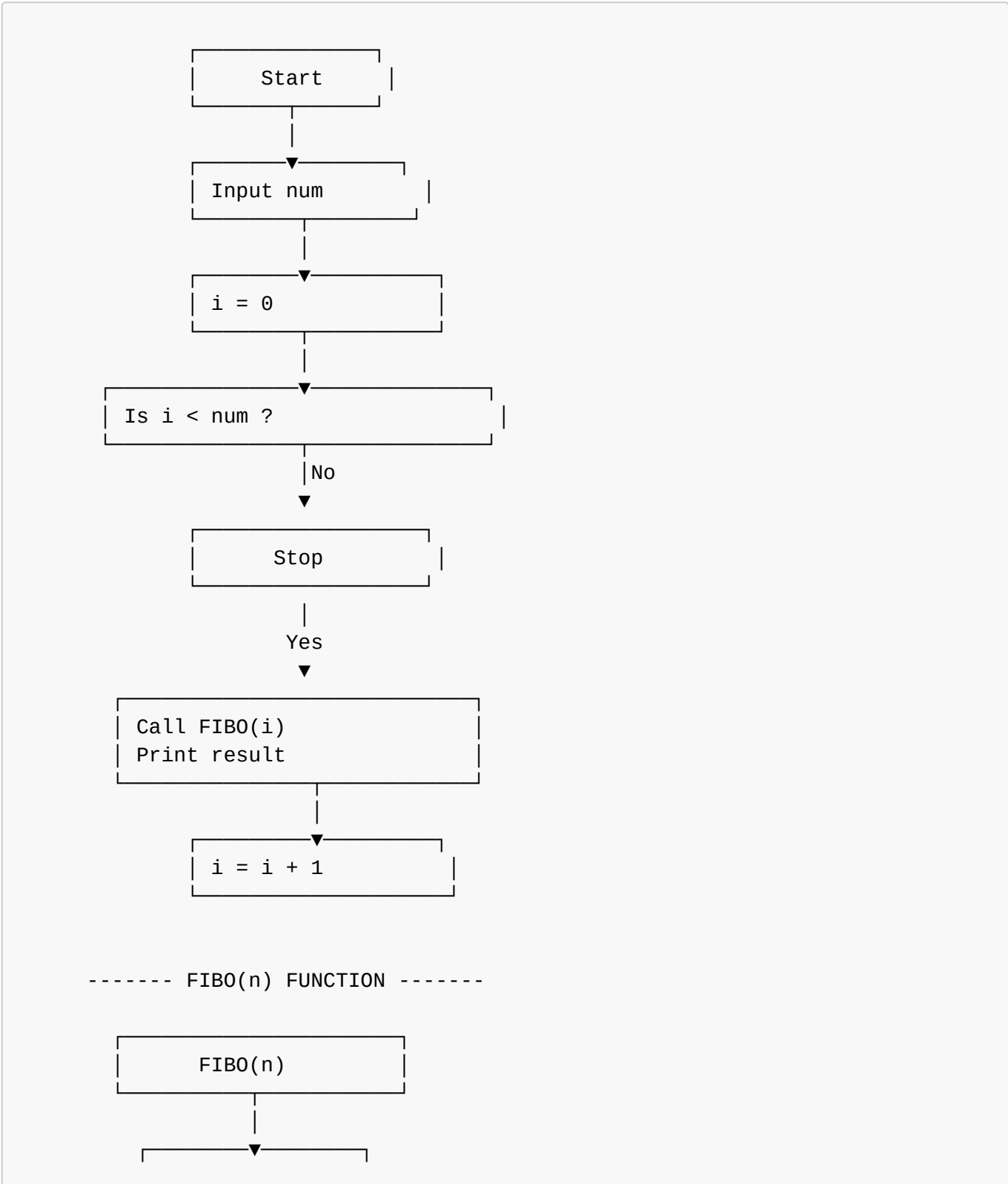
    printf("Fibonacci Sequence up to %d terms:\n", num);
```

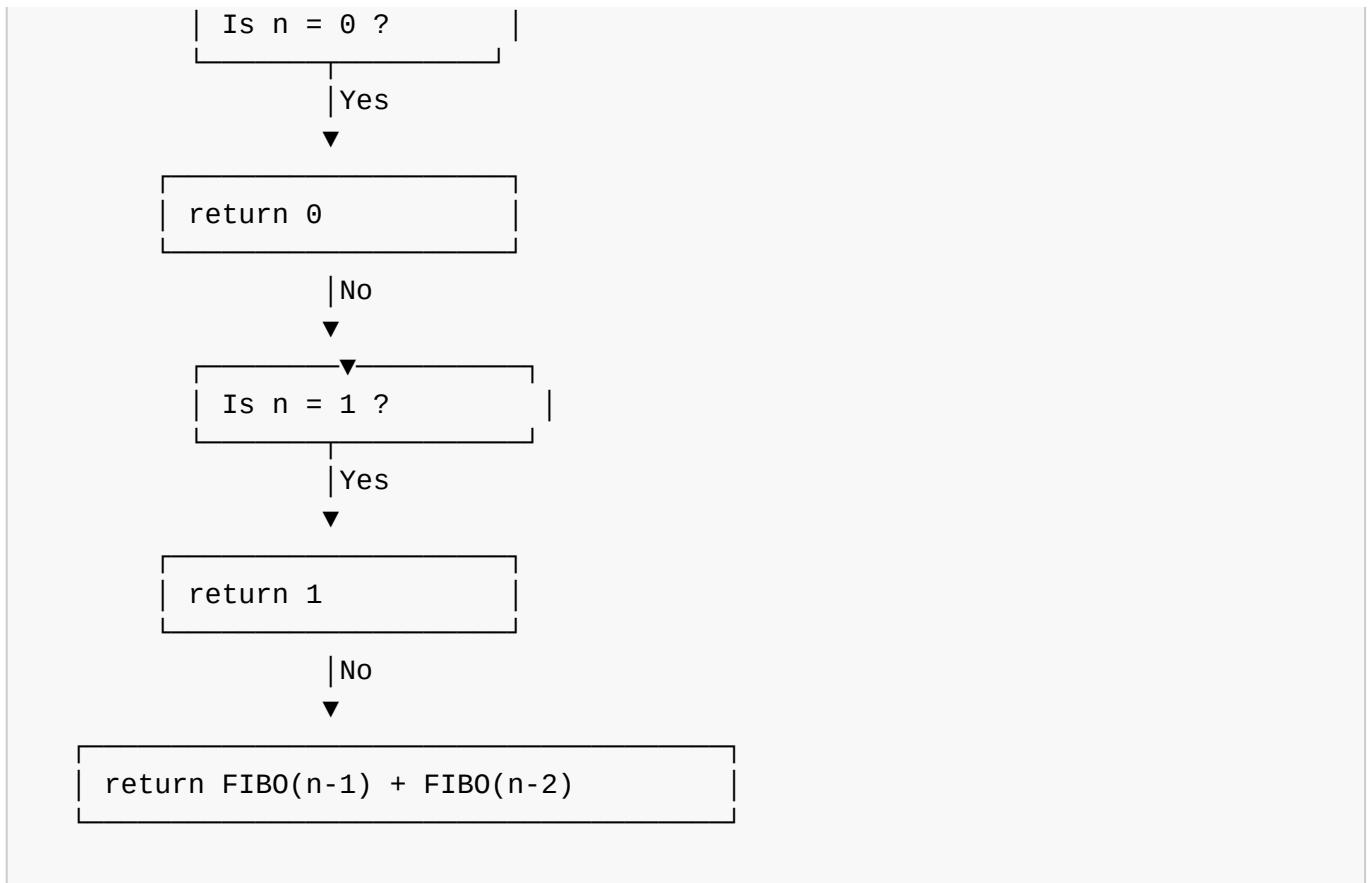
```
for (int i = 0; i < num; i++) {
    printf("%d ", FIB0(i));
}

printf("\n");

return 0;
}
```

Flowchart:





Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q3.c -o exp6Q3 && "/home/aaditya/experiments-in-c/exp6/"exp6Q3
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q3.c -o exp6Q3 && "/home/aaditya/experiments-in-c/exp6/"exp6Q3
Enter the number of terms: 2
Fibonacci Sequence up to 2 terms:
0 1
aaditya@fedora:~/experiments-in-c/exp6$

```

Q4. Develop a C function ISPRIME (num) that accepts an integer argument and returns 1 if the argument is prime, a 0 otherwise. Write a C program that invokes this function to generate prime numbers between the given ranges.

Algorithm:

Step 1: Start Step 2: Read the lower limit start and upper limit end Step 3: For each number i from start to end

Call ISPRIME(i) If function returns 1 → Print i

Step 4: End loop Step 5: Stop Algorithm for ISPRIME(num):

If num ≤ 1, return 0

For i = 2 to num/2 If num % i == 0, return 0

Return 1 (number is prime)

Code:

```
#include <stdio.h>

// Function to check if number is prime
int ISPRIME(int num) {
    if (num <= 1)
        return 0;

    for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0)
            return 0; // Not prime
    }

    return 1; // Prime
}

int main() {
    int start, end;

    printf("Enter the start of range: ");
    scanf("%d", &start);

    printf("Enter the end of range: ");
    scanf("%d", &end);

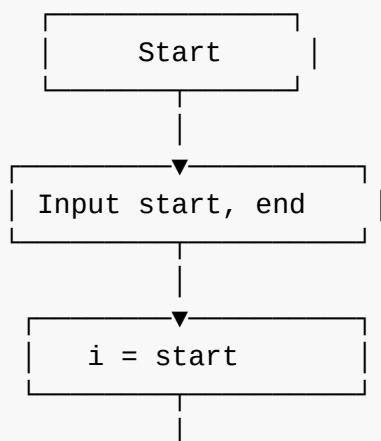
    printf("Prime numbers between %d and %d are:\n", start, end);

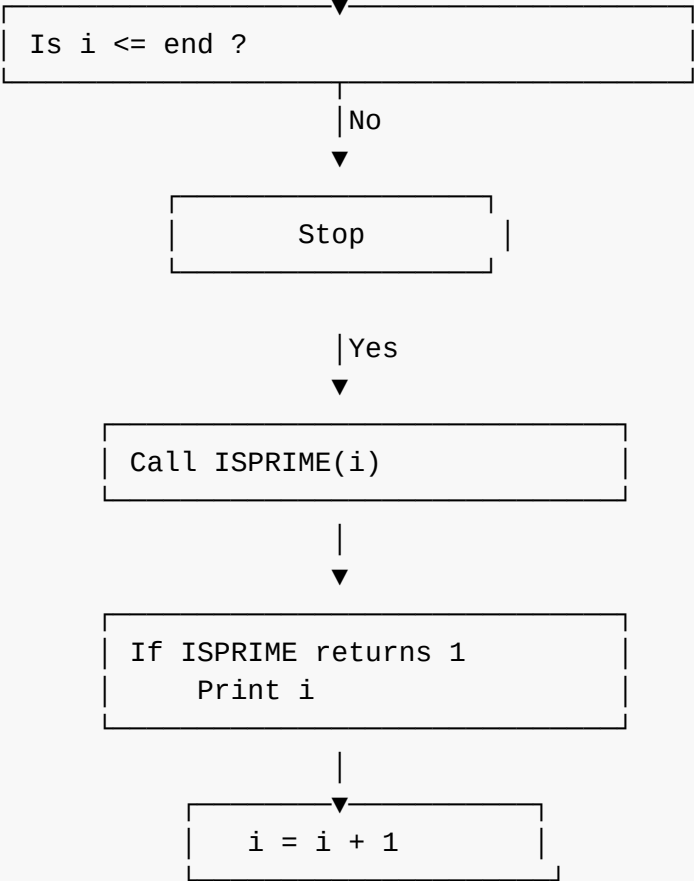
    for (int i = start; i <= end; i++) {
        if (ISPRIME(i))
            printf("%d ", i);
    }

    printf("\n");

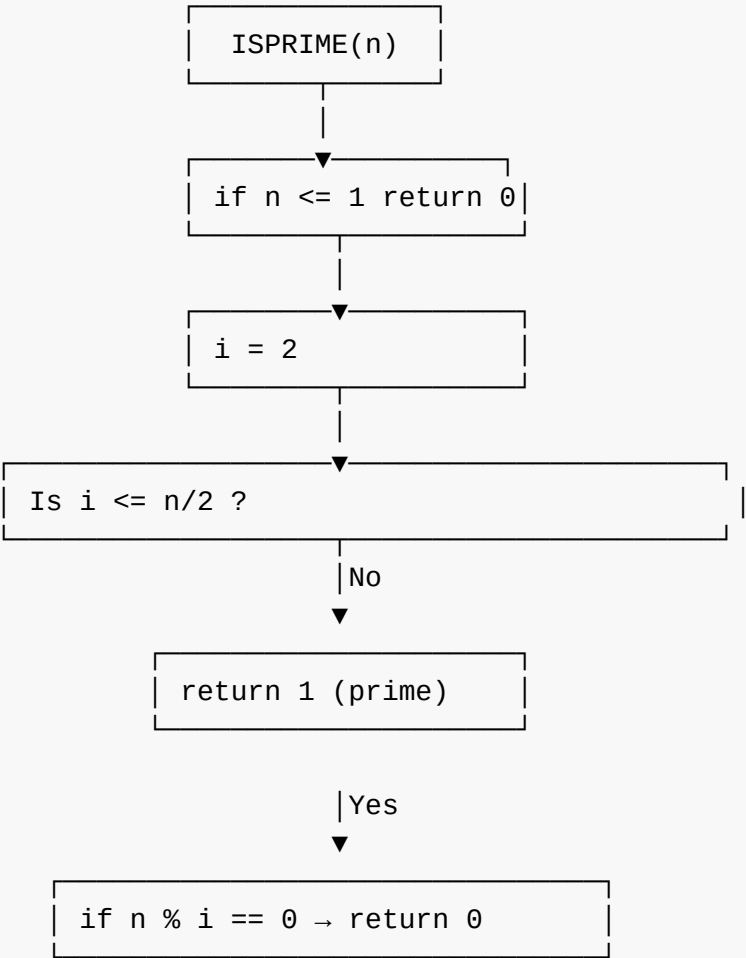
    return 0;
}
```

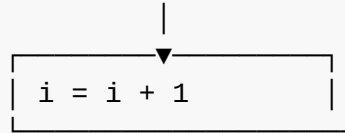
Flowchart:





----- FLOWCHART FOR ISPRIME(num) -----





Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q4.c -o exp6Q4 && "/home/aaditya/experiments-in-c/exp6/"exp6Q4
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q4.c -o exp6Q4 && "/home/aaditya/experiments-in-c/exp6/"exp6Q4
Enter the start of range: 4
Enter the end of range: 2
Prime numbers between 4 and 2 are:

aaditya@fedora:~/experiments-in-c/exp6$ cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q4.c -o exp6Q4 && "/home/aaditya/experiments-in-c/exp6/"exp6Q4
Enter the start of range: 1
Enter the end of range: 10
Prime numbers between 1 and 10 are:
2 3 5 7
aaditya@fedora:~/experiments-in-c/exp6$
  
```

Q5. Develop a function REVERSE (str) that accepts a string argument. Write a C program that invokes this function to find the reverse of a given string.

Algorithm:

Step 1: Start Step 2: Input a string str Step 3: Call the function REVERSE(str) Step 4: Inside REVERSE(str):

Find the length of the string

Swap characters from start and end

Continue until the middle of the string

Step 5: Return the reversed string Step 6: Display the reversed string in main() Step 7: Stop

Code:

```

#include <stdio.h>
#include <string.h>

// Function to reverse a string
void REVERSE(char str[]) {
    int i, j;
    char temp;

    j = strlen(str) - 1; // last index

    for (i = 0; i < j; i++, j--) {
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
    }
}

int main() {
  
```



```

char str[100];

printf("Enter a string: ");
gets(str);  // (Use fgets in modern compilers)

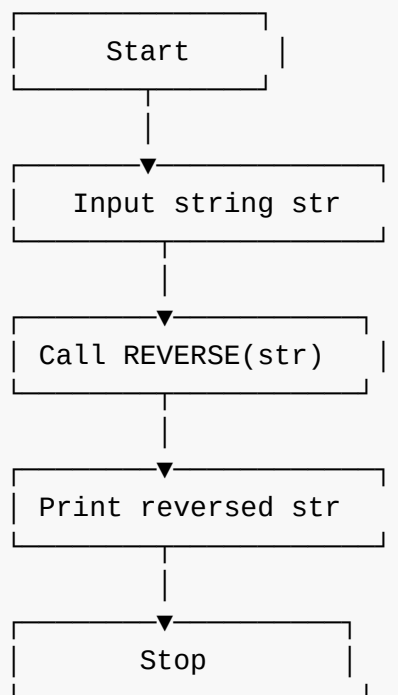
REVERSE(str);  // function call

printf("Reversed string: %s\n", str);

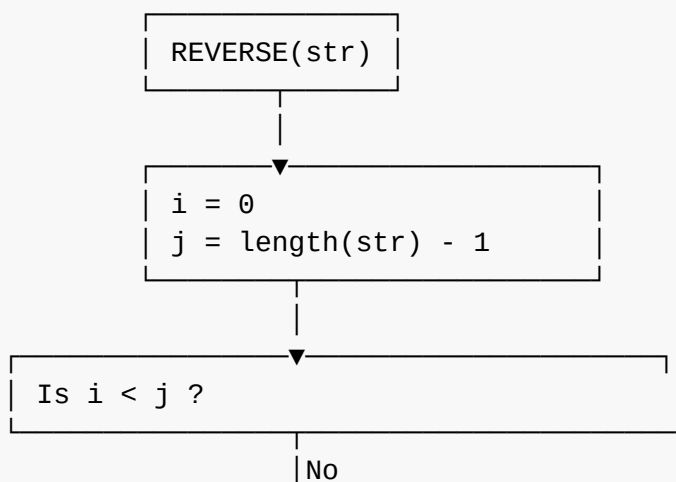
return 0;
}

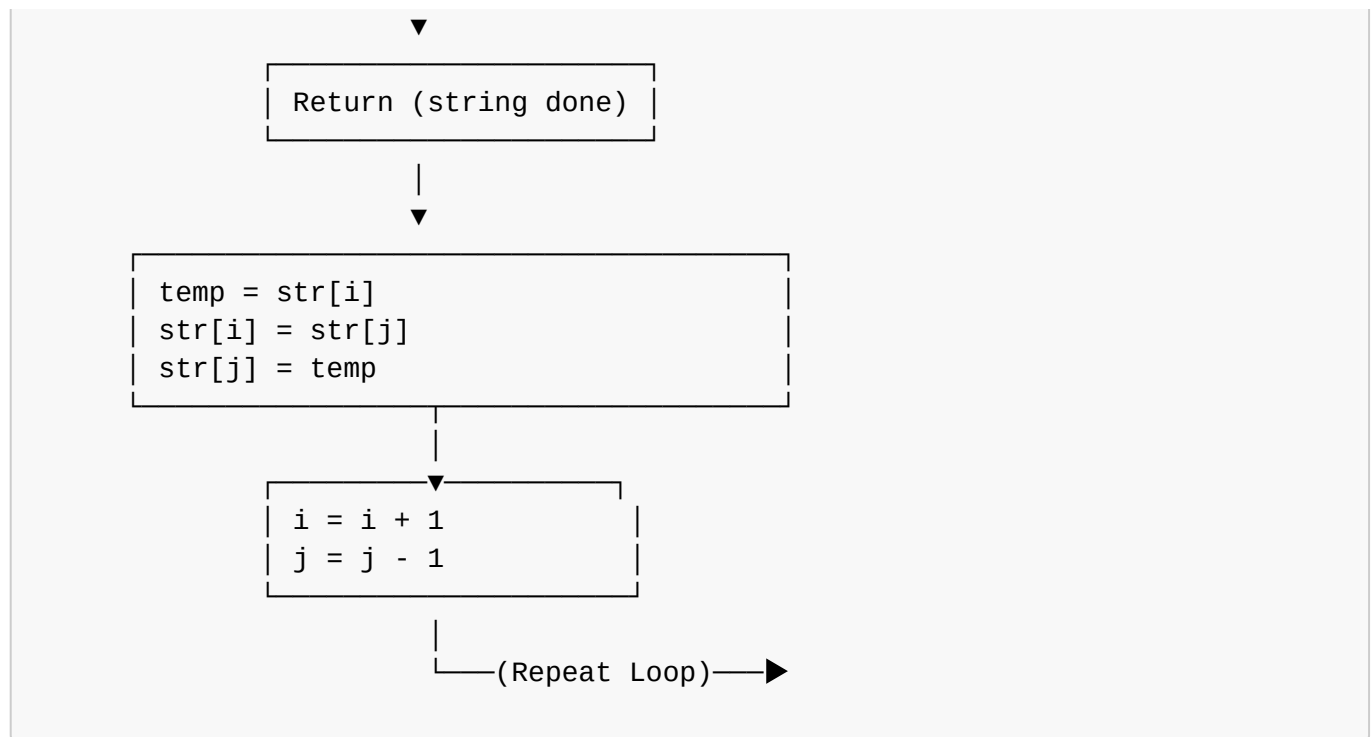
```

Flowchart:



----- FUNCTION: REVERSE(str) -----





Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q5.c -o exp6Q5 && "/home/aaditya/experiments-in-c/exp6/"exp6Q5
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q5.c -o exp6Q5 && "/home/aaditya/experiments-in-c/exp6/"exp6Q5
Enter a string: aditya
Reversed string: ytidA
aaditya@fedora:~/experiments-in-c/exp6$
```

EXPERIMENT:7 Structures and Union

Q1. Write a C program that uses functions to perform the following operations:

- a) Reading a complex number.
- b) Writing a complex number.
- c) Addition and subtraction of two complex numbers

NOTE: represent complex number using a structure.

Algorithm:

Step 1: Start Step 2: Define a structure complex with fields:

- real
- imag

Step 3: Create functions:

- readComplex() → reads a complex number
- *omplex() → displays a complex number
- addComplex() → returns addition
- subComplex() → returns subtraction

Step 4: In main()

- Read two complex numbers c1 and c2

Compute:

- sum = addComplex(c1, c2)
- diff = subComplex(c1, c2)

Display both

Step 5: Stop

Code:

```
#include <stdio.h>

// structure for complex number
struct complex {
    float real;
    float imag;
};
```

```
// function to read a complex number
struct complex readComplex() {
    struct complex c;
    printf("Enter real part: ");
    scanf("%f", &c.real);
    printf("Enter imaginary part: ");
    scanf("%f", &c.imag);
    return c;
}

// function to write a complex number
void writeComplex(struct complex c) {
    if (c.imag >= 0)
        printf("%.2f + %.2fi\n", c.real, c.imag);
    else
        printf("%.2f - %.2fi\n", c.real, -c.imag);
}

// function for addition
struct complex addComplex(struct complex c1, struct complex c2) {
    struct complex result;
    result.real = c1.real + c2.real;
    result.imag = c1.imag + c2.imag;
    return result;
}

// function for subtraction
struct complex subComplex(struct complex c1, struct complex c2) {
    struct complex result;
    result.real = c1.real - c2.real;
    result.imag = c1.imag - c2.imag;
    return result;
}

int main() {
    struct complex c1, c2, sum, diff;

    printf("Enter first complex number:\n");
    c1 = readComplex();

    printf("\nEnter second complex number:\n");
    c2 = readComplex();

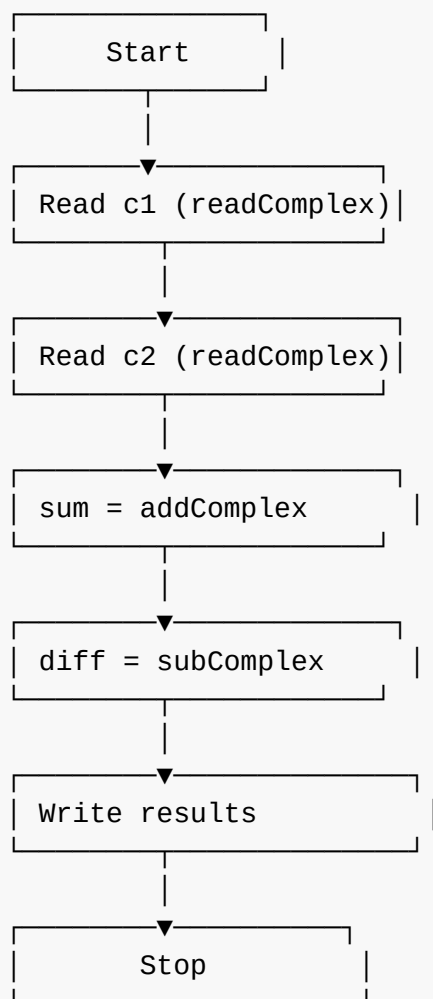
    sum = addComplex(c1, c2);
    diff = subComplex(c1, c2);

    printf("\n--- Results ---\n");
    printf("First number : ");
    writeComplex(c1);

    printf("Second number: ");
    writeComplex(c2);
}
```

```
printf("\nAddition      : ");  
writeComplex(sum);  
  
printf("Subtraction    : ");  
writeComplex(diff);  
  
return 0;  
}
```

Flowchart:



Output:


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp7/" && gcc exp7Q1.c -o exp7Q1 && "/home/aaditya/experiments-in-c/exp7/"exp7Q1
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp7/" && gcc exp7Q1.c -o exp7Q1 && "/home/aaditya/experiments-in-c/exp7/"exp7Q1
Enter first complex number:
Enter real part: 1
Enter imaginary part: 3i

Enter second complex number:
Enter real part: Enter imaginary part: 2

--- Results ---
First number : 1.00 + 3.00i
Second number: 0.00 + 2.00i

Addition      : 1.00 + 5.00i
Subtraction   : 1.00 + 1.00i
aaditya@fedora:~/experiments-in-c/exp7$

```

Q2. Write a C program to compute monthly pay of 100 employees using each employee's name, basic pay. The DA is computed as 52% of the basic pay. Gross Salary (basic pay + DA). Print the employee's name and gross salary.

Algorithm:

Step 1: Start Step 2: Declare variables:

name (string)

basic pay (float)

DA (float)

gross salary (float)

Step 3: Loop from 1 to 100 employees

3.1 Read employee name 3.2 Read basic pay 3.3 Compute DA = $0.52 \times \text{basic pay}$ 3.4 Compute gross salary = basic pay + DA 3.5 Print employee name and gross salary

Step 4: End loop Step 5: Stop

Code:

```

#include <stdio.h>

int main() {
    char name[50];
    float basic, da, gross;

    printf("Enter details for 100 employees:\n\n");

    for (int i = 1; i <= 100; i++) {
        printf("Employee %d:\n", i);

        printf("Enter name: ");
        scanf("%s", name);          // reads name without spaces

        printf("Enter basic pay: ");
        scanf("%f", &basic);
    }
}

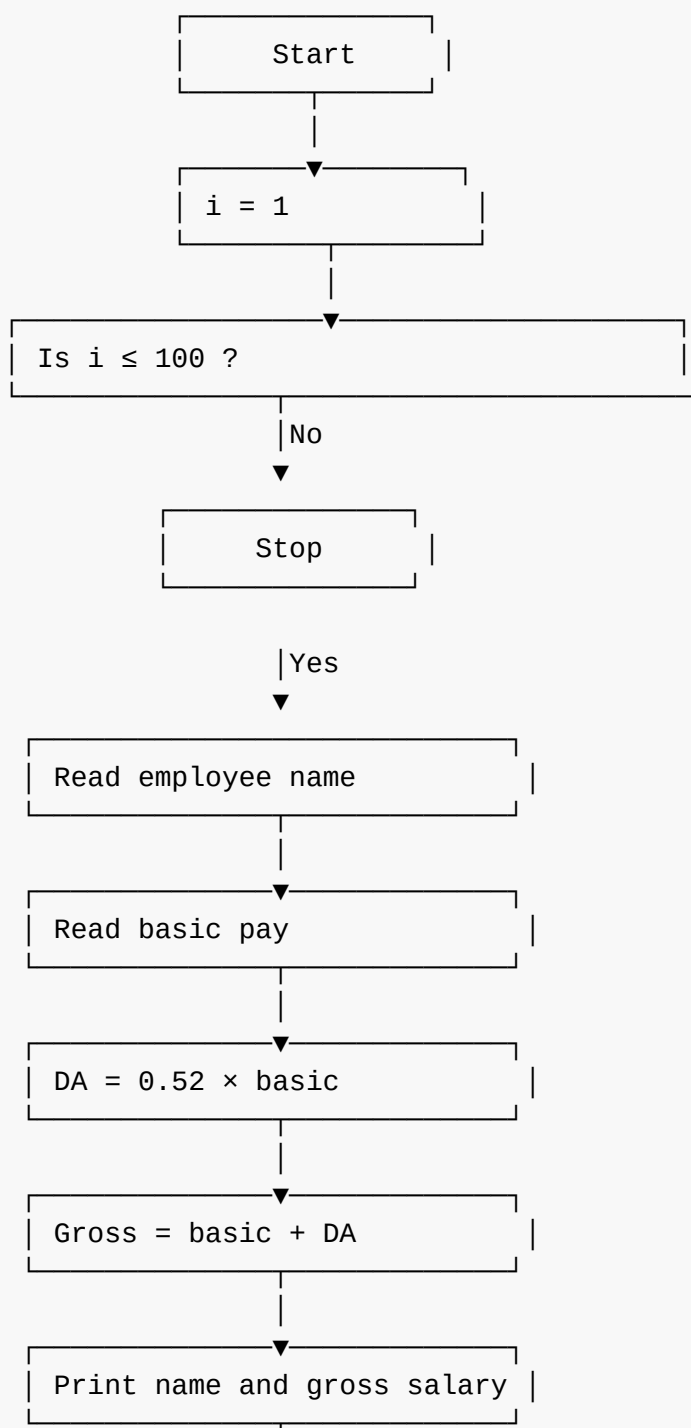
```

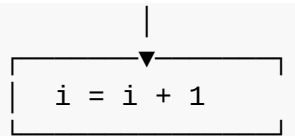
```
    da = 0.52 * basic;           // DA = 52% of basic
    gross = basic + da;          // Gross = basic + DA

    printf("Employee Name : %s\n", name);
    printf("Gross Salary  : %.2f\n\n", gross);
}

return 0;
}
```

Flowchart:





Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp7/" && gcc exp7Q2.c -o exp7Q2 && "/home/aaditya/experiments-in-c/exp7/"exp7Q2
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp7/" && gcc exp7Q2.c -o exp7Q2 && "/home/aaditya/experiments-in-c/exp7/"exp7Q2
Enter details for 100 employees:

Employee 1:
Enter name: ram
Enter basic pay: 23000
Employee Name : ram
Gross Salary : 34960.00

Employee 2:
Enter name: karn
Enter basic pay: 25000
Employee Name : karn
Gross Salary : 38000.00

Employee 3:
Enter name: stuart
Enter basic pay: 56000
Employee Name : stuart
Gross Salary : 85120.00

Employee 4:
Enter name: krish
Enter basic pay: 17890
Employee Name : krish
Gross Salary : 27192.00

Employee 5:
Enter name: krishna
Enter basic pay: 89000
Employee Name : krishna
Gross Salary : 135280.00

Employee 6:
Enter name:
  
```

Q3.Create a book structure containing book_id,title,author name and price.Write a C program to pass structure as a function argument and print the details.

Algorithm:

Step 1: Start Step 2: Define a structure Book with fields:

- book_id (int)
- title (string)
- author (string)
- price (float)

Step 3: Create a function displayBook(Book b) to print the details. Step 4: In main()

- Declare a variable of type Book
- Input book details (id, title, author, price)
- Call displayBook(b) by passing structure as argument

End program

Step 5: Stop

Code:

```
#include <stdio.h>

// structure definition
struct Book {
    int book_id;
    char title[50];
    char author[50];
    float price;
};

// function to display book details (structure as argument)
void displayBook(struct Book b) {
    printf("\n--- Book Details ---\n");
    printf("Book ID    : %d\n", b.book_id);
    printf("Title       : %s\n", b.title);
    printf("Author      : %s\n", b.author);
    printf("Price       : %.2f\n", b.price);
}

int main() {
    struct Book b;

    printf("Enter Book ID: ");
    scanf("%d", &b.book_id);

    printf("Enter Book Title: ");
    scanf("%s", b.title);

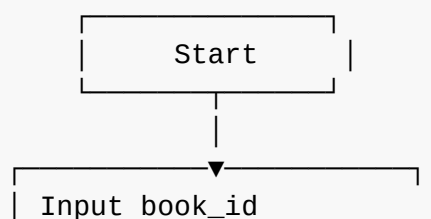
    printf("Enter Author Name: ");
    scanf("%s", b.author);

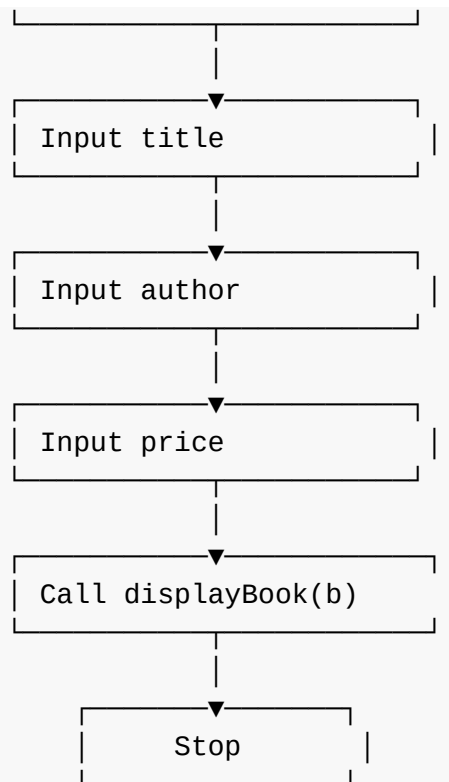
    printf("Enter Price: ");
    scanf("%f", &b.price);

    // pass structure to function
    displayBook(b);

    return 0;
}
```

Flowchart:





Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp7/" && gcc exp7Q3.c -o exp7Q3 && "/home/aaditya/experiments-in-c/exp7/"exp7Q3
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp7/" && gcc exp7Q3.c -o exp7Q3 && "/home/aaditya/experiments-in-c/exp7/"exp7Q3
Enter Book ID: 1098
Enter Book Title: harry potter
Enter Author Name: Enter Price: jk rowling

--- Book Details ---
Book ID   : 1098
Title     : harry
Author    : potter
Price     : 0.00
aaditya@fedora:~/experiments-in-c/exp7$
  
```

Q4.Create a Union containing 6 strings : name,home_address,hostel_address,city ,state and zip.Write a C program to display your present address.

Algorithm:

Step 1: Start Step 2: Define a union Address with members:

name

home_address

hostel_address

city

state

zip

Step 3: Declare a variable of union type (A). Step 4: Copy a string into A.name using strcpy()

Print the value

Step 5: Copy a string into A.home_address

Print the value

Step 6: Copy a string into A.hostel_address

Print

Step 7: Copy a string into A.city

Print

Step 8: Copy a string into A.state

Print

Step 9: Copy a string into A.zip

Print

Step 10: Stop

Code:

```
#include <stdio.h>
#include <string.h>

union Address {
    char name[50];
    char home_address[100];
    char hostel_address[100];
    char city[50];
    char state[50];
    char zip[10];
};

int main() {
    union Address A;

    printf("---- Present Address ----\n");

    strcpy(A.name, "Aaditya Mishra");
    printf("Name          : %s\n", A.name);

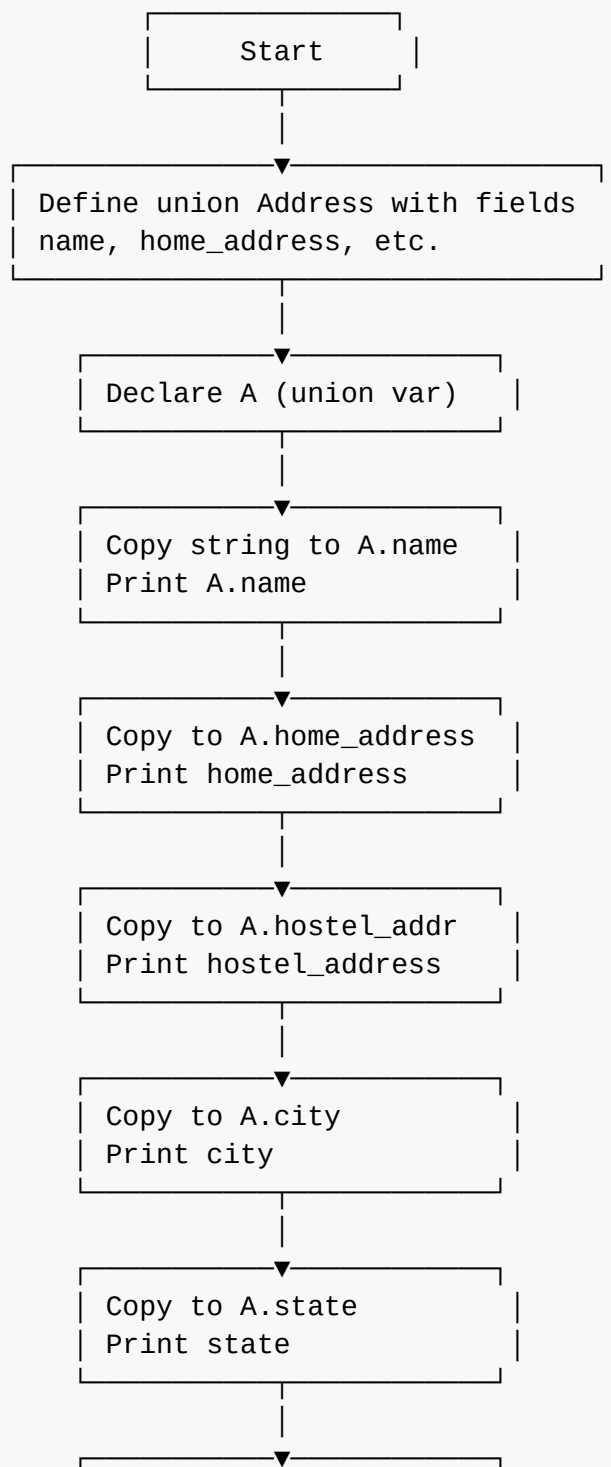
    strcpy(A.home_address, "123, MG Road");
    printf("Home Address   : %s\n", A.home_address);

    strcpy(A.hostel_address, "Hostel Block B");
    printf("Hostel Address: %s\n", A.hostel_address);

    strcpy(A.city, "Ranchi");
    printf("City           : %s\n", A.city);
}
```

```
strcpy(A.state, "Jharkhand");  
printf("State      : %s\n", A.state);  
  
strcpy(A.zip, "834001");  
printf("ZIP Code   : %s\n", A.zip);  
  
return 0;  
}
```

Flowchart:



```
Copy to A.zip
Print ZIP code
```

```
Stop
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp7/" && gcc exp7Q4.c -o exp7Q4 && "/home/aaditya/experiments-in-c/exp7/"exp7Q4
• aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp7/" && gcc exp7Q4.c -o exp7Q4 && "/home/aaditya/experiments-in-c/exp7/"exp7Q4
---- Present Address ----
Name       : Aaditya Mishra
Home Address : 123, MG Road
Hostel Address: Hostel Block B
City       : Ranchi
State      : Jharkhand
ZIP Code   : 834001
o aaditya@fedora:~/experiments-in-c/exp7$
```

EXPERIMENT:8 POINTERS

Q1.Declare different type of pointers(int,float,char)and initialize them with the addresses of variables.Print the values of both the pointers and the variable they point to.

Algorithm:

Step 1: Start Step 2: Declare variables:

- an integer variable a
- a float variable b
- a char variable c

Step 3: Declare pointers:

- int *p1
- float *p2
- char *p3

Step 4: Initialize pointers with addresses:

- p1 = &a
- p2 = &b
- p3 = &c

Step 5: Print pointer addresses Step 6: Print the values using:

- the variables themselves
- dereferencing the pointers (*p1, *p2, *p3)

Step 7: Stop

code:

```
#include <stdio.h>

int main() {
    int a = 10;
    float b = 20.5;
    char c = 'A';

    int *p1 = &a;
    float *p2 = &b;
    char *p3 = &c;

    printf("----- Pointer Demonstration -----\\n");
```

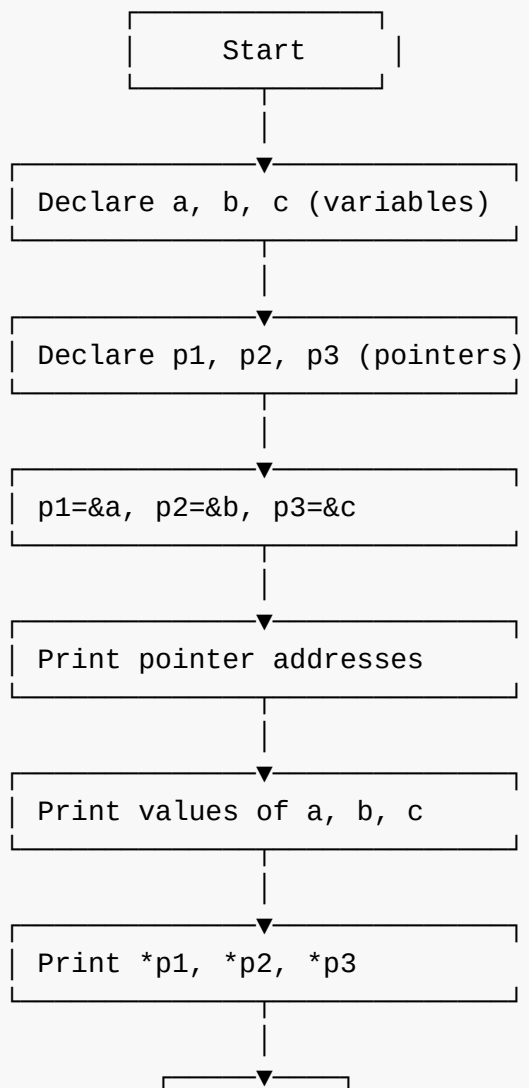
```
printf("\nInteger:\n");
printf("Address stored in p1 = %p\n", p1);
printf("Value of a = %d\n", a);
printf("Value through pointer *p1 = %d\n", *p1);

printf("\nFloat:\n");
printf("Address stored in p2 = %p\n", p2);
printf("Value of b = %.2f\n", b);
printf("Value through pointer *p2 = %.2f\n", *p2);

printf("\nCharacter:\n");
printf("Address stored in p3 = %p\n", p3);
printf("Value of c = %c\n", c);
printf("Value through pointer *p3 = %c\n", *p3);

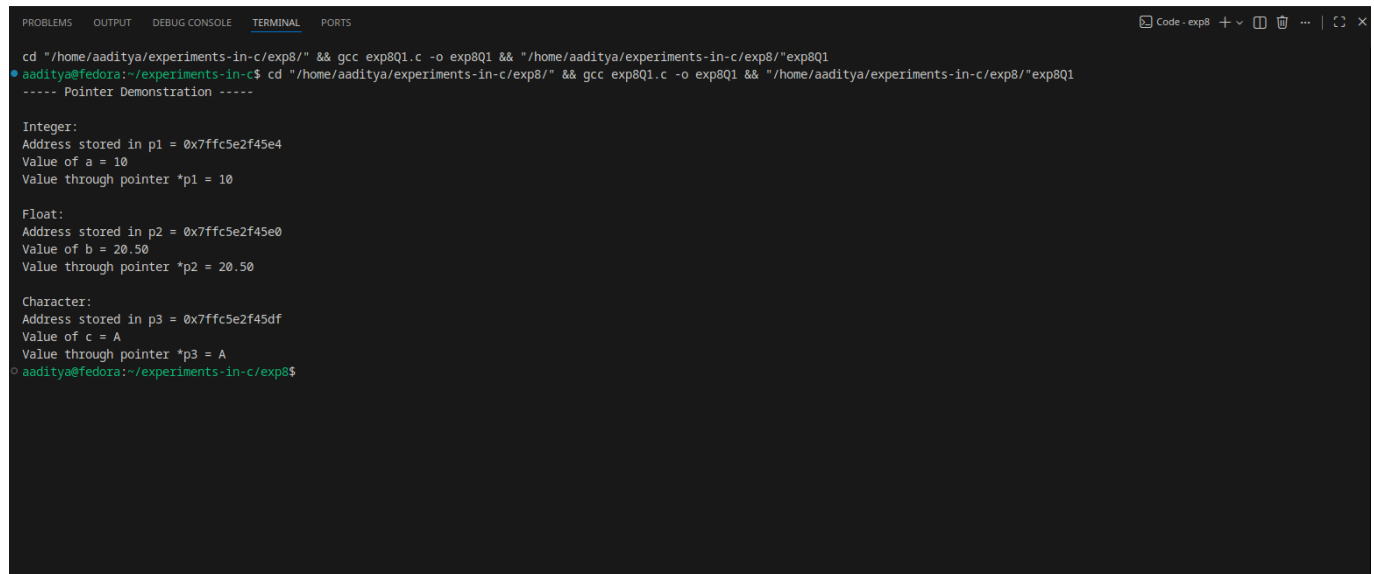
return 0;
}
```

Flowchart:



Stop

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp8/" && gcc exp8Q1.c -o exp8Q1 && "/home/aaditya/experiments-in-c/exp8/"exp8Q1
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp8/" && gcc exp8Q1.c -o exp8Q1 && "/home/aaditya/experiments-in-c/exp8/"exp8Q1
----- Pointer Demonstration -----

Integer:
Address stored in p1 = 0x7ffc5e2f45e4
Value of a = 10
Value through pointer *p1 = 10

Float:
Address stored in p2 = 0x7ffc5e2f45e0
Value of b = 20.50
Value through pointer *p2 = 20.50

Character:
Address stored in p3 = 0x7ffc5e2f45df
Value of c = A
Value through pointer *p3 = A
aaditya@fedora:~/experiments-in-c/exp8$
```

Q2.Perform pointer arithmetic (increament and decreament)on pointers of different data types.observe how the memory addresses changes and effects on data access.

Algorithm:

Step 1: Start Step 2: Declare variables:

- `int a = 10`
- `float b = 5.5`
- `char c = 'X'`

Step 3: Declare pointers:

- `int *p1 = &a`
- `float *p2 = &b`
- `char *p3 = &c`

Step 4: Print initial pointer values (addresses) Step 5: Perform pointer increment:

- `p1++`
- `p2++`
- `p3++`
- Print new addresses.

Step 6: Perform pointer decrement:

- p1--
- p2--
- p3-- Print new addresses.

Step 7: End Program

Code:

```
#include <stdio.h>

int main() {
    int a = 10;
    float b = 5.5;
    char c = 'X';

    int *p1 = &a;
    float *p2 = &b;
    char *p3 = &c;

    printf("Initial Addresses:\n");
    printf("p1 = %p (int)\n", p1);
    printf("p2 = %p (float)\n", p2);
    printf("p3 = %p (char)\n", p3);

    // Increment
    p1++;
    p2++;
    p3++;

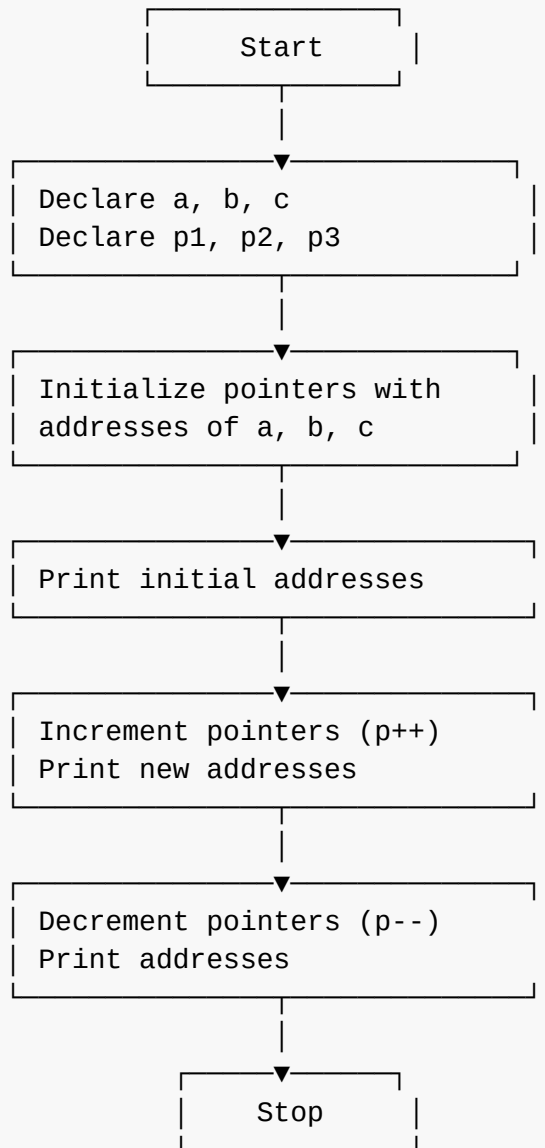
    printf("\nAfter Increment:\n");
    printf("p1 = %p (+ %lu bytes)\n", p1, sizeof(int));
    printf("p2 = %p (+ %lu bytes)\n", p2, sizeof(float));
    printf("p3 = %p (+ %lu bytes)\n", p3, sizeof(char));

    // Decrement
    p1--;
    p2--;
    p3--;

    printf("\nAfter Decrement (Back to Original):\n");
    printf("p1 = %p\n", p1);
    printf("p2 = %p\n", p2);
    printf("p3 = %p\n", p3);

    return 0;
}
```

Flowchart:



Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp8/" && gcc exp8Q2.c -o exp8Q2 && "/home/aaditya/experiments-in-c/exp8/"exp8Q2
• aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp8/" && gcc exp8Q2.c -o exp8Q2 && "/home/aaditya/experiments-in-c/exp8/"exp8Q2
Initial Addresses:
p1 = 0x7ffd468d8a24 (int)
p2 = 0x7ffd468d8a20 (float)
p3 = 0x7ffd468d8a1f (char)

After Increment:
p1 = 0x7ffd468d8a28 (+ 4 bytes)
p2 = 0x7ffd468d8a24 (+ 4 bytes)
p3 = 0x7ffd468d8a20 (+ 1 bytes)

After Decrement (Back to Original):
p1 = 0x7ffd468d8a24
p2 = 0x7ffd468d8a20
p3 = 0x7ffd468d8a1f
• aaditya@fedora:~/experiments-in-c/exp8$

```

Q3. Write a function that accepts pointers as parameters. pass variables by reference using pointers and modify their values within function.

Algorithm:

Step 1: Start Step 2: Declare two integer variables a and b. Step 3: Read values of a and b from the user. Step 4: Call a function modify(&a, &b) and pass the addresses of variables. Inside modify(x, y):

Add 10 to the value pointed by x

Multiply the value pointed by y by 2

Step 5: Return to main() Step 6: Print modified values of a and b. Step 7: Stop

Code:

```
#include <stdio.h>

// function that modifies values using pointers
void modify(int *x, int *y) {
    *x = *x + 10;    // add 10 to value of x
    *y = *y * 2;     // multiply y by 2
}

int main() {
    int a, b;

    printf("Enter value of a: ");
    scanf("%d", &a);

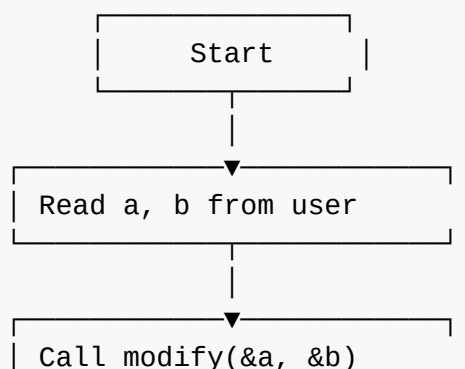
    printf("Enter value of b: ");
    scanf("%d", &b);

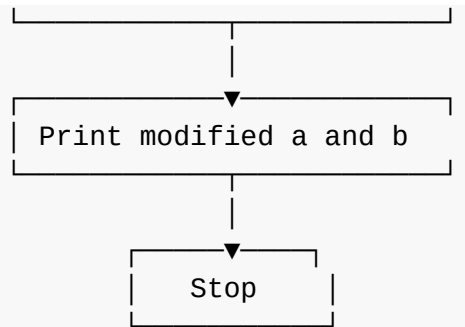
    // calling function with addresses
    modify(&a, &b);

    printf("\nAfter modification:\n");
    printf("a = %d\n", a);
    printf("b = %d\n", b);

    return 0;
}
```

Flowchart:





Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
cd "/home/aaditya/experiments-in-c/exp8/" && gcc exp8Q3.c -o exp8Q3 && "/home/aaditya/experiments-in-c/exp8/"exp8Q3
• aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp8/" && gcc exp8Q3.c -o exp8Q3 && "/home/aaditya/experiments-in-c/exp8/"exp8Q3
Enter value of a: 5
Enter value of b: 4

After modification:
a = 15
b = 8
o aaditya@fedora:~/experiments-in-c/exp8$
```

EXPERIMENT:9 FILE HANDLING IN C

Q1. Write a program to create a new file and write text into it.

Algorithm:

1* Start

2* Declare a file pointer FILE *fp.

3* Open a file in write mode using: fp = fopen("output.txt", "w");

4* If file opening fails (fp == NULL), show an error message and exit.

5* Ask the user to enter text.

6* Read the text from the user.

7* Write the text into the file using fprintf() or fputs().

8* Close the file using fclose(fp).

9* Display a message that the file has been written successfully.

10* Stop

Code:

```
#include <stdio.h>

int main() {
    FILE *fp;
    char text[200];

    // Open file in write mode
    fp = fopen("output.txt", "w");

    // Check if file opened successfully
    if (fp == NULL) {
        printf("Error! Could not create file.\n");
        return 1;
    }

    // Get text from user
    printf("Enter text to write into the file:\n");
    fgets(text, sizeof(text), stdin);

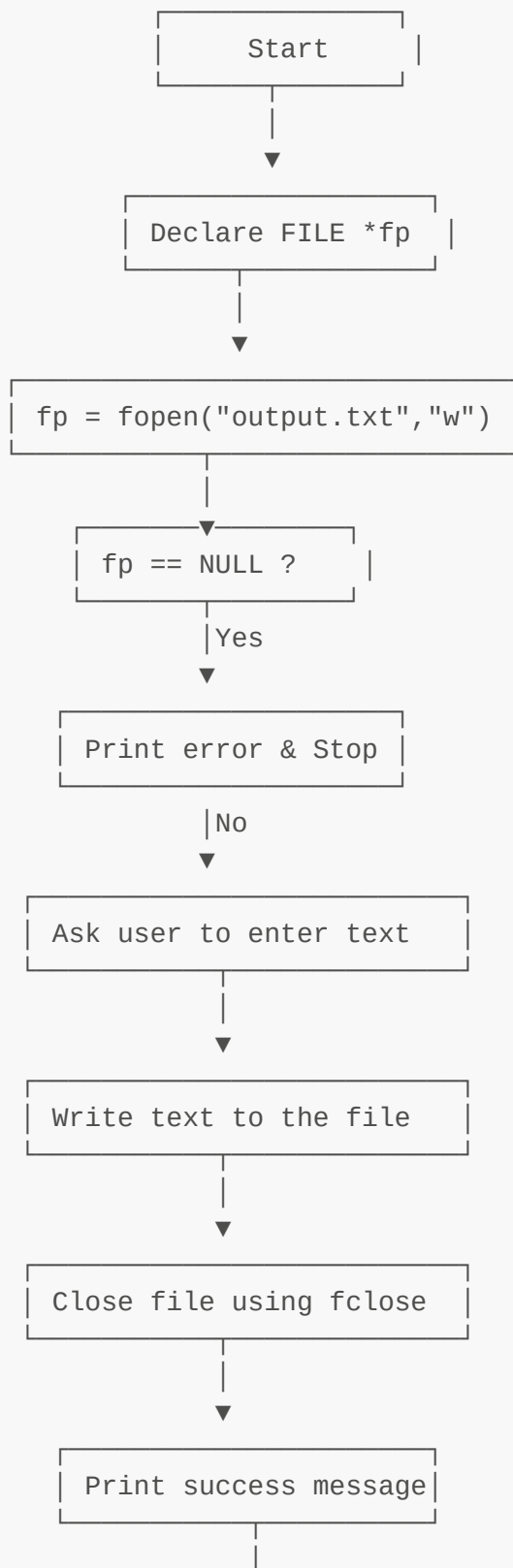
    // Write text into file
    fprintf(fp, "%s", text);

    // Close file
    fclose(fp);
}
```



```
    printf("File written successfully!\n");  
  
    return 0;  
}
```

Flowchart:





Output:

```

cd "/home/aaditya/experiments-in-c/exp9/" && gcc exp9Q1.c -o exp9Q1 && "/home/aaditya/experiments-in-c/exp9/"exp9Q1
• aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp9/" && gcc exp9Q1.c -o exp9Q1 && "/home/aaditya/experiments-in-c/exp9/"exp9Q1
Enter text to write into the file:
hello myself aditya
File written successfully!
• aaditya@fedora:~/experiments-in-c/exp9$ █
  
```

Q2.Open an existing file and read its content character by character and then close the file.

Flowchart:

- 1-Start
- 2-Declare a file pointer FILE *fp.
- 3-Open the file in read mode using: fp = fopen("input.txt", "r");
- 4-If the file fails to open (fp == NULL), display an error message and stop.
- 5-Declare a character variable ch.
- 6-Read a character from the file using: ch = fgetc(fp)
- 7-While ch != EOF
- 8-Display the character on the screen
- 9-Read the next character
- 10-After reading all characters, close the file using fclose(fp).
- 11-Stop

Code:

```

#include <stdio.h>

int main() {
    FILE *fp;
    char ch;

    // Open the file in read mode
  
```

```
fp = fopen("input.txt", "r");

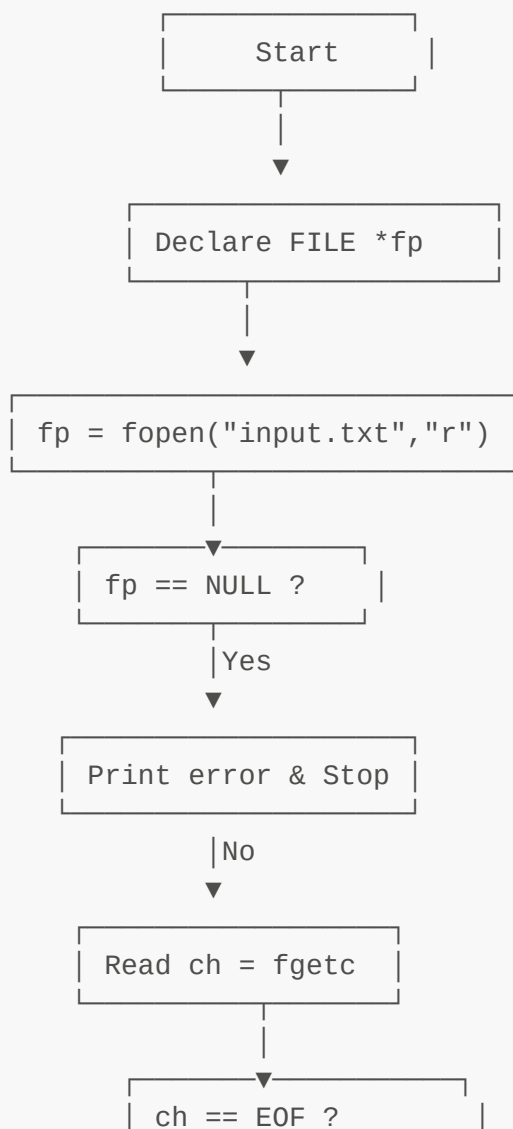
// Check if file opened successfully
if (fp == NULL) {
    printf("Error! Could not open file.\n");
    return 1;
}

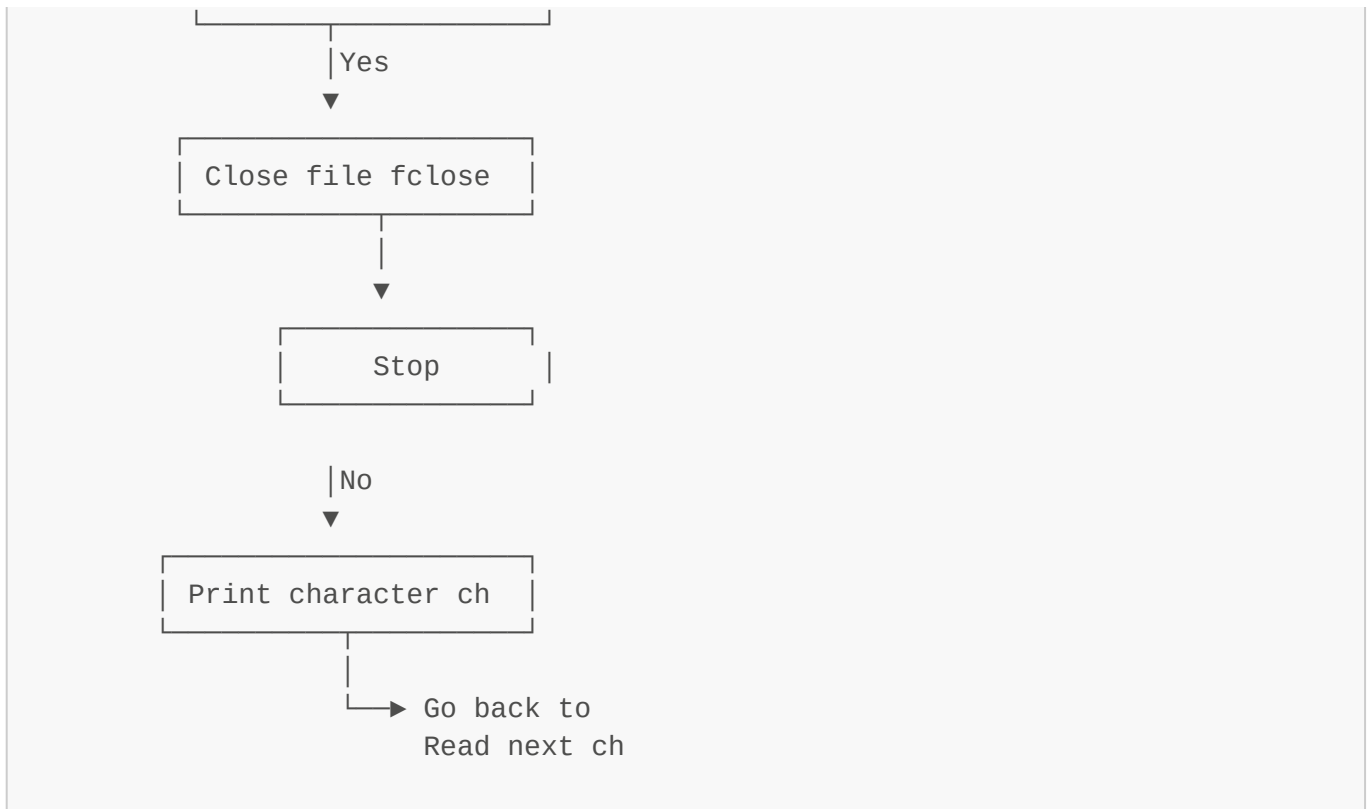
// Read and display characters until EOF
while ((ch = fgetc(fp)) != EOF) {
    putchar(ch);
}

// Close the file
fclose(fp);

return 0;
}
```

Flowchart:





Output:

```

cd "/home/aaditya/experiments-in-c/exp9/" && gcc exp9Q2.c -o exp9Q2 && "/home/aaditya/experiments-in-c/exp9/"exp9Q2
• aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp9/" && gcc exp9Q2.c -o exp9Q2 && "/home/aaditya/experiments-in-c/exp9/"exp9Q2
hello myself aditya
• aaditya@fedora:~/experiments-in-c/exp9$
  
```

Q3.Open a file read its content line by line and display each line on the console:

Algorithm:

- 1-Start
- 2-Declare a file pointer FILE *fp.
- 3-Declare a string buffer (e.g., char line[200] 😊).
- 4-Open the file in read mode using: fp = fopen("input.txt", "r");
- 5-If fp == NULL, print an error message and stop.
- 6-Read a line using: fgets(line, sizeof(line), fp)
- 7-While the line is not NULL:
- 8-Display the line on the console using printf("%s", line);
- 9-Read the next line using fgets()

10-Close the file using fclose(fp).

11-Stop

Code:

```
#include <stdio.h>

int main() {
    FILE *fp;
    char line[200];

    // Open file in read mode
    fp = fopen("input.txt", "r");

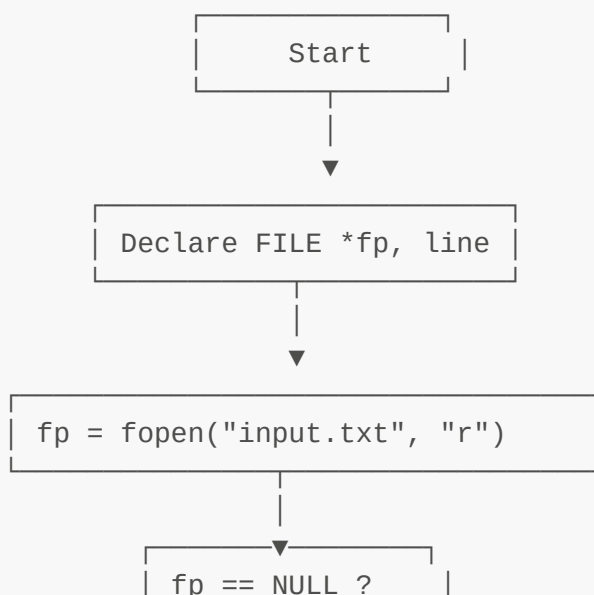
    if (fp == NULL) {
        printf("Error! Could not open file.\n");
        return 1;
    }

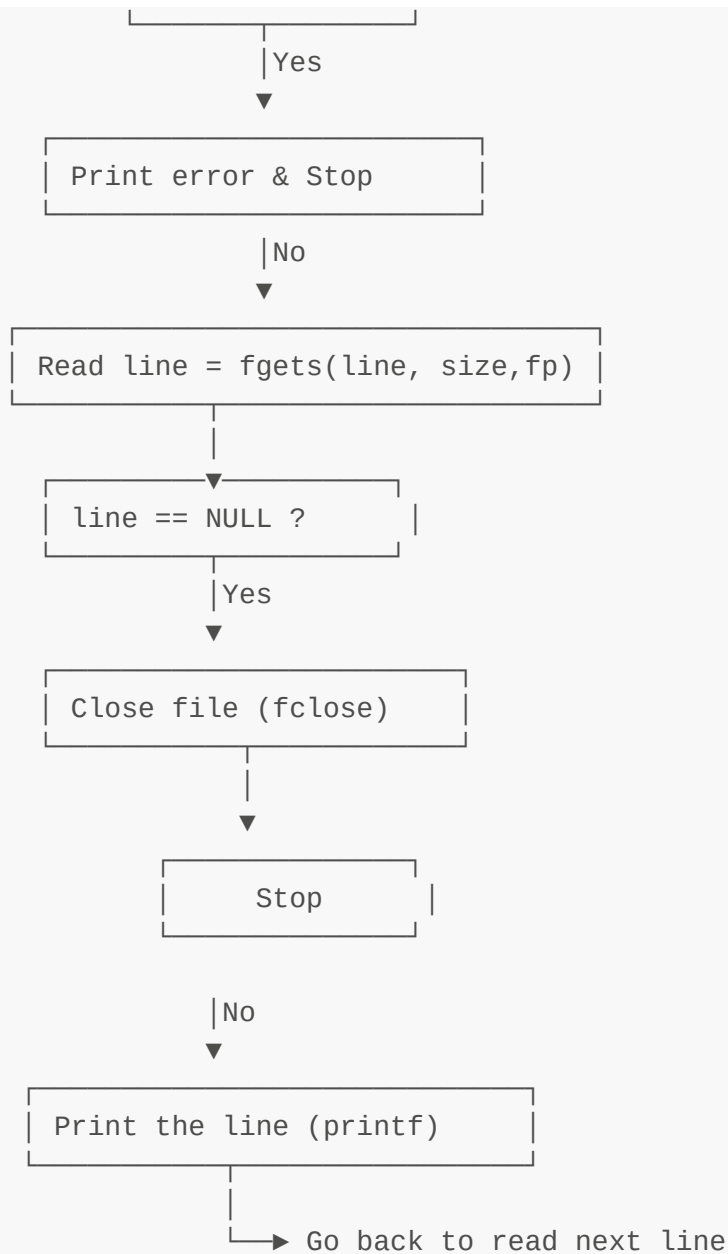
    // Read and display each line
    while (fgets(line, sizeof(line), fp) != NULL) {
        printf("%s", line);
    }

    // Close the file
    fclose(fp);

    return 0;
}
```

Flowchart:





Output:

```
cd "/home/aaditya/experiments-in-c/exp9/" && gcc exp9Q3.c -o exp9Q3 && "/home/aaditya/experiments-in-c/exp9/"exp9Q3
• aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp9/" && gcc exp9Q3.c -o exp9Q3 && "/home/aaditya/experiments-in-c/exp9/"exp9Q3
hello myself aditya
◦ aaditya@fedora:~/experiments-in-c/exp9$
```


EXPERIMENT:10 DYNAMIC MEMORY ALLOCATION

Q1. Write a program to create a simple linked list in C using pointer and structure.

Algorithm:

Goal: create a singly linked list, insert nodes at the end, display list contents, then free memory.

1-Start.

2-Initialize head = NULL.

3-Repeat while user wants to add node:

4-Allocate memory for a new node (malloc).

5-Read data into new_node->data.

6-Set new_node->next = NULL.

7-If head == NULL, set head = new_node (list was empty).

8-Else traverse from head to the last node and set last->next = new_node.

9-After insertion finished, traverse list from head and print each node's data.

10-Free all nodes (traverse and free).

11-End.

Code:

```
#include <stdio.h>
#include <stdlib.h>

/* Node structure */
typedef struct Node {
    int data;
    struct Node *next;
} Node;

/* Function to create a new node with given value */
Node* create_node(int value) {
    Node *new_node = (Node*)malloc(sizeof(Node));
    if (new_node == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        exit(EXIT_FAILURE);
    }
    new_node->data = value;
    new_node->next = NULL;
    return new_node;
}
```

```
/* Insert at the end of the list */
void insert_end(Node **head_ref, int value) {
    Node *new_node = create_node(value);
    if (*head_ref == NULL) {
        *head_ref = new_node;
        return;
    }
    Node *temp = *head_ref;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
}

/* Display the list */
void display_list(Node *head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Linked list contents: ");
    Node *temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" -> ");
        temp = temp->next;
    }
    printf("\n");
}

/* Free all nodes */
void free_list(Node **head_ref) {
    Node *current = *head_ref;
    Node *next;
    while (current != NULL) {
        next = current->next;
        free(current);
        current = next;
    }
    *head_ref = NULL;
}

int main(void) {
    Node *head = NULL;
    int value;
    char choice;

    printf("Simple Singly Linked List (insert at end)\n");

    do {
        printf("Enter an integer to insert: ");
        if (scanf("%d", &value) != 1) {
            // invalid input handling - clear stdin
            fprintf(stderr, "Invalid input. Exiting.\n");
        }
        insert_end(&head, value);
        display_list(head);
        printf("Do you want to continue? (y/n): ");
        choice = getchar();
        while (choice != 'y' & choice != 'n')
            choice = getchar();
    } while (choice == 'y');
```

```

        free_list(&head);
        return EXIT_FAILURE;
    }
    insert_end(&head, value);

    // ask user whether to continue
    printf("Add another node? (y/n): ");
    // consume leftover newline, then read char
    while ((choice = getchar()) != EOF && (choice == '\n')) { /* skip
*/ }

    if (choice == EOF) choice = 'n';
    // if the user typed more than one char, only first non-newline is
used
    // normalize to lowercase
    if (choice >= 'A' && choice <= 'Z') choice = choice - 'A' + 'a';

    // If we got a char that is not newline, there may still be more
input on the line,
    // clear until newline
    int c;
    while ((c = getchar()) != '\n' && c != EOF) { /* flush rest */ }

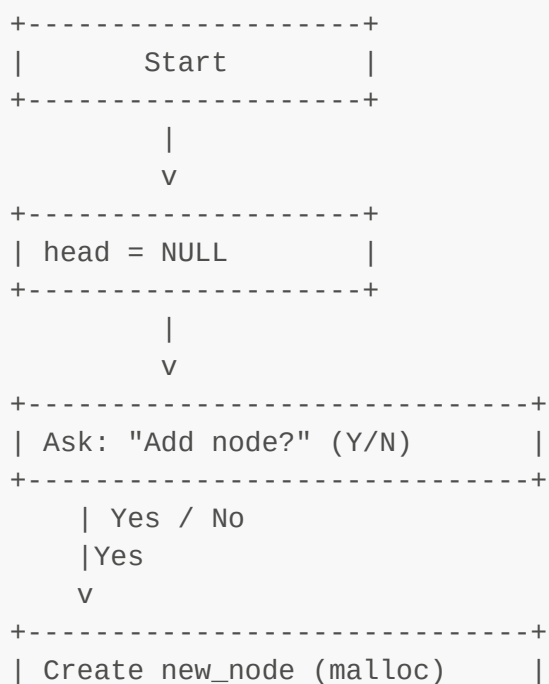
    } while (choice == 'y');

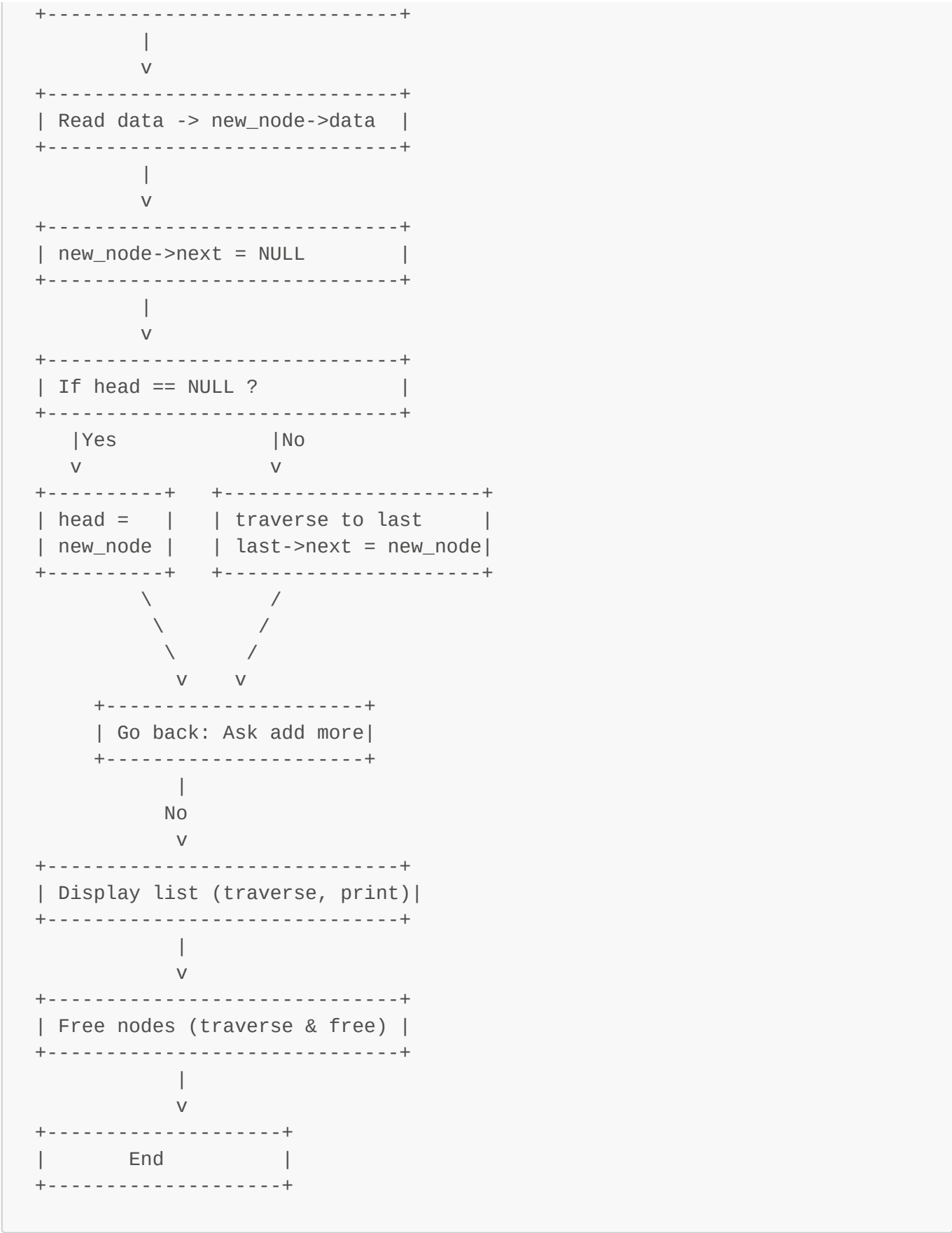
    printf("\n");
    display_list(head);

    free_list(&head);
    return 0;
}

```

Flowchart:





Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Code - exp10 + ▾ 🗑️ ... | 🔍 ✕

cd "/home/aaditya/experiments-in-c/exp10/" && gcc exp10Q1.c -o exp10Q1 && "/home/aaditya/experiments-in-c/exp10/"exp10Q1
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp10/" && gcc exp10Q1.c -o exp10Q1 && "/home/aaditya/experiments-in-c/exp10/"exp10Q1
Simple Singly Linked List (insert at end)
Enter an integer to insert: 1 2 3
Add another node? (y/n):
Linked list contents: 1
aaditya@fedora:~/experiments-in-c/exp10$ cd "/home/aaditya/experiments-in-c/exp10/" && gcc exp10Q1.c -o exp10Q1 && "/home/aaditya/experiments-in-c/exp10/"exp10Q1
Simple Singly Linked List (insert at end)
Enter an integer to insert: 10
Add another node? (y/n): y
Enter an integer to insert: 45
Add another node? (y/n): y
Enter an integer to insert: n
Invalid input. Exiting.
aaditya@fedora:~/experiments-in-c/exp10$

```

Q2. Write a program to insert item in middle of linked list.

Algorithm:

1-Start

2-Initialize head (assume list already exists or empty).

3-Read the value item to be inserted.

4-Count the number of nodes in the list using count_nodes():

5-Set count = 0

6- Traverse each node until NULL

7-Increment count for every node

8-Compute $\text{middle} = \text{count} / 2$

9-Create a new node with data = item and next = NULL.

10-If:

11-List is empty (head == NULL) → make new node the head.

12-middle == 0 → insert at beginning (new_node → next = head; head = new_node)

13-Otherwise:

Set a temporary pointer temp = head.

14-Move temp forward (middle - 1) times.

15-Insert by:

```
new node → next = temp → next
```

```
temp → next = new_node
```

16-Display the list.

17-End

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

/* Create new node */
Node* create_node(int value) {
    Node *new_node = (Node*)malloc(sizeof(Node));
    new_node->data = value;
    new_node->next = NULL;
    return new_node;
}

/* Insert at end */
void insert_end(Node **head_ref, int value) {
    Node *new_node = create_node(value);
    if (*head_ref == NULL) {
        *head_ref = new_node;
        return;
    }
    Node *temp = *head_ref;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = new_node;
}

/* Count nodes */
int count_nodes(Node *head) {
    int count = 0;
    while (head != NULL) {
        count++;
        head = head->next;
    }
    return count;
}

/* Insert in middle */
void insert_middle(Node **head_ref, int value) {
    int length = count_nodes(*head_ref);
    int middle = length / 2;

    Node *new_node = create_node(value);

    /* If list empty or middle = 0, insert at beginning */
    if (*head_ref == NULL || middle == 0) {
        new_node->next = *head_ref;
        *head_ref = new_node;
    }
}
```



```
        return;
    }

    Node *temp = *head_ref;
    for (int i = 1; i < middle; i++)
        temp = temp->next;

    new_node->next = temp->next;
    temp->next = new_node;
}

/* Display list */
void display(Node *head) {
    while (head != NULL) {
        printf("%d", head->data);
        if (head->next != NULL) printf(" -> ");
        head = head->next;
    }
    printf("\n");
}

int main() {
    Node *head = NULL;
    int n, value;

    printf("Enter number of nodes to create: ");
    scanf("%d", &n);

    printf("Enter %d values:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &value);
        insert_end(&head, value);
    }

    printf("Current list: ");
    display(head);

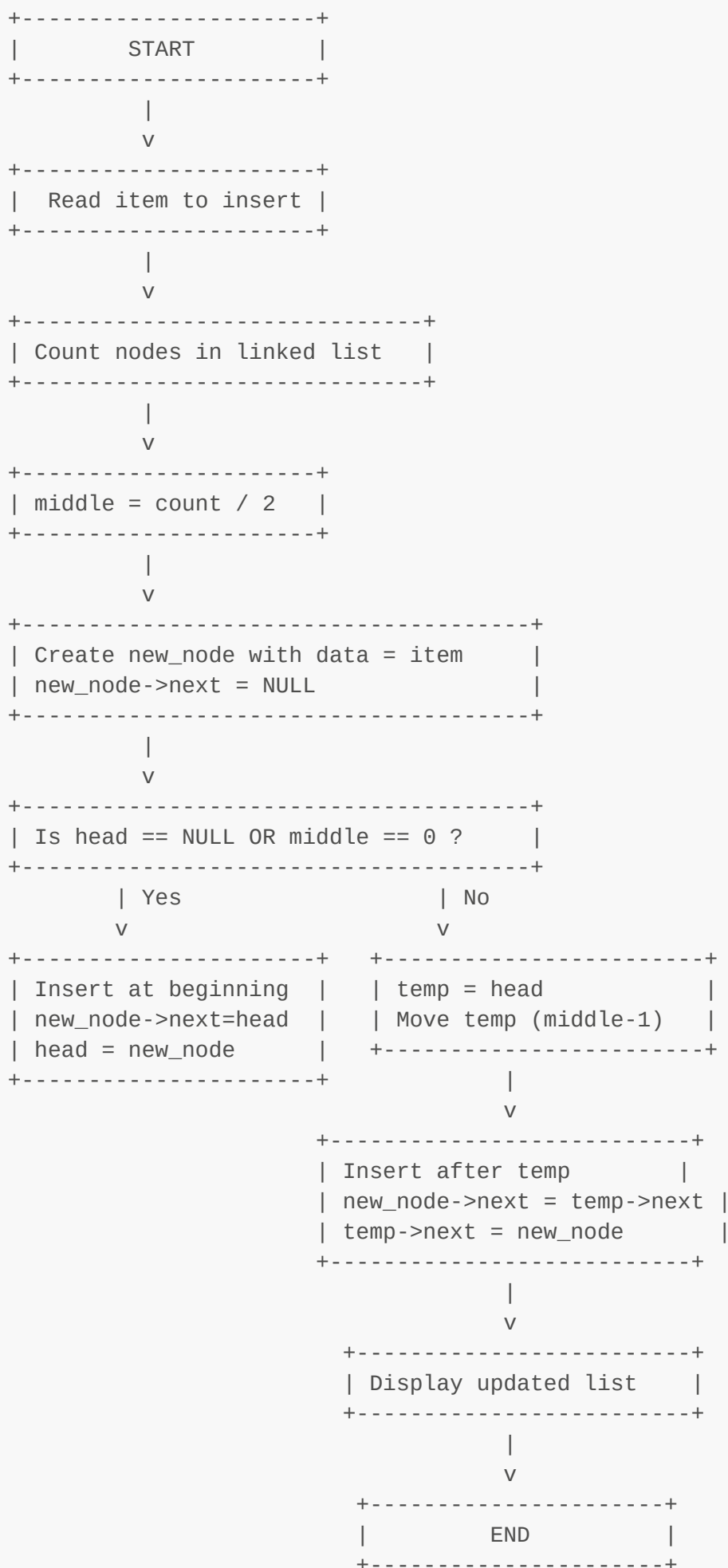
    printf("Enter value to insert in middle: ");
    scanf("%d", &value);

    insert_middle(&head, value);

    printf("List after inserting in the middle: ");
    display(head);

    return 0;
}
```

Flowchart:



Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Code - exp10 + v [ ] [ ] ... | [ ] [ ] X

cd "/home/aaditya/experiments-in-c/exp10/" && gcc exp10Q2.c -o exp10Q2 && "/home/aaditya/experiments-in-c/exp10/"exp10Q2
• aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp10/" && gcc exp10Q2.c -o exp10Q2 && "/home/aaditya/experiments-in-c/exp10/"exp10Q2
Enter number of nodes to create: 1 2 3 4 5
Enter 1 values:
Current list: 2
Enter value to insert in middle: List after inserting in the middle: 3 -> 2
• aaditya@fedora:~/experiments-in-c/exp10$ cd "/home/aaditya/experiments-in-c/exp10/" && gcc exp10Q2.c -o exp10Q2 && "/home/aaditya/experiments-in-c/exp10/"exp10Q2
Enter number of nodes to create: 2
Enter 2 values:
1 2
Current list: 1 -> 2
Enter value to insert in middle: 3
List after inserting in the middle: 1 -> 3 -> 2
• aaditya@fedora:~/experiments-in-c/exp10$
```

EXPERIMENT:11 BITWISE OPERATOR

Q1. Write a program to apply bitwise OR, AND and NOT operators on bit level.

Algorithm:

Step 1: Start Step 2: Declare two integers a and b Step 3: Read input values for a and b Step 4: Compute

- $AND = a \& b$
- $OR = a | b$
- $NOT = \sim a$ (bitwise NOT of a)

Step 5: Display results Step 6: Stop

Code:

```
#include <stdio.h>

int main() {
    int a, b;
    int and_result, or_result, not_result;

    printf("Enter value of a: ");
    scanf("%d", &a);

    printf("Enter value of b: ");
    scanf("%d", &b);

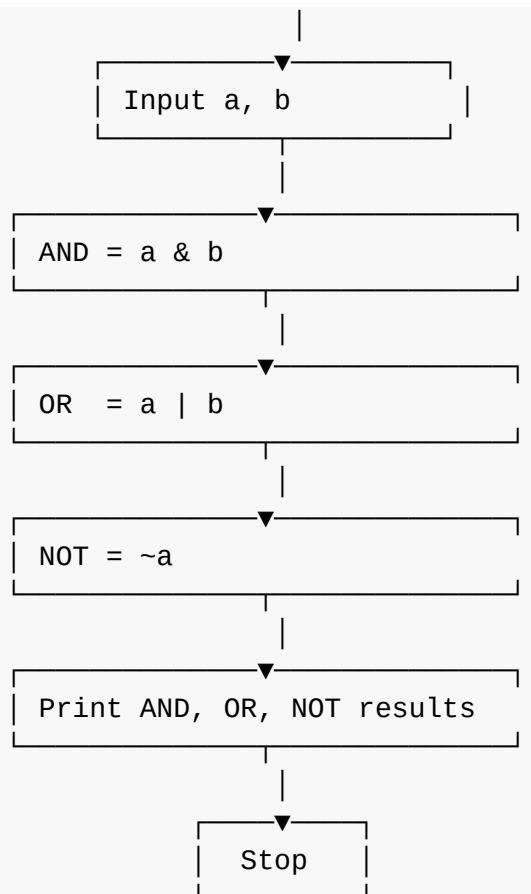
    // Bitwise operations
    and_result = a & b;    // AND
    or_result  = a | b;    // OR
    not_result = ~a;       // NOT (only on a)

    printf("\n---- Bitwise Results ----\n");
    printf("a & b (AND) = %d\n", and_result);
    printf("a | b (OR)  = %d\n", or_result);
    printf("~a (NOT)   = %d\n", not_result);

    return 0;
}
```

Flowchart:





Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp11/" && gcc exp11Q1.c -o exp11Q1 && "/home/aaditya/experiments-in-c/exp11/"exp11Q1
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp11/" && gcc exp11Q1.c -o exp11Q1 && "/home/aaditya/experiments-in-c/exp11/"exp11Q1
Enter value of a: 1
Enter value of b: 2

---- Bitwise Results ----
a & b (AND) = 0
a | b (OR) = 3
~a (NOT) = -2
aaditya@fedora:~/experiments-in-c/exp11$
```

Q2.write a program to apply left shift and right shift operator.

Algorithm:

Step 1: Start Step 2: Declare an integer variable num and shift amount shift Step 3: Read num and shift from the user Step 4: Compute

- left = num << shift
- right = num >> shift

Step 5: Display results Step 6: Stop

Code:

```
#include <stdio.h>

int main() {
    int num, shift;
    int left_result, right_result;

    printf("Enter a number: ");
    scanf("%d", &num);

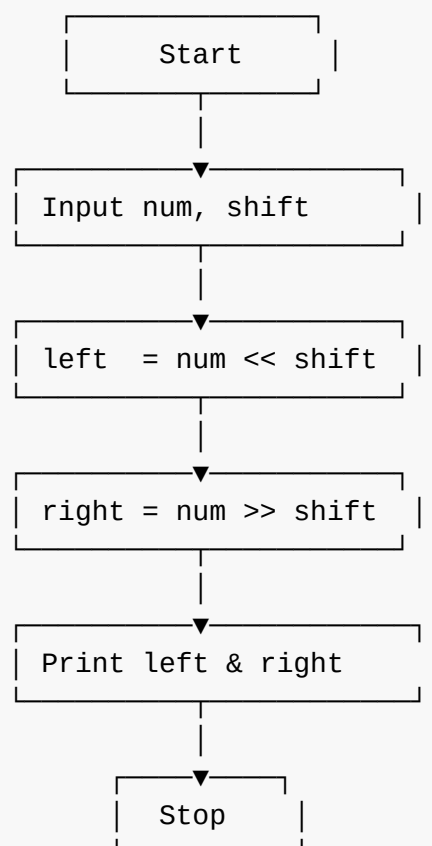
    printf("Enter shift amount: ");
    scanf("%d", &shift);

    // Perform shift operations
    left_result = num << shift;    // Left shift
    right_result = num >> shift;   // Right shift

    printf("\n--- Shift Results ---\n");
    printf("Left Shift (num << shift) = %d\n", left_result);
    printf("Right Shift (num >> shift) = %d\n", right_result);

    return 0;
}
```

Flowchart:




```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp11/" && gcc exp11Q2.c -o exp11Q2 && "/home/aaditya/experiments-in-c/exp11/"exp11Q2
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp11/" && gcc exp11Q2.c -o exp11Q2 && "/home/aaditya/experiments-in-c/exp11/"exp11Q2
Enter a number: 12
Enter shift amount: 5

--- Shift Results ---
Left Shift (num << shift) = 384
Right Shift (num >> shift) = 0
aaditya@fedora:~/experiments-in-c/exp11$
```